ETAS RTA-BSW v12.7.0

RTA-Base Stack Reference Guide

www.etas.com

Copyright

ETAS RTA-BSW 12.7.0 - RTA-Base Stack Reference Guide R01 EN – 05.2025

Contents

1 Default Error Tracer

Module name	Det
Package	RTA-BASE
AUTOSAR Module ID	15
AUTOSAR Version	23-11

1.1 Introduction

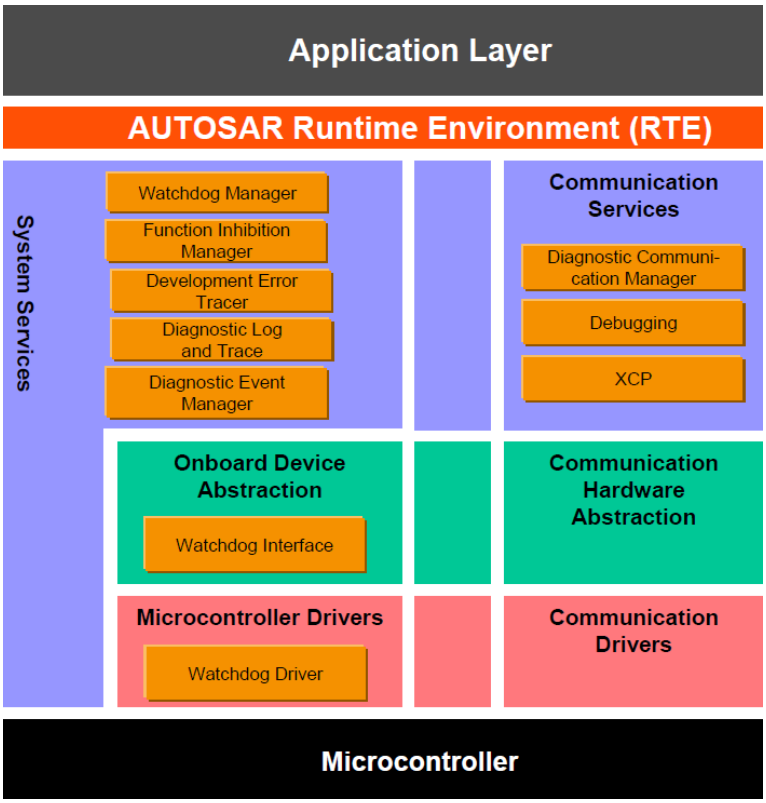


Figure 1.1.Error Services

The Default Error Tracer (Det) provides an API for the BSW and SW-Cs to report and trace errors. Although this can be done during development or runtime, in general the Det is used during the development phase for debugging a BSW module or SW-C.

Det is a system service, so it can be accessed by modules at all layers. It provides the following capabilities:

1. Receive a report of all errors detected during development or at runtime.
2. Provide a trace containing the AUTOSAR Module ID, Function ID, Instance ID and Error ID (The type of error depends on the API used).
3. Debugging and error tracing is supported through the use of error

hooks called for every notified error.

Module Configuration

This module is not currently supported by the RTA-BSW Configuration Generator (ConfGen) and must be manually configured.

1.2 Supported Features

1.2.1 Reporting Errors

There are three error classifications which can be reported, and these errors can be raised during both development and runtime.

The errors, and the API through which they are reported, are as follows:

- Development errors (Det_ReportError)
- Runtime errors (Det_ReportRuntimeError)
- Transient faults (Det_ReportTransientFault)

Only the Det_ReportError API traces the error reported.

The call of the Det_ReportError function will cause Det to store the given error information in a buffer, the size of which is configurable. If the buffer is full, it will not store additional information. The buffer can be configured to filter duplicates.

1.2.2 Error Hooks

Error hooks are used to forward error notifications, and these notifications depend on the project level configuration of the error hook (via DetNotification). All the configured error hooks are called when the Det_ReportError API is called.

1.2.3 Logging Errors

When the Dlt module is integrated into a project, and if DetForwardToDlt is set to TRUE, Det can use the Dlt module to log reported errors. It does this by calling Dlt_DetForwardErrorTrace. For further information about Dlt, please refer to the Dlt reference guide.

1.3 Extensions

1.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

1.3.1.1 Det/DetConfigSet/DetModule/DetRbModuleEcucPartitionRef

Short Name	DetRbModuleEcucPartitionRef

Description	Refers the Det module to one ECUC partition to limit the access to this Det module. The ECUC partition referenced is within the subset of the ECUC partition where the Det Module is mapped.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

1.3.1.2 Det/DetGeneral/DetRbBuffer

Short Name	DetRbBuffer
Description	Contains proprietary configuration parameters for the error entry buffer.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

1.3.1.3 Det/DetGeneral/DetRbBuffer/DetRbAtomicIncrementAvailable

Short Name	DetRbAtomicIncrementAvailable
Description	Parameter to indicate if Atomic Increment function is available in order to support multiple write operations. The parameter needs to be set to TRUE only if the Atomic Increment operation is available through rba_BswSrv.
Introduction	TRUE: AtomicIncrement operation is supported by the hardware, the buffer entry uses lockfree mechanism. FALSE: AtomicIncrement operation is not suppoted by the hardware, the buffer entry uses lock mechanism.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.4 Det/DetGeneral/DetRbBuffer/DetRbErrorBufferSize

Short Name	DetRbErrorBufferSize
Description	Parameter to set the size of the error buffer.
Introduction	If the buffer is enabled the min size is 1 and the max size is 4,294,967,296. Feasible range suggested is below 1000.
Multiplicity	1..1
Value Range	1..4294967296
Default Value	-

Type	ECUC-INTEGER-PARAM-DEF

1.3.1.5 Det/DetGeneral/DetRbBuffer/DetRbFilterDuplicateBufferEntries

Short Name	DetRbFilterDuplicateBufferEntries
Description	Parameter to filter duplicate error entires in Det buffer.
Introduction	TRUE: Duplicate error entries in the buffer are filtered. FALSE: Duplicate error entries in the buffer are not filtered.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.6 Det/DetGeneral/DetRbBuffer/DetRbForwardErrorEntries

Short Name	DetRbForwardErrorEntries
Description	Parameter to enable / disable the forwarding of the development error entry buffer i.e Det_ErrorEntryBuffer to rba_ArxmlGen .
Introduction	True: DetRbForwardErrorEntries enabled. False: DetRbForwardErrorEntries disabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.7 Det/DetGeneral/DetRbBuffer/DetRbUseErrorBufferApi

Short Name	DetRbUseErrorBufferApi
Description	Pre-processor switch to enable / disable the API for error buffer handling.
Introduction	TRUE: Buffer handler APIs enabled. FALSE: Buffer handler APIs disabled.
Multiplicity	1..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

1.3.1.8 Det/DetGeneral/DetRbEcucPartition

Short Name	DetRbEcucPartition
Description	Contains the ECUC Partition references and fifo size for each ECUC Partition.
Multiplicity	0..65535
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

1.3.1.9 Det/DetGeneral/DetRbEcucPartition/DetRbEcucPartitionFifoSize

Short Name	DetRbEcucPartitionFifoSize
Description	Parameter to set the size of the FIFO buffer for the configured partition. Note: The configuration values to be configured considering the size of FifoMsg which is of 5bytes + 1byte for msgTailPad which is gap to the next message. This addition bytes are needed for the bswsrvlib component to handle the lock free FIFO mechanism.
Multiplicity	1..1
Value Range	5..4294967296
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

1.3.1.10 Det/DetGeneral/DetRbEcucPartition/DetRbEcucPartitionRef

Short Name	DetRbEcucPartitionRef
Description	Maps the Det to one or multiple ECUC partitions to make its API's available in the respective partition
Multiplicity	1..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

1.3.1.11 Det/DetGeneral/DetRbMasterEcucPartitionRef

Short Name	DetRbMasterEcucPartitionRef
Description	Maps the Det master to zero or one ECUC partition to assign the master functionality to a certain core. The ECUC partition referenced is a subset of the ECUC partitions where the Det is mapped to.
Multiplicity	0..1
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

1.3.1.12 Det/DetGeneral/DetRbMaxReportsProcessingPerCycle

Short Name	DetRbMaxReportsProcessingPerCycle
Description	This parameter specifies number of satellite Det reports to be processed per main function call.
Multiplicity	0..1

Value Range	1..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

1.3.1.13 Det/DetGeneral/DetRbTaskTime

Short Name	DetRbTaskTime
Description	Allow to configure the time for the periodic cyclic task. Please note: This configuration value shall be equal to the value in the Basic Software Scheduler configuration of the RTE module.
Introduction	The AUTOSAR configuration standard is to use SI units, so this parameter is defined as float value in seconds. Det configuration tools must convert this float value to the appropriate value format for the use in the software implementation of Det. min: A negative value is not allowed. max: After Det error was reported, processing shall be completed within 100ms in order to have the error entry information updated as soon as possible. upperMultiplicity: If the Det supports multi partition then exactly one TaskTime must be specified per configuration. lowerMultiplicity: If the Det supports multi partition then exactly one TaskTime must be specified per configuration.
Multiplicity	0..1
Value Range	0..INF
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

1.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

1.3.2.1 Det_GetLastBufferIndex

Syntax	`Det_GetLastBufferIndex(Det_BufferIndexType* buffIdx)`
Parameters	`buffIdx` Pointer to store the index.
Description	Service to provide the index of the last stored error entry.

1.3.2.2 Det_GetBufferEntry

Syntax	`Det_GetBufferEntry(Det_BufferIndexType buffIdx, Det_ErrorEntryBufferType* buffEntry)`
Parameters	`buffIdx` Index in buffer for the error entry that shall be provided. `entry` Pointer to store the error entry information.
Description	Service to provide the error entry information for the given index.

1.4 Limitations

Version v12.7.0 of the Det module has the following limitations.

- The Det module stores a limited number of errors, and this number is dependent on the value set for `DetRbErrorBufferSize` configured in `DetRbBuffer`.

- The Det module must not be called recursively, but it does not contain any protection against this. You should consider if it is necessary to implement protection against recursion.

- The Det module can be called in ECU STARTUP I, which is before the OS is initialised. For this reason, the range of OS calls permitted in the error hooks will be limited when Det is called in ECU STARTUP I.

1.5 Exclusions

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- There are no known exclusions in this version.

1.6 Deviations

1.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

1.6.1.1 Det

[RTA-BSW] Description	Det configuration includes the functions to be called at notification. On one side the application functions are specified and in general it can be decided whether DIt shall be called at each call of Det.
[AUTOSAR] Description	Det configuration includes the functions to be called at notification. On one

1.7 API Definition

List of supported API

`Det_Init()`	Called by EcuM; initializes the Det module
`Det_Start()`	Triggers the Default Error Tracer if needed e.g. in case of completed NVRAM initialization for persistent error storage.
`Det_GetVersionInfo()`	Returns the version information of this module
`Det_ReportError()`	Reports Development errors
`Det_ReportRuntimeError()`	Reports Runtime errors
`Det_ReportTransientFault()`	Report Transient faults

For Further information please consult the AUTOSAR specification AUTOSAR_SWS_DefaultErrorTracer.pdf

1.8 Configuration Advice

> **NOTE**
>
> Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

- A stub version of the Det module will be provided through configuration generation in EcuM_EcucValues.arxml. This is a basic configuration in order to generate a version of Det that is capable of performing the operations you require. You will need to make your own project specific changes.
- Modules will only report errors through Det when the appropriate configuration parameter is set to TRUE (e.g. CanDevErrorDetection for CAN modules).
- EcuM will initialise Det when the `EcuMIncludeDet` parameter is set to TRUE.
- Extern defines for the Callout functions are provided in Det_Cfg.h. The callout function's operation will depend on the stub provided. The callout function in `DetReportRuntimeErrorCallout` is called in `Det_ReportRuntimeError()`, and the callout function in `DetReportTransientFaultCallout` is called in `Det_ReportTransientFault()`.
- If RTE is used, then the `DetRbCheckApiConsistency` parameter must be set to FALSE.

1.9 Integration Advice

The Det module supports the generation of a Det_Cfg_SWCD.arxml file, which generates a service software component and a client-server interface. This allows the creation of ports for the service component to connect to other component's ports. With the client-server interface, the software component can trigger functions to report errors.

1.9.1 Init Functions

The Det module is initialized by a call to Det_Init by EcuM.

1.9.2 DeInit Functions

Det has no DeInit functions.

1.9.3 Scheduled Functions

Det has no periodic functions.

1.9.4 Multi-Partition Handling

The purpose of a multi-partition setup is to extend the standard methods provided by the RTE in order to optimize runtime and to ensure Freedom From Interference (FFI). The Det module supports this use-case by splitting all internal RAM cells into EcucPartition-specific sections, queuing the Det reports into partition specific FIFOs and ensuring that the Det reports are processed only by the Det-Master-Partition. However, Det does not enforce or provide a technical protection mechanism.

The Det reports are not processed directly by the satellite partitions, and E_OK is always returned back directly as the Det reports usually either do not have a return value or are always return E_OK. (Exception: *Det_ReportTransientFault()*)

When integrating it is expected that a suitable MPU (memory-protection unit) setup will be provided in the system, and it is configured according to the requirements stated below. In addition, ALL the following integration steps shall be completed (this is the assumption of use in a safety-related context).

DetModule and SoftwareComponent Assignment:

The configured assignment of each Det Module to a dedicated EcucPartition shall correspond to the EcucPartition assignments of the Module that reports to the Det error.

This will ensure that all Det reports that are reported by the Det Module have access to the partition specific to the data structure. Or in the opposite way, this will ensure that all software parts executed from another EcucPartition do not have write-access to this partition specific data if not needed due to other reasons.

Master MainFunction Integration:

Det_MainFunction has to be scheduled in a suitable task period within the Det-Master-Partition.

Satellite InitFunction Integration:

Det generates an InitFunction for each EcucPartition in which the modules are assigned to. Therefore, it is necessary to call these functions during initialization within the related EcucPartition.

Lock Free-FIFOs:

The Det multi partition uses the lock free FIFO feature provided by the rba_B-

swSrvcomponent to handle the transfer of data from the satellite to the Master partition. Using this abstracts the actual handling and also the memory mapping of the FIFO buffers to Det. This is because Det uses the push and pop interfaces provided by the rba_BswSrv component to handle the FIFO buffers, and with the rba_BswSrv ensuring the FFI of the FIFO buffers.

Det forwards the configuration needed for the partition specific FIFO buffers to the rba_BswSrv. But the container parameter *rba_BswSrv_FifoConfig* has to be manually configured during integration to ensure successful forwarding of the FIFO configuration.

Error Hooks, Callouts and Dlt:

As all the Det reports are processed asynchronously from the Det-Master-Partition, the Error Hooks, Callouts and Dlt callouts will also be called from the Det-Master-Partition. Therefore the modules implementing these Error Hooks, Callouts and Dlt shall be placed in the same EcucPartition as the Det-Master-Partition.

1.9.4.1 Memory Mapping

Det code is located in the default code section. Det internal variables are either located in the saved RAM zone or in the default section of these items.

When the multi-partition feature is used then all allocations are in addition, which are split between Det-Master (MemMap-macros having a simple"Det") or the related EcucPartition (MemMap-macros starting with "Det_<EcucPartition>").

Memory Protection Setup

Det configures (or requires) for each used partition a dedicated set of MemMap sections. These MemMap sections have to be placed into memory-protected regions by the User. The protection setup shall fulfill following requirements:

- Read access shall be possible from the EcucPartition as a minimum and from the Det-Master-partition (there is no safety-issue if every EcucPartition has read access).
- Write access shall be possible from the EcucPartition as a minimum and from the Det-Master-partition. Making sure that code executed from the EcucPartitions has a lower safety-level than partitions that do not have write-access.
- The MPU shall perform a suitable response if an unauthorized write access was performed on the memory of the block in the EcucPartition. Suitable, in this context means that further code execution within the Det is stopped permanently until the next system/partition-reset.

1.9.4.2 Ecuc Assignment

In EcucPartitionCollection the partitions can be configured.

1.9.4.3 Det Ecuc Partition Assignment

To enable the multi-partition feature in Det, all the valid Det partitions in DetRbE-

cucPartitionhave to be configured. Each DetRbEcucPartition consists of mandatory parameters *DetRbEcucPartitionRef* and *DetRbEcucPartitionFifoSize*. The parameter *DetRbEcucPartitionRef* under the container DetRbEcucPartition should be referenced to one ECUC Partition. *DetRbEcucPartitionRef* is a (sub-)set of EcucPartitionCollection.

1.9.4.4 Master Partition Assignment

The master partition is set using DetRbMasterEcucPartitionRef.

The configured reference of DemRbMasterEcucPartitionRef needs to be a valid subset of DetRbEcucPartition.

1.9.4.5 Satellite Det Report Process

All the stored Det reports in the FIFO buffers of all the partitions will be fetched and processed in the Det main function. The maximum number of reports to be processed in the Det main function is specified by the configured value in the parameter *DetRbMaxReportsProcessingPerCycle*.

IMPORTANT CONSTRAINT: All the Det reports (ReportError, ReportRuntimeError and ReportTransientFault) which are done from the DetModule(s), and are assigned to a given Det Satellite partition in DetRbModuleEcucPartitionRef shall always be done from the same Os Task.

The current implementation uses a queuing mechanism to queue the reports in the satellite, which are then processed asynchronously by the master. The queuing mechanism implemented is optimized for multi-ASIL (lock-free) handling and therefore does not support reentrant calls to push the reports to the queue.

Example 1:

If “DetModule_1” is mapped to the Det satellite “Det_PartitionA” using DetRbModuleEcucPartitionRef, then all the Det reports for the “DetModule_1” shall always be done from the same Os Task context.

When this constraint cannot be ensured, then the “DetModule_1” shall not be assigned to the Det satellite “Det_PartitionA”. It shall be either explicitly assigned to the Master partition “Det_MasterPartition” or when DetRbModuleEcucPartitionRef has not been configured.

Example 2:

If “DetModule_1” and “DetModule_2” are mapped to the Det satellite “Det_PartitionA” using DetRbModuleEcucPartitionRef, then all the Det reports for the “DetModule_1” and “DetModule_2” shall always be done from the same Os Task context.

When this constraint cannot be ensured, then either both or one of “DetModule_1” and “DetModule_2” shall not be assigned to the Det satellite “Det_PartitionA”. It shall be either explicitly assigned to the Master partition “Det_MasterPartition” or when DetRbModuleEcucPartitionRef has not been configured.

1.9.4.6 Deviation from Autosar

The ReportTransientfault callout can also return E_NOT_OK ,since the logical or

(sum) of the callout return values shall be returned. When used in the Multipartition support the satellites reports will always return E_OK.

When Multi-Partition is enabled, then the Error Hook calls and the callouts to the application and to the Dlt are not called synchronously. These are called asynchronously from the Det-Master-Partition while processing the FIFOs filled by the Satellite partitions.

1.9.4.7 Implementation Note

All Satellite reports will return E_OK irresepective of those reports that are stored in FIFO buffer or they are processed.

If FIFO buffer is full, the Det report will not be stored in the FIFO buffer, but the return value will always return E_OK as this is unavoidable as per Autosar specification.

1.9.4.8 Det Multi-Partition Integration Hints

Each Ecuc partition configured in DetRbEcucPartition must be referenced to at least one of the Det modules. Each DetRbEcucPartition should be configured with mandatory parameters

- **DetRbMasterEcucPartitionRef:** This parameter should be referenced to one of the Ecuc Partition listed in EcucPartitionCollection.
- **DetRbEcucPartitionFifoSize:** FIFO buffer size should be configured considering the size of the FifoMsg, which is of 5bytes + 1byte for msgTailPad which is gap to the next message. The additional bytes are needed for the bswsrvlib component to handle the lock free FIFO mechanism.
    - The size of this parameter is pre-configured and this value will be forwarded by the Det to rba_BswSrv for further FIFO buffer size calculation in the rba_BswSrv.
    - Ideally to store a single error report , total of 11bytes is to be considered and configure the FIFO size accordingly.
    - Note: The size of FIFO buffer rba_BswSrv might also vary depending on their implementation in the future.
- DetRbMasterEcucPartitionRef must be configured one of the DetRbEcucPartition.

DetRbTaskTime:

- Please note: This configuration value shall be equal to the value in the Basic Software Scheduler configuration of the RTE module. The AUTOSAR configuration standard is to use SI units, so this parameter is defined as a float value in seconds. Det configuration tools must convert this float value to the appropriate value format for the use in the software implementation of Det.
- A negative value is not allowed.
- After a Det error was reported, processing shall be completed within 100ms in order to have the error entry information updated as soon as possible.
- If the Det supports multi partition then exactly one TaskTime must be speci-

fiedper configuration.

As *DetRbMaxReportsProcessingPerCycle* specifies the number of Satellite Det reports to be processed in one main function. This parameter should be configured with value of less than 255 and greater than 0.

Configuration shall be done in such a way that the FIFO buffers have enough size to store multiple reports at the same time, and also the Det_MainFunction is scheduled in such a way that the reports stored in the FIFOs are processed as soon as possible by the Det-Master-Partition.

1.9.4.9 Api Call Hints

Det_MainFunction shall be triggered as per the periodic cycle time configured in the DetRbTaskTime

Satellite partition Push Init functions trigger must be triggered and FIFO should be initialised before triggering any Satellite partition reports. Refer to the generated Satellite partition header file for the push Init functions for each configured Det Satellite partition.

1.9.5 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The Det module uses the following OS resources:

- Det_Monitoring

1.10 Hardware Requirements

Table 1.19.Det Hardware Requirements

Support	Usage
FLASH	327 bytes
RAM	609 bytes

Usage calculated based on a sample configuration.

2 ECU State Manager

Module name	EcuM
Package	RTA-BASE
AUTOSAR Module ID	010
AUTOSAR Version	22-11

2.1 Introduction

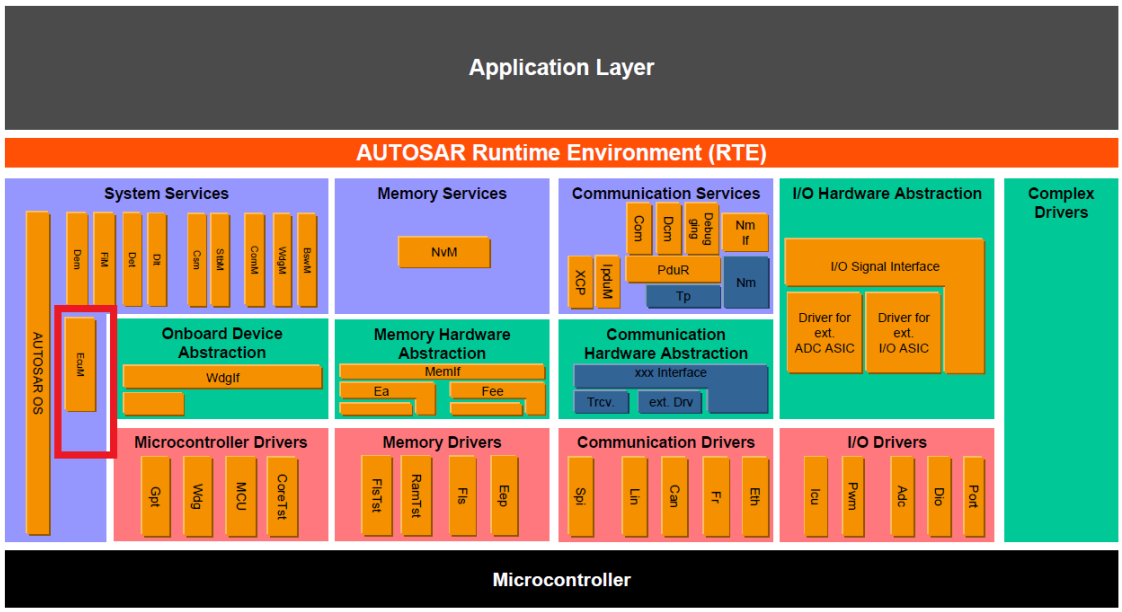


Figure 2.1.EcuM Architecture

The ECU State Manager module (EcuM) manages common aspects of ECU states, and is specifically responsible for:

- Initialising and de-initialising the OS, SchM, BswM, MCAL drivers and some basic software driver modules.
- Configuring the ECU for SLEEP and SHUTDOWN, when requested.
- Managing all wakeup events on the ECU, and providing the wakeup validation protocol to distinguish real wakeup events from incorrect ones.

The main supported phases of the EcuM module are:

- STARTUP
- UP
- SLEEP
- WAKEUP

- SHUTDOWN

There are two variants of ECU management: Flexible and Fixed, but only the Flexible EcuM is supported in version v12.7.0. See the Supported Features section for more details.

Module Configuration

This module is not currently supported by the RTA-BSW Configuration Generator (ConfGen) and must be manually configured.

2.2 Supported Features

2.2.1 Flexible EcuM

Although there are two variants of AUTOSAR ECU management, Flexible EcuM and Fixed EcuM, the RTA-BSW implementation of EcuM currently only supports the Flexible EcuM.

With Flexible EcuM, most ECU states are not implemented within EcuM itself. In general, EcuM is only active when the generic mode management facilities are unavailable in:

- Early STARTUP phases
- Late SHUTDOWN phases
- SLEEP phases where the facilities are locked out by the scheduler.

The Flexible EcuM provides the following features:

- Partial or fast startup, where the ECU starts up with limited capabilities and then later, as determined by the application, continues the startup step-by-step.
- Multiple operational states, where the ECU has more than one RUN state.
- In a Multi-Core ECU environment, STARTUP, SHUTDOWN, SLEEP and WAKEUP are coordinated on all cores of the ECU.

2.2.2 Startup Phase

The Startup phase is initiated through EcuM_Init. During this phase, EcuM will control the ECU startup procedure to the point when Generic Mode Management facilities (typically provided by BswM) are available. These actions include:

- Calling EcuM_AL_DriverInitZero and EcuM_AL_DriverInitOne to initialise BSW modules and drivers that are required to start the OS.
- Calling StartOS to initialise the OS.
- Calling SchM_Start_Integration to initialise the SchM if applicable. See the Integration Advice for more information.
- Calling BswM_Init to start the BswM.
- Calling SchM_Init to initialise the SchM.

- Calling SchM_StartTiming_Integration to initialise the SchM if applicable. See the Integration Advice for more information.

The EcuMRbStartupDuration extension parameter can be used to enable/disable the measurement of EcuM_Init duration.

2.2.2.1 Startup I

When EcuM_Init is called, EcuM enters Startup I phase. Basic software modules without Post-Build configuration data can be initialized, followed by the modules with Post-Build configuration.

Minimal initialization is done during EcuM_Init so that the OS can be started as quickly as possible.

At the end of the Startup I phase, control goes from EcuM to OS. The initialization of other BSW modules will be dependent on their configuration. If multi-core is enabled, all the slave cores will be started from the master core's EcuM_Init shortly before the StartOs.

2.2.2.2 Startup II

EcuM enters into Startup II phase when the EcuM_StartupTwo function is called.

The call to EcuM_StartupTwo must be implemented in an OS task, which should be started automatically after StartOs so that EcuM will gain back the control.

In the Startup II phase, the Schedule Manager (Schm) and BSW Mode Manager (BswM) will get initialized.

In a Multi-Core environment, EcuM_StartupTwo must be called from all cores, and only the Schm is initialized from the slave cores.

2.2.3 Up Phase

The Up phase is started once BswM_Init has been called. EcuM_MainFunction is called regularly and performs the following actions:

- Checks whether EcuMGoDown (or EcuM_GoDownHaltPoll) has been called, and initiates the Shutdown phase if necessary.
- Arbitrates the RUN and POST_RUN requests and releases.

2.2.4 Sleep Modes (Halt and Poll)

EcuM provides a configurable set of sleep modes that offer a trade-off between power consumption and the time required to restart the ECU.

- Halt mode: No code is executed, but power is still supplied.
- Polling mode: Minimal code is executed in a reduced clock cycle.

These two modes are implemented through EcuM_GoHalt and EcuM_GoPoll (described below) and the ECU is wakeable in both the modes, if configured accordingly.

Note: In the Autosar specification, the Sleep operation is synchronous, but in

order to achieve the core synchronization (see Multi-core below), the Sleep operation implemented as asynchronous.

2.2.4.1 EcuM_GoHalt (Sleep Halt)

When EcuM_GoHalt is selected as the shutdown target, the MCU enters the selected low power mode. In this mode, the ECU Manager does not execute any code, but OS is not shut down. Upon detection of a wakeup source (see "Wakeup Handling" below), EcuM regains control and exits the Sleep phase, and configured drivers are re-initialized.

EcuM_GoHalt requires that the MCU supports low power modes, and the Sleep-mode container has to be configured with sleep modes mapped to MCU low power modes that halt the code execution (i.e. EcuMSleepModeSuspend is TRUE). Halt sleep modes must have WakeupSourceMasks that are only mapped to halting wakeupsources (i.e. with EcuMWakeupSourcePolling set to FALSE).

2.2.4.2 EcuM_GoPoll (Sleep Poll)

When EcuM_GoPoll is selected as the shutdown target, the MCU enters the selected low power consumption of the MCU, but the ECU Manager module still executes code. Upon detection of a wakeup source (see "Wakeup Handling" below), EcuM regains control and exits the Sleep phase, and configured drivers are re-initialized.

EcuM_GoPoll requires that the MCU supports low power modes, and the Sleep-mode container in the EcuM configuration must have the appropriate sleep modes mapped to MCU low power modes.

2.2.4.3 Wakeup Handling

If a wakeup event (e.g. toggling a wakeup line, communication on a CAN bus etc.) occurs while the ECU is in Sleep Halt or Poll, then EcuM will regain control and exit the SLEEP phase by executing the WakeupRestart sequence.

EcuM wakes up the ECU in response to wakeup events, which are reported to it via EcuM_SetWakeupEvent. For each state change of a wakeup source, EcuM issues a mode change indication to BswM via BswM_EcuM_CurrentWakeup.

Some wakeup events can be generated unintentionally (e.g. EVM spike on a CAN line), so EcuM provides a protocol to validate wakeup events before the ECU resumes its full operation.

First, the validity of the received message is confirmed from the interrupt source. However, even if a new wakeup source is detected, EcuM won't immediately take it as a valid source, and it then starts a wakeup validation (done in EcuM_MainFunction).

EcuM will only invoke wakeup validation where EcuMValidationTimeout for a wakeup source is configured with a non-zero value.

Once the validation is successful (EcuM_ValidateWakeupEvent) EcuM will consider it as an intended wakeup source.

If a validated wakeup source is configured as a communication channel (ComM), then this will be reported to ComM (with the channel Id) via the ComM_E-

cuM_WakeUpIndication.Only wakeup sources related to communication channels get validated in the UP phase.

Note that EcuM only indicates the state changes of the Wakeup sources to BswM and ComM - it does not change the Communication channel modes. It is up to you to configure rules in BswM so that the ECU reacts correctly to the wakeup events, as this reaction depends fully on the current ECU (not ECU Management) state.

2.2.4.4 Wakeup Handling (Asynchronous Check)

If the check of the Wakeup Source is done asynchronously (e.g. asynchronous CanTrcv Driver), the EcuMCheckWakeupTimeout parameter can be used to configure a CheckWakeupTimer to delay a shutdown of the ECU.

When resumed by an asynchronous WakeupSource, the WakeRestart sequence has to be executed to re-start the mainfunctions.

EcuM waits for the duration of the CheckWakeupTimer until the check of the wakeup is finished with a positive result (EcuM_SetWakeupEvent is called for that wakeup).

If EcuM_SetWakeupEvent is called for that wakeup, the timer is cancelled and the sleep phase is exited.

If the CheckWakeupTimer expires before EcuM_SetWakeupEvent is called, the check of this wakeup source is finished and the system re-enters the sleep phase.

If EcuMCheckWakeupTimeout is less than or equal to 0 the call of EcuM_StartCheckWakeup is ignored.

2.2.5 Shutdown Phase

Before starting the shutdown phase, a call to EcuM_SelectShutdownTarget is required to identify the shutdown target. The shutdown targets supported in version v12.7.0 are:

- `ECUM_SHUTDOWN_TARGET_RESET`
- `ECUM_SHUTDOWN_TARGET_OFF`
- `ECUM_SHUTDOWN_TARGET_SLEEP`

The shutdown phase is then initiated when EcuM_GoDown (or EcuM_GoDownHaltPoll) is called. During this phase, the basic software modules will be shutdown with the selected shutdown target. These actions include:

- Calling BswM_Deinit to de-initialise the BswM.
- Calling SchM_Deinit to de-initialise the SchM.
- Calling ShutdownOS to stop the OS.
- Calling either EcuM_AL_SwitchOff or EcuM_AL_Reset to reach the shutdown target.

The AUTOSAR specification states that EcuM should call ShutdownOS with void,

but according to the OSEK specification, ShutdownOS and ShutdownAllCores accept an error id as a parameter. For this reason, the RTA-BSW implementation of EcuM calls ShutdownOS and ShutdownAllCores with a E_OS_SYS_NO_RESTART parameter.

Once the ECU is ready for shutdown, it can be initiated by EcuM_GoDown, or by EcuM_GoDownHaltPoll, which combines the functionality of EcuM_GoDown, EcuM_GoHalt and Ecum_GoPoll.

- When the shutdown target is OFF or RESET, EcuM_GoDownHaltPoll achieves the functionality of EcuM_GoDown.
- If the shutdown target is SLEEP, EcuM_GoDownHaltPoll achieves the functionality of EcuM_GoHalt or Ecum_GoPoll, based on whether the Sleep mode parameter is Suspend or not.

In order to achieve backward compatibility, EcuM_GoDown, EcuM_GoHalt and Ecum_GoPoll are still supported in their own right. Only valid users (configured in EcuMGoDownAllowedUsers) can call EcuM_GoDown or EcuM_GoDownHaltPoll.

The shutdown activities of EcuM are divided into 2 sequences: OffPreOs and OffPostOs.

2.2.5.1 OffPreOS

In the OffPreOs sequence, control is transferred to EcuM from BswM, and EcuM proceeds with the shutdown. In this sequence, as described above, EcuM will de-initialize the BswM and SchM modules, and, as a final activity, shutdown the OS.

In a Multi-core environment, the EcuM_GoDown (or EcuM_GoDownHaltPoll) running on the master core will initiate the shutdown synchronization of all cores. The EcuM_GoDown/EcuM_GoDownHaltPoll from any core will set a shutdown readiness flag in all slave cores, or it will trigger the shutdown of all cores. For a complete ECU shutdown, the EcuM on the Master core will wait until all the cores are shutdown, or for a pre-defined period. If the timer for that period expires, the Master EcuM will proceed with a forced shutdown of all cores.

The default value of the slave core timeout counter is 100 ms. If EcuM_MainFunction is not scheduled at every core, then the shutdown is delayed at least by 100ms, and SchM_DeInit is not called at the slave core(s).

2.2.5.2 OffPostOS

The shutdown hook calls EcuM_Shutdown to terminate the shutdown process, and this will initiate the OffPostOs sequence.

EcuM_Shutdown will not return, but will either switch off the ECU or issue a reset, depending on the selected shutdown target and reset mode. This is executed only in the EcuM on the Master core. An ISR may be invoked to handle the wakeup event, but this depends on the hardware and the driver implementation.

2.2.6 EcuM_GetLastShutdownTarget

The EcuM_GetLastShutdownTarget API returns the shutdown target and mode

of the previous power cycle shutdown. This shutdown information is stored in a dedicated block in NvM for EcuM.

See the Integration Advice section for more details on EcuM_GetLastShutdownTarget.

2.2.7 EcuM_Rb_GetLastShutdownInfo

The RTA-BSW implementation of EcuM provides the EcuM_Rb_GetLastShutdownInfo API to measure the time taken for the initialisation of drivers configured in the EcuMDriverInitListOne for each core, as well as the time taken for the system shutdown. This information is stored in the same dedicated block in NvM as previously described for EcuM_GetLastShutdownTarget.

See the Integration Advice section for more details on EcuM_Rb_GetLastShutdownInfo.

2.2.8 Multi-core

The EcuM supports multi-core ECUs via a master/slave configuration, with the master EcuM being responsible for starting and stopping all the slaves. In this configuration, one instance of the EcuM must be run on each of the physical cores. This allows the Startup, Up and Shutdown phases to be managed and synchronised across all cores.

Additional features:

- Multicore support for Run Time Measurement of the initialisation of drivers configured in EcuMDriverInitListOne. (This was previously only present in the startup core).
- Multicore support for Core-Specific Initialization in both EcuMDriverInitListOne and EcuMDriverInitListZero, by selecting the core via EcuMEcucCoreDefinitionRef for each EcuMDriverInitItem.
- Initialization of BSW modules from different cores (applicable only to EcuMDriverInitListOne).
- Synch Point functions are provided (within the EcuMDriverInitListOne Container) which enable the setting of Synch Points across multiple cores. See the Configuration Advice section for more details.

2.3 Extensions

2.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

2.3.1.1 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMDefaultShutdownTarget/EcuMDefaultState

Short Name	EcuMDefaultState

Description	This parameter describes the state part of the default shutdown target selected when the ECU comes out of reset. That is OFF or RESET. If EcuMStateReset is selected, the parameter EcuMDefaultResetModeRef selects the specific Reset mode.
Introduction	This parameter describes the state part of the default shutdown target selected when the ECU comes out of reset. That is OFF , RESET or SLEEP .If EcuMStateSleep is selected, the parameter EcuMDefaultSleepModeRef selects the specific sleep mode that is corresponding MCU mode. If EcuMStateReset is selected, the parameter EcuMDefaultResetModeRef selects the specific Reset mode.
Multiplicity	1..1
Default Value	-
Enumeration Literals	EcuMStateOff, EcuMStateReset, EcuMStateSleep
Type	ECUC-ENUMERATION-PARAM-DEF

2.3.1.2 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMDriverInitListOne/EcuMDriverInitItem/EcuMModuleID

Short Name	EcuMModuleID
Description	Short name of the module to be initialized, e.g. Mcu, Gpt etc. If both EcuMModuleRef and EcuMModuleId parameters are configured EcuMModuleRef will be discarded by code generators.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.3 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMDriverInitListOne/EcuMDriverInitItem/EcuMRbMonitoringCapable

Short Name	EcuMRbMonitoringCapable
Description	When EcuMRbMonitoringCapable parameter is set to TRUE for a PB capable driver module with valid post-build pointer, that particular driver module will be considered for Monitoring service generation. If it is set to FALSE or not configured then that particular module will not be considered for the monitoring service generation. Also if EcuMRbMonitoringCapable parameter set to TRUE for a module which is not having valid postbuild pointer(VOID/NULL_PTR) then that module will not be considered for monitoring service.

Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.4 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMDriverInitListOne/EcuMDriverInitItem/EcuMRbSequenceID

Short Name	EcuMRbSequenceID
Description	This ID determines the sequence in which the configured callouts appear. If all driver list items are provided with a unique ID, generation of callouts is done based on this ID. If some sequence IDs are duplicate or some sequence IDs are missing, error is thrown. If ID is not configured for any of the EcuMDriverInitItems, then the callouts are generated based on the order in which they are configured.
Multiplicity	0..1
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.5 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMDriverInitListZero/EcuMDriverInitItem/EcuMModuleID

Short Name	EcuMModuleID
Description	Short name of the module to be initialized, e.g. Mcu, Gpt etc. If both EcuMModuleRef and EcuMModuleId parameters are configured EcuMModuleRef will be discarded by code generators.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.6 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMDriverInitListZero/EcuMDriverInitItem/EcuMRbSequenceID

Short Name	EcuMRbSequenceID
Description	This ID determines the sequence in which the configured callouts appear. If all driver list items are provided with a unique ID, generation of callouts is done based on this ID. If some sequence IDs are duplicate or some sequence IDs are missing, error is thrown. If ID is not configured for any of the EcuMDriverInitItems, then the callouts are generated based on the order in which they are configured.

Multiplicity	0..1
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.7 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMDriverRestartList/EcuMDriverInitItem/EcuMModuleID

Short Name	EcuMModuleID
Description	Short name of the module to be initialized, e.g. Mcu, Gpt etc. If both EcuMModuleRef and EcuMModuleId parameters are configured EcuMModuleRef will be discarded by code generators.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.8 EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMDriverInitListBswM/EcuMDriverInitItem/EcuMModuleID

Short Name	EcuMModuleID
Description	Short name of the module to be initialized, e.g. Mcu, Gpt etc. If both EcuMModuleRef and EcuMModuleId parameters are configured EcuMModuleRef will be discarded by code generators.
Multiplicity	0..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.9 EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMDriverInitListBswM/EcuMDriverInitItem/EcuMRbSequenceID

Short Name	EcuMRbSequenceID
Description	This ID determines the sequence in which the configured callouts appear. If all driver list items are provided with a unique ID, generation of callouts is done based on this ID. If some sequence IDs are duplicate or some sequence IDs are missing, error is thrown. If ID is not configured for any of the EcuMDriverInitItems, then the callouts are generated based on the order in which they are configured.
Multiplicity	0..1
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.10 EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMFlexUserConfig/EcuMRbStateRequestQueueLength

Short Name	EcuMRbStateRequestQueueLength
Long Name	Queue length for EcuMStateRequest Operations
Description	This container contains the Queue Length parameters of different operations in EcuMStateRequest port instances
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.11 EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMFlexUserConfig/EcuMRbStateRequestQueueLength/EcuMRbReleasePOSTRUNQueueLength

Short Name	EcuMRbReleasePOSTRUNQueueLength
Description	This element specifies queue length of ReleasePOSTRUN operation for EcuMStateRequest , the length of the ServercomSpec (which is generated by the RTE) on the provider port. Setting value of 1 implies that incoming requests are rejected while another request that arrived earlier is being processed
Multiplicity	0..1
Value Range	1..255
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.12 EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMFlexUserConfig/EcuMRbStateRequestQueueLength/EcuMRbReleaseRUNQueueLength

Short Name	EcuMRbReleaseRUNQueueLength
Description	This element specifies queue length of ReleaseRUN operation for EcuMStateRequest , the length of the ServercomSpec (which is generated by the RTE) on the provider port. Setting value of 1 implies that incoming requests are rejected while another request that arrived earlier is being processed
Multiplicity	0..1
Value Range	1..255
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.13 EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMFlexUserConfig/EcuMRbStateRequestQueueLength/EcuMRbRequestPOSTRUNQueueLength

Short Name	EcuMRbRequestPOSTRUNQueueLength
Description	This element specifies queue length of RequestPOSTRUN operation for EcuMStateRequest , the length of the ServercomSpec (which is generated by the RTE) on the provider port. Setting value of 1 implies that incoming requests are rejected while another request that arrived earlier is being processed
Multiplicity	0..1
Value Range	1..255
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.14 EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMFlexUserConfig/EcuMRbStateRequestQueueLength/EcuMRbRequestRUNQueueLength

Short Name	EcuMRbRequestRUNQueueLength
Description	This element specifies queue length of RequestRUN operation for EcuMStateRequest , the length of the ServercomSpec (which is generated by the RTE) on the provider port. Setting value of 1 implies that incoming requests are rejected while another request that arrived earlier is being processed
Multiplicity	0..1
Value Range	1..255
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.15 EcuM/EcuMGeneral/EcuMIncludeDet

Short Name	EcuMIncludeDet
Description	If defined, the according BSW module will be initialized by the ECU State Manager
Multiplicity	1..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.16 EcuM/EcuMGeneral/EcuMRbAISwitchOffCallout

Short Name	EcuMRbAISwitchOffCallout
Description	Container used for defining EcuM_AL_SwitchOff callout. If this is not configured integratopr should manually call the functions inside EcuM_Callout_Stubs.c.
Introduction	This container holds a list of function call that will be called on EcuMAL_SwitchOFF Callout.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.17 EcuM/EcuMGeneral/EcuMRbAISwitchOffCallout/EcuMRbCallOutItem

Short Name	EcuMRbCallOutItem
Description	Container used for defining EcuM callout function calls.
Multiplicity	1..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.18 EcuM/EcuMGeneral/EcuMRbAISwitchOffCallout/EcuMRbCallOutItem/EcuMRbCalloutCondition

Short Name	EcuMRbCalloutCondition
Description	Condition needed to be checked before function calls. Note: This condition check is only possible with Run time. Pre-Compile time check is not possible.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.19 EcuM/EcuMGeneral/EcuMRbAISwitchOffCallout/EcuMRbCallOutItem/EcuMRbCalloutCondition/EcuMRbCalloutHeaderList

Short Name	EcuMRbCalloutHeaderList
Description	Header needed to be included for condition check.
Multiplicity	1..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.20 EcuM/EcuMGeneral/EcuMRbAISwitchOffCallout/EcuMRbCallOutItem/EcuMRbCalloutCondition/EcuMRbCalloutHeaderList/EcuMRbCalloutConditionHeader

Short Name	EcuMRbCalloutConditionHeader
Description	Header to be included.
Multiplicity	1..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.21 EcuM/EcuMGeneral/EcuMRbAISwitchOffCallout/EcuMRbCallOutItem/EcuMRbCalloutCondition/EcuMRbCalloutHeaderList/EcuMRbServiceIsNonAutosar

Short Name	EcuMRbServiceIsNonAutosar
Description	Flag, which is set true, if the module is Non-AUTOSAR specified. By default if this parameter is not configured the value is taken as FALSE. if the parameter is FALSE, then for the header inclused by EcuMRbModuleID will be followed by a mdule version check.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.22 EcuM/EcuMGeneral/EcuMRbAISwitchOffCallout/EcuMRbCallOutItem/EcuMRbCalloutCondition/EcuMRbCondition

Short Name	EcuMRbCondition
Description	If there is any condition needs to be checked before calling the function then condition can be added in this parameter. Note: This condition check is only possible with Run time. Pre-Compile time check is not possible.
Multiplicity	1..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.23 EcuM/EcuMGeneral/EcuMRbAISwitchOffCallout/EcuMRbCallOutItem/EcuMRbCalloutFunction

Short Name	EcuMRbCalloutFunction

Description	Function to be called from the callout. Both void and non-void functions are allowed. Function calls with parameters can be passed directly as '$Function_Name($parameter)' with no restriction on the number of parameters. Function calls without parameters can be passed as '$Function_Name'.
Multiplicity	1..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.24 EcuM/EcuMGeneral/EcuMRbAISwitchOffCallout/EcuMRbCallOutItem/EcuMRbModuleID

Short Name	EcuMRbModuleID
Description	Module name of the function call. This is needed for the header inclusion needed for EcuMRbCalloutFunction configured. Example : NvM
Multiplicity	1..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.25 EcuM/EcuMGeneral/EcuMRbAISwitchOffCallout/EcuMRbCallOutItem/EcuMRbSequenceID

Short Name	EcuMRbSequenceID
Description	This ID determines the sequence in which the configured callouts appear. If all driver list items are provided with a unique ID, generation of callouts is done based on this ID. If some sequence IDs are duplicate or some sequence IDs are missing, error is thrown. If ID is not configured for any of the EcuMRbCallOutItems, then the callouts are generated based on the order in which they are configured.
Multiplicity	0..1
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.26 EcuM/EcuMGeneral/EcuMRbAISwitchOffCallout/EcuMRbCallOutItem/EcuMRbServiceIsNonAutosar

Short Name	EcuMRbServiceIsNonAutosar

Description	Flag, which is set true, if the module is Non-AUTOSAR specified. By default if this parameter is not configured the value is taken as FALSE. if the parameter is FALSE, then for the header inclused by EcuMRbModuleID will be followed by a mdule version check.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.27 EcuM/EcuMGeneral/EcuMRbConfigCCASInit

Short Name	EcuMRbConfigCCASInit
Description	If this parameter is set as TRUE,then the Autosar conventional Startup and Shutdown of all cores and OS will be disabled.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.28 EcuM/EcuMGeneral/EcuMRbCubecCheckReference

Short Name	EcuMRbCubecCheckReference
Description	Hidden Reference parameter for the validation of CUBEC Compatibility Switches
Multiplicity	0..0
Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

2.3.1.29 EcuM/EcuMGeneral/EcuMRbCubecCompCheckInteger

Short Name	EcuMRbCubecCompCheckInteger
Description	Hidden Integer parameter for the validation of CUBEC Compatibility Switches
Multiplicity	0..0
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.30 EcuM/EcuMGeneral/EcuMRbCubecCompCheckString

Short Name	EcuMRbCubecCompCheckString
Description	Hidden String parameter for the validation of CUBEC Compatibility Switches
Multiplicity	0..0

Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.31 EcuM/EcuMGeneral/EcuMRbDeterminePb

Short Name	EcuMRbDeterminePb
Description	Post Build configuration determination criterion. Configure this container only if post build selectable features is used. Currently, detection via only Port pins is supported
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.32 EcuM/EcuMGeneral/EcuMRbDeterminePb/EcuMRbBootCtrlRelevant

Short Name	EcuMRbBootCtrlRelevant
Description	If set to true the port configuration will be forwarded to rba_BootCtrl module. If this parameter is set to false or left unconfigured, there will be no forwarding to rba_BootCtrl
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.33 EcuM/EcuMGeneral/EcuMRbDeterminePb/EcuMRbPbVariant

Short Name	EcuMRbPbVariant
Description	Here we configure the patterns of the pin value corresponding to the ECU logical number. Right now a maximum of 65535 ECUs are supported.
Multiplicity	2..65536
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.34 EcuM/EcuMGeneral/EcuMRbDeterminePb/EcuMRbPbVariant/EcuMRbPattern

Short Name	EcuMRbPattern
Description	Pin Pattern corresponding to the ECU configured above
Multiplicity	1..1

Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.35 EcuM/EcuMGeneral/EcuMRbDeterminePb/EcuMRbPbVariant/EcuMRbPbVariantIndex

Short Name	EcuMRbPbVariantIndex
Description	Logical number of the ECU
Multiplicity	1..1
Value Range	0..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.36 EcuM/EcuMGeneral/EcuMRbDeterminePb/EcuMRbPortConfiguration

Short Name	EcuMRbPortConfiguration
Description	This is port configuration which is used to determine the post build selectable configuration. Only one port is supported at the moment. Maximum of three pins can be configured inside the Port.
Multiplicity	1..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.37 EcuM/EcuMGeneral/EcuMRbDeterminePb/EcuMRbPortConfiguration/EcuMRbPin

Short Name	EcuMRbPin
Description	Pin number within the port
Multiplicity	1..16
Value Range	0..15
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.38 EcuM/EcuMGeneral/EcuMRbDeterminePb/EcuMRbPortConfiguration/EcuMRbPort

Short Name	EcuMRbPort
Description	Port number
Multiplicity	1..1
Default Value	-

Type	ECUC-STRING-PARAM-DEF

2.3.1.39 EcuM/EcuMGeneral/EcuMRbDeterminePb/EcuMRbSwcRelevant

Short Name	EcuMRbSwcRelevant
Description	This parameter is applicable only in case of a Master/Slave configuration of ECU. If set to true the API EcuM_Rb_GetEcuType() will be generated. If this parameter is set to false or left unconfigured, this API will not be generated
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.40 EcuM/EcuMGeneral/EcuMRbDisableNvM

Short Name	EcuMRbDisableNvM
Description	Optional parameter provided to disable NvM. If NvM module is configured in a project and the parameter EcuMRbDisableNvM is either set as FALSE or is not configured then shutdown related data will be forwarded to NvM. Otherwise no shutdown related data will be forwarded to NvM and the EcuM API's EcuM_Rb_GetLastShutdownInfo() and EcuM_ GetLastShutdownTarget() turns out to be irrelevant or inaccessible.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.41 EcuM/EcuMGeneral/EcuMRbIncludeComm

Short Name	EcuMRbIncludeComm
Description	If this parameter is set as TRUE, call to COMM functoins is enabled. If this parameter is set as FALSE, call to COMM functions will be disabled. If EcuMRbIncludeComm is set to FALSE ensure that the Parameters EcuMComMChannelRef and EcuMComMPNCRef are not configured in Wakeupsource. This is to ensure ComM Parameters are not defined when it is disabled.
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.42 EcuM/EcuMGeneral/EcuMRbIncludeHeaderList

Short Name	EcuMRbIncludeHeaderList
Description	EcuMRbIncludeHeader is a custom place holder which will be included in EcuM.h. This parameter can be used to export API's of other modules.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.43 EcuM/EcuMGeneral/EcuMRbIncludeHeaderList/EcuMRbModule-Header

Short Name	EcuMRbModuleHeader
Description	Shortname of the moudle to be included or header to be included. Example : Can, Gpt.
Multiplicity	1..*
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.44 EcuM/EcuMGeneral/EcuMRbModeSwitchQueueLength

Short Name	EcuMRbModeSwitchQueueLength
Description	This element specifies whether the mode switch communication is queued or not, the length of the mode switch queue (which is generated by the RTE) on the provider port. Unqueued communication - When a mode manager initiates a mode switch, RTE will begin processing the request. When the mode switch is currently in progress for the mode instance, any further mode switches will be discarded/over-written by RTE. Queued communication - When a mode manager initiates a mode switch, RTE will begin processing the request. When the mode switch is currently in progress for the mode instance, any further mode switches will be queued. The size of the mode switch queue is determined by EcuMRbMod-eSwitchQueueLength.
Multiplicity	0..1
Value Range	1..255
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.45 EcuM/EcuMGeneral/EcuMRbNvMBlockDeviceId

Short Name	EcuMRbNvMBlockDeviceId
Description	This optional parameter defines the NVRAM device ID where the NVRAM block is located.If the EcuM parameter EcuMRbNvMBlockDeviceId is not configured and FEE module is present, then NvMNvramDeviceId forwarded with a value of 0.If FEE module not present and EA module is present, then NvMNvramDeviceId forwarded with a value of 1, along with those warning info message is passed.In case both module are not present only info message will passed.
Multiplicity	0..1
Value Range	0..254
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.46 EcuM/EcuMGeneral/EcuMRbNvMCallbackSignature45

Short Name	EcuMRbNvMCallbackSignature45
Description	If this parameter is set to true or left unconfigured, the AR45 version NvMSingleBlockCallback api is realized. If this parameter is set to false, then AR42 version NvM Callback api is realized.This parameter should have similar configuration of NvMRbCallback SignatureAR45.
Multiplicity	0..1
Default Value	true
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.47 EcuM/EcuMGeneral/EcuMRbOnGoOffOneCallout

Short Name	EcuMRbOnGoOffOneCallout
Description	Container used for defining EcuM_OnGoOffOne callout. If this is not configured integratopr should manually call the functions inside EcuM_Callout_Stubs.c.
Introduction	This container holds a list of function call that will be called on EcuM_OnGoOffOne Callout.
Multiplicity	0..1
Default Value	-

Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.48 EcuM/EcuMGeneral/EcuMRbOnGoOffOneCallout/EcuMRbCallOutItem

Short Name	EcuMRbCallOutItem
Description	Container used for defining EcuM callout function calls.
Multiplicity	1..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.49 EcuM/EcuMGeneral/EcuMRbOnGoOffOneCallout/EcuMRbCallOutItem/EcuMRbCalloutCondition

Short Name	EcuMRbCalloutCondition
Description	Condition needed to be checked before function calls. Note: This condition check is only possible with Run time. Pre-Compile time check is not possible.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.50 EcuM/EcuMGeneral/EcuMRbOnGoOffOneCallout/EcuMRbCallOutItem/EcuMRbCalloutCondition/EcuMRbCalloutHeaderList

Short Name	EcuMRbCalloutHeaderList
Description	Header needed to be included for condition check.
Multiplicity	1..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.51 EcuM/EcuMGeneral/EcuMRbOnGoOffOneCallout/EcuMRbCallOutItem/EcuMRbCalloutCondition/EcuMRbCalloutHeaderList/EcuMRbCalloutConditionHeader

Short Name	EcuMRbCalloutConditionHeader
Description	Header to be included.
Multiplicity	1..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.52 EcuM/EcuMGeneral/EcuMRbOnGoOffOneCallout/EcuMRbCall-

OutItem/EcuMRbCalloutCondition/EcuMRbCalloutHeaderList/
EcuMRbServiceIsNonAutosar

Short Name	EcuMRbServiceIsNonAutosar
Description	Flag, which is set true, if the module is Non-AUTOSAR specified. By default if this parameter is not configured the value is taken as FALSE. if the parameter is FALSE, then for the header inclused by EcuMRbModuleID will be followed by a mdule version check.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.53 EcuM/EcuMGeneral/EcuMRbOnGoOffOneCallout/EcuMRbCallOutItem/EcuMRbCalloutCondition/EcuMRbCondition

Short Name	EcuMRbCondition
Description	If there is any condition needs to be checked before calling the function then condition can be added in this parameter. Note: This condition check is only possible with Run time. Pre-Compile time check is not possible.
Multiplicity	1..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.54 EcuM/EcuMGeneral/EcuMRbOnGoOffOneCallout/EcuMRbCallOutItem/EcuMRbCalloutFunction

Short Name	EcuMRbCalloutFunction
Description	Function to be called from the callout. Both void and non-void functions are allowed. Function calls with parameters can be passed directly as '$Function_Name($parameter)' with no restriction on the number of parameters. Function calls without parameters can be passed as '$Function_Name'.
Multiplicity	1..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.55 EcuM/EcuMGeneral/EcuMRbOnGoOffOneCallout/EcuMRbCallOutItem/EcuMRbModuleID

Short Name	EcuMRbModuleID

Description	Module name of the function call. This is needed for the header inclusion needed for EcuMRbCalloutFunction configured. Example : NvM
Multiplicity	1..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.56 EcuM/EcuMGeneral/EcuMRbOnGoOffOneCallout/EcuMRbCallOutItem/EcuMRbSequenceID

Short Name	EcuMRbSequenceID
Description	This ID determines the sequence in which the configured callouts appear. If all driver list items are provided with a unique ID, generation of callouts is done based on this ID. If some sequence IDs are duplicate or some sequence IDs are missing, error is thrown. If ID is not configured for any of the EcuMRbCallOutItems, then the callouts are generated based on the order in which they are configured.
Multiplicity	0..1
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.57 EcuM/EcuMGeneral/EcuMRbOnGoOffOneCallout/EcuMRbCallOutItem/EcuMRbServiceIsNonAutosar

Short Name	EcuMRbServiceIsNonAutosar
Description	Flag, which is set true, if the module is Non-AUTOSAR specified. By default if this parameter is not configured the value is taken as FALSE. if the parameter is FALSE, then for the header inclused by EcuMRbModuleID will be followed by a mdule version check.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.58 EcuM/EcuMGeneral/EcuMRbOnGoOffTwoCallout

Short Name	EcuMRbOnGoOffTwoCallout
Description	Container used for defining EcuM_OnGoOffTwo callout. If this is not configured integratopr should manually call the functions inside EcuM_Callout_Stubs.c.
Introduction	This container holds a list of function call that will be called on EcuMShutdownOnGoOffTwo callout.

Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.59 EcuM/EcuMGeneral/EcuMRbOnGoOffTwoCallout/EcuMRbCallOutItem

Short Name	EcuMRbCallOutItem
Description	Container used for defining EcuM callout function calls.
Multiplicity	1..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.60 EcuM/EcuMGeneral/EcuMRbOnGoOffTwoCallout/EcuMRbCallOutItem/EcuMRbCalloutCondition

Short Name	EcuMRbCalloutCondition
Description	Condition needed to be checked before function calls. Note: This condition check is only possible with Run time. Pre-Compile time check is not possible.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.61 EcuM/EcuMGeneral/EcuMRbOnGoOffTwoCallout/EcuMRbCallOutItem/EcuMRbCalloutCondition/EcuMRbCalloutHeaderList

Short Name	EcuMRbCalloutHeaderList
Description	Header needed to be included for condition check.
Multiplicity	1..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.62 EcuM/EcuMGeneral/EcuMRbOnGoOffTwoCallout/EcuMRbCallOutItem/EcuMRbCalloutCondition/EcuMRbCalloutHeaderList/EcuMRbCalloutConditionHeader

Short Name	EcuMRbCalloutConditionHeader
Description	Header to be included.
Multiplicity	1..1
Default Value	-

Type	ECUC-STRING-PARAM-DEF

2.3.1.63 EcuM/EcuMGeneral/EcuMRbOnGoOffTwoCallout/EcuMRbCallOutItem/EcuMRbCalloutCondition/EcuMRbCalloutHeaderList/EcuMRbServiceIsNonAutosar

Short Name	EcuMRbServiceIsNonAutosar
Description	Flag, which is set true, if the module is Non-AUTOSAR specified. By default if this parameter is not configured the value is taken as FALSE. if the parameter is FALSE, then for the header inclused by EcuMRbModuleID will be followed by a mdule version check.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.64 EcuM/EcuMGeneral/EcuMRbOnGoOffTwoCallout/EcuMRbCallOutItem/EcuMRbCalloutCondition/EcuMRbCondition

Short Name	EcuMRbCondition
Description	If there is any condition needs to be checked before calling the function then condition can be added in this parameter. Note: This condition check is only possible with Run time. Pre-Compile time check is not possible.
Multiplicity	1..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.65 EcuM/EcuMGeneral/EcuMRbOnGoOffTwoCallout/EcuMRbCallOutItem/EcuMRbCalloutFunction

Short Name	EcuMRbCalloutFunction
Description	Function to be called from the callout. Both void and non-void functions are allowed. Function calls with parameters can be passed directly as '$Function_Name($parameter)' with no restriction on the number of parameters. Function calls without parameters can be passed as '$Function_Name'.
Multiplicity	1..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.66 EcuM/EcuMGeneral/EcuMRbOnGoOffTwoCallout/EcuMRbCallOutItem/EcuMRbModuleID

Short Name	EcuMRbModuleID
Description	Module name of the function call. This is needed for the header inclusion needed for EcuMRbCalloutFunction configured. Example : NvM
Multiplicity	1..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

2.3.1.67 EcuM/EcuMGeneral/EcuMRbOnGoOffTwoCallout/EcuMRbCallOutItem/EcuMRbSequenceID

Short Name	EcuMRbSequenceID
Description	This ID determines the sequence in which the configured callouts appear. If all driver list items are provided with a unique ID, generation of callouts is done based on this ID. If some sequence IDs are duplicate or some sequence IDs are missing, error is thrown. If ID is not configured for any of the EcuMRbCallOutItems, then the callouts are generated based on the order in which they are configured.
Multiplicity	0..1
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.68 EcuM/EcuMGeneral/EcuMRbOnGoOffTwoCallout/EcuMRbCallOutItem/EcuMRbServiceIsNonAutosar

Short Name	EcuMRbServiceIsNonAutosar
Description	Flag, which is set true, if the module is Non-AUTOSAR specified. By default if this parameter is not configured the value is taken as FALSE. if the parameter is FALSE, then for the header inclused by EcuMRbModuleID will be followed by a mdule version check.
Multiplicity	0..1
Default Value	-
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.69 EcuM/EcuMGeneral/EcuMRbRtePortInterfaces

Short Name	EcuMRbRtePortInterfaces

Description	This container contains the Queue Length parameters for EcuM port instances that are created for different operations and modes.
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.70 EcuM/EcuMGeneral/EcuMRbRtePortInterfaces/EcuMRbBootTargetQueueLength

Short Name	EcuMRbBootTargetQueueLength
Long Name	Queue length for BootTarget Operations
Description	This container contains the Queue Length parameters of different operations in BootTarget port instances
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.71 EcuM/EcuMGeneral/EcuMRbRtePortInterfaces/EcuMRbBootTargetQueueLength/EcuMRbGetBootTargetQueueLength

Short Name	EcuMRbGetBootTargetQueueLength
Description	This element specifies queue length of GetbootTarget, the length of the ServercomSpec (which is generated by the RTE) on the provider port. Setting value of 1 implies that incoming requests are rejected while another request that arrived earlier is being processed
Multiplicity	0..1
Value Range	1..255
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.72 EcuM/EcuMGeneral/EcuMRbRtePortInterfaces/EcuMRbBootTargetQueueLength/EcuMRbSelectBootTargetQueueLength

Short Name	EcuMRbSelectBootTargetQueueLength
Description	This element specifies queue length of Select bootTarget, the length of the ServercomSpec (which is generated by the RTE) on the provider port. Setting value of 1 implies that incoming requests are rejected while another request that arrived earlier is being processed
Multiplicity	0..1

Value Range	1..255
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.73 EcuM/EcuMGeneral/EcuMRbRtePortInterfaces/EcuMRbShutdownTargetQueueLength

Short Name	EcuMRbShutdownTargetQueueLength
Long Name	Queue length for ShutdownTarget Operations
Description	This container contains the Queue Length parameters of different operations in ShutdownTarget port instances
Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

2.3.1.74 EcuM/EcuMGeneral/EcuMRbRtePortInterfaces/EcuMRbShutdownTargetQueueLength/EcuMRbGetLastShutdownTargetQueueLength

Short Name	EcuMRbGetLastShutdownTargetQueueLength
Description	This element specifies queue length of GetLastShutdownTarget, the length of the ServercomSpec (which is generated by the RTE) on the provider port. Setting value of 1 implies that incoming requests are rejected while another request that arrived earlier is being processed
Multiplicity	0..1
Value Range	1..255
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.75 EcuM/EcuMGeneral/EcuMRbRtePortInterfaces/EcuMRbShutdownTargetQueueLength/EcuMRbGetShutdownCauseQueueLength

Short Name	EcuMRbGetShutdownCauseQueueLength
Description	This element specifies queue length of GetShutdownCause, the length of the ServercomSpec (which is generated by the RTE) on the provider port. Setting value of 1 implies that incoming requests are rejected while another request that arrived earlier is being processed
Multiplicity	0..1
Value Range	1..255

Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.76 EcuM/EcuMGeneral/EcuMRbRtePortInterfaces/EcuMRbShutdownTargetQueueLength/EcuMRbGetShutdownTargetQueueLength

Short Name	EcuMRbGetShutdownTargetQueueLength
Description	This element specifies queue length of GetShutdownTarget, the length of the ServercomSpec (which is generated by the RTE) on the provider port. Setting value of 1 implies that incoming requests are rejected while another request that arrived earlier is being processed
Multiplicity	0..1
Value Range	1..255
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.77 EcuM/EcuMGeneral/EcuMRbRtePortInterfaces/EcuMRbShutdownTargetQueueLength/EcuMRbSelectShutdownCauseQueueLength

Short Name	EcuMRbSelectShutdownCauseQueueLength
Description	This element specifies queue length of SelectShutdownCause, the length of the ServercomSpec (which is generated by the RTE) on the provider port. Setting value of 1 implies that incoming requests are rejected while another request that arrived earlier is being processed
Multiplicity	0..1
Value Range	1..255
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.78 EcuM/EcuMGeneral/EcuMRbRtePortInterfaces/EcuMRbShutdownTargetQueueLength/EcuMRbSelectShutdownTargetQueueLength

Short Name	EcuMRbSelectShutdownTargetQueueLength
Description	This element specifies queue length of SelectShutdownTarget, the length of the ServercomSpec (which is generated by the RTE) on the provider port. Setting value of 1 implies that incoming requests are rejected while another request that arrived earlier is being processed

Multiplicity	0..1
Value Range	1..255
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

2.3.1.79 EcuM/EcuMGeneral/EcuMRbRunMinimumDuration

Short Name	EcuMRbRunMinimumDuration
Description	Duration given in seconds. If this is configured with a non-Zero value, after the startup EcuM will continue to stay in RUN state for that much duration even in the case of no run request from any users. When configuring this value Integrator should consider the time by when Rte is getting started.
Multiplicity	0..1
Default Value	-
Type	ECUC-FLOAT-PARAM-DEF

2.3.1.80 EcuM/EcuMGeneral/EcuMRbSlaveCoreEarlierStart

Short Name	EcuMRbSlaveCoreEarlierStart
Description	If this parameter is set as TRUE, Slave cores would be started earlier i.e., Slave cores would be started by master just before initializing EcuMDriverInitListOne.If this parameter is set as FALSE, then Slave cores would be started at the end phase of EcuM_Init.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.81 EcuM/EcuMGeneral/EcuMRbStartupCore

Short Name	EcuMRbStartupCore
Description	If Configured with a ecucCoreDefinition reference, then the configured core behaves like Master Core where all the startup,wakeup and shutdown sequences are carried out. If not Configured,then Core 0 is the Master Core.This parameter is added to fulfill the safety requirement of IFX-Device 4 that complete startup should run on a core with lockstep.
Multiplicity	0..1
Default Value	-

Type	ECUC-REFERENCE-DEF

2.3.1.82 EcuM/EcuMGeneral/EcuMRbStartupDuration

Short Name	EcuMRbStartupDuration
Description	If this parameter is set to TRUE ,then the startup duration time measurement is enabled.8 bytes will be forwareded as the size of ECUM_CFG_NVM_BLOCK block.If this is set to FALSE or not configured then the startup duration time measurement is disabled .4 bytes will be forwareded as the size of ECUM_CFG_ NVM_BLOCK block
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.1.83 EcuM/EcuMGeneral/EcuMRbUserTypeUpdate

Short Name	EcuMRbUserTypeUpdate
Description	This parameter can be used to configure the Imple-mentation type of EcuM_UserType in the file EcuM_ Cfg_SWCD.arxml. If this parameter is set as TRUE, then EcuM_UserType will have type uint8. If the parameter is set as FALSE OR not configured then EcuM_UserType will have type uint16.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

2.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

2.3.2.1 EcuM_GoDownHaltPoll

Syntax	`EcuM_GoDownHaltPoll( uint16 caller)`
Parameter	`Caller`
Description	Instructs the ECU State Manager module to perform a power off or reset, or initiate sleep depending on the selected shutdown target. If Sleep is chosen Halt or Poll is called depending on the selected sleep mode

2.3.2.2 EcuM_Rb_NvMSingleBlockCallbackFunction

Syntax	**EcuM_Rb_NvMSingleBlockCallbackFunction** AR45: (NvM_BlockRequestType BlockRequest, NvM_RequestResultType JobResult) AR42: :code:`(ServiceId, NvM_RequestResultType JobResult)
Parameters In	**AR45: BlockRequest** JobResult **AR42: ServiceId** JobResult
Return Value	E_OK, E_NOT_OK
Description	Invoked by NvM on termination of each access to the EcuM configured NvM block.

BlockRequest EcuM_Rb_GetLastShutdownInfo “”””””””””””””””””””””””””

Syntax	EcuM_Rb_GetLastShutdownInfo(EcuM_ShutdownInfoType * shutdownCauseInfo)
Parameter Out	shutdownCauseInfo
Description	Returns the shutdown cause as well as time taken for the previous shutdown process if time measurement is enabled.

2.3.2.3 EcuM_KillAllRUNRequests

Syntax	EcuM_KillAllRUNRequests(void)
Description	Unconditionally releases all pending requests to RUN.

2.3.2.4 EcuM_KillAllPostRUNRequests

Syntax	EcuM_KillAllPostRUNRequests(void)
Description	Unconditionally releases all pending requests to PostRUN.

2.3.2.5 EcuM_Rb_KillAllRequests

Syntax	EcuM_Rb_KillAllRequests (void)
Description	Unconditionally releases all pending requests to RUN and PostRUN.

2.3.2.6 EcuM_Rb_CurrentState

Syntax	EcuM_Rb_CurrentState
Description	Called by RTE mode switch acknowledgement event. It reads the current RTE mode and provides a mapping between mode to state and notify the BswM about current state.

2.4 Limitations

Version v12.7.0 has following limitations.

- The optional Alarm Clock feature is not supported.
- If a wakeup event is in pending status during the OffPreOS sequence, the shutdown target will not be switched from OFF to RESET.
- The Multi-partition concept is not currently implemented in EcuM, so the multiplicity of EcuMPartitionRef is 0.
- In the Autosar specification, the multiplicity of EcuMWakeupSourcePolling is 1, but EcuMWakeupSourcePolling for pre-defined WakeupSources is not clear, so the multiplicity is 0.
- In the Autosar specification, EcuMModuleRef has a multiplicity 1, but to avoid backward incompatibility, the multiplicity is 0.
- EcuMEcucCoreDefinitionRef is not implemented in the DriverInitItem of EcuMDriverInitListBswM and EcuMDriverRestartList. Currently, however, EcuMEcucCoreDefinitionRef is implemented in EcuMDriverInitListZero and EcuMDriverInitListOne.
- The Wakeup Source state of ECUM_WKSTATUS_DISABLED is not supported.
- The Master/Slave (Multi-Core) feature is only supported if the number of port pins used for Post-Build data selection is less than or equal to 15.
- In a multi-core scenario as per Autosar, Pb Determination (EcuM_DeterminePbConfiguration) happens in both the master and slave cores. In RTA-BSW v12.7.0, however, it happens only in the master core.
- Hash Computation is not supported.
- The sequence of initialising peripherals in initialisation sequences zero and one must not cause any problems
- Each of these functions is called when the OS is not running and, therefore, has restrictions on the OS calls that are allowed to be made
- The EcuM may use an area of non-initialised ECU RAM. The EcuM Specification states that “The system designer is responsible for establishing an initialization strategy for the no init area of the ECU RAM.”

2.5 Exclusions

The following features from AR v22-11 are not supported in version v12.7.0.

API functions not currently implemented:

- `EcuM_SetRelWakeupAlarm`
- `EcuM_SetAbsWakeupAlarm`

- `EcuM_AbortWakeupAlarm`
- `EcuM_GetCurrentTime`
- `EcuM_GetWakeupTime`
- `EcuM_SetClock`

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMConfigConsistencyHash
- EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMDefaultShutdownTarget/EcuMDefaultShutdownTarget
- EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMDriverRestartList/EcuMDriverInitItem/EcuMEcucCoreDefinitionRef
- EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMAlarmClock
- EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMAlarmClock/EcuMAlarmClockId
- EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMAlarmClock/EcuMAlarmClockTimeOut
- EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMAlarmClock/EcuMAlarmClockUser
- EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMDriverInitListBswM/EcuMDriverInitItem/EcuMEcucCoreDefinitionRef
- EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMFlexUserConfig/EcuMFlexEcucPartitionRef
- EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMPartitionRef
- EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMSetClockAllowedUsers
- EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMSetClockAllowedUsers/EcuMSetClockAllowedUserRef
- EcuM/EcuMFlexGeneral/EcuMAlarmWakeupSource
- EcuM/EcuMFlexGeneral/EcuMResetLoopDetection
- EcuM/EcuMGeneral/EcuMAlarmClockPresent

2.6 Deviations

2.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

2.6.1.1 EcuM

[RTA-BSW] Description	Configuration of the EcuM (ECU State Manager) module (with vendor specific enhancements of RB).
[AUTOSAR] Description	Configuration of the EcuM (ECU State Manager) module.

2.6.1.2 EcuM/EcuMConfiguration

[RTA-BSW] Description	This container contains the configuration (parameters) of the ECU State Manager. For every post-build variant, one instance of EcuMConfiguration should be created. If the project is configured with post-build variant criterion approach a single instance only needed.
[AUTOSAR] Description	This container contains the configuration (parameters) of the ECU State Manager.

2.6.1.3 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMDefaultAppMode

[RTA-BSW] Description	The default OS application mode loaded when the ECU comes out of reset.
[AUTOSAR] Description	The default application mode loaded when the ECU comes out of reset.

2.6.1.4 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMDefaultShutdownTarget

[RTA-BSW] Description	This container describes the default shutdown target to be selected by EcuM. It can be OFF or RESET. The shutdown target selected by this configuration can be overridden by the EcuM_SelectShutdownTarget service.
[AUTOSAR] Description	This container describes the default shutdown target to be selected by EcuM. The actual shutdown target may be overridden by the EcuM_SelectShutdownTarget service.

2.6.1.5 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMDefaultShutdownTarget/EcuMDefaultResetModeRef

[RTA-BSW] Description	If EcuMDefaultShutdownTarget is EcuMStateReset, this parameter selects the default reset mode. In all other cases this parameter can be ignored.
[AUTOSAR] Description	If EcuMDefaultShutdownTarget is EcuMShutdownTargetReset, this parameter selects the default reset mode. Otherwise this parameter may be ignored.

2.6.1.6 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMDefaultShutdownTarget/EcuMDefaultSleepModeRef

[RTA-BSW] Description	If EcuMDefaultShutdownTarget is EcuMStateSleep, this parameter selects the default Sleep mode. In all other cases this parameter can be ignored.
[AUTOSAR] Description	If EcuMDefaultShutdownTarget is EcuMShutdownTargetSleep, this parameter selects the default sleep mode. Otherwise this parameter may be ignored.

2.6.1.7 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMDriverInitListOne

[RTA-BSW] Introduction	This container holds a list of module IDs that will be initialised from EcuM_AL_DriverInitOne. All modules that should be initialized before StartOs and having post-build dependency for initialization should be configured here. Multi-Core initialization is supported that is modules can be initialized based on the Os Core. Note that the driver init list configured here is pre-compile. So the order and init items should be same in all configured EcuM variant. The init order will be same as that given in first EcuMConfiguration container.
[AUTOSAR] Introduction	This container holds a list of modules to be initialized. Each module in the list will be called for initialization in the list order. All modules in this list are initialized before the OS is started and so these modules require no OS support.

2.6.1.8 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMDriverInitListOne/EcuMDriverInitItem

[RTA-BSW] Description	These containers describe the entries in a driver initialization list. Each module in the list will be called for initialisation from EcuM_Init in the same order as configured in this list.
[AUTOSAR] Description	These containers describe the entries in a driver init list.

2.6.1.9 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMDriverInitListOne/EcuMDriverInitItem/EcuMModuleParameter

[RTA-BSW] Description	Definition of the function prototype and the parameter passed to the function. If EcuMModuleParameter → “POSTBUILD” then EcuM will call Dem_PreInit(&Dem_Config) , If EcuMModuleParameter → “NULL_PTR” then EcuM will call Dem_PreInit (NULL_PTR) , If EcuMModuleParameter → “VOID” then EcuM will call Dem_PreInit ().
[AUTOSAR] Description	Definition of the function prototype and the parameter passed to the function.

2.6.1.10 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuM-

DriverInitListOne/EcuMDriverInitItem/EcuMModuleRef

[RTA-BSW] Description	Foreign reference to the configuration of a module instance which shall be initialized by EcuM. If EcuMModuleRef and EcuMModuleId both are configured EcuMModuleRef will be discarded by code generators.
[AUTOSAR] Description	Foreign reference to the configuration of a module instance which shall be initialized by EcuM
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

2.6.1.11 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMDriverInitListOne/EcuMDriverInitItem/EcuMModuleService

[RTA-BSW] Description	The service to be called to initialize that module, e.g. Init, PreInit, Start etc. If the parameter is left unconfigured, by default 'Init' will be considerd
[AUTOSAR] Description	The service to be called to initialize that module, e.g. Init, PreInit, Start etc. If nothing is defined "Init" is taken by default.
[ADDED] Introduction	If the service is Init and the parameter EcuMModuleConfigurationRef has been set for that module, the corresponding pointer to the init structure (<Module>_ConfigType) and in case of multiple instantiation an uint8 value to identify the instance of the module(<MSN>_CtrlIdx) shall be passed as arguments.

2.6.1.12 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMDriverInitListZero

[RTA-BSW] Introduction	This container holds a list of module IDs that will be initialised from EcuM_AL_DriverInitZero. All modules in this list are initialised before the post-build configuration has been loaded and the OS is initialized. Therefore, these modules may not use post-build configuration. Note that the driver init list configured here is pre-compile. So the order and init items should be same in all configured EcuM variant. The init order will be same as that given in first EcuMConfiguration container.
[AUTOSAR] Introduction	This container holds a list of modules to be initialized. Each module in the list will be called for initialization in the list order. All modules in this list are initialized before the post-build configuration has been loaded and the OS is initialized. Therefore, these modules may not use post-build configuration.

2.6.1.13 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuM-DriverInitListZero/EcuMDriverInitItem/EcuMModuleParameter

[RTA-BSW] Description	Definition of the function prototype and the parameter passed to the function. If EcuMModuleParameter → “NULL_PTR” then EcuM will call Dem_PreInit (NULL_PTR) , If EcuMModuleParameter → “VOID” then EcuM will call Dem_PreInit ().
[AUTOSAR] Description	Definition of the function prototype and the parameter passed to the function.
[RTA-BSW] Enumeration Literals	NULL_PTR, VOID
[AUTOSAR] Enumeration Literals	NULL_PTR, POSTBUILD_PTR, VOID

2.6.1.14 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuM-DriverInitListZero/EcuMDriverInitItem/EcuMModuleRef

[RTA-BSW] Description	Foreign reference to the configuration of a module instance which shall be initialized by EcuM. If EcuMModuleRef and EcuMModuleId both are configured EcuMModuleRef will be discarded by code generators.
[AUTOSAR] Description	Foreign reference to the configuration of a module instance which shall be initialized by EcuM
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

2.6.1.15 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuM-DriverInitListZero/EcuMDriverInitItem/EcuMModuleService

[RTA-BSW] Description	The service to be called to initialize that module, e.g. Init, PreInit, Start etc. If the parameter is left unconfigured, by default ‘Init’ will be considerd
[AUTOSAR] Description	The service to be called to initialize that module, e.g. Init, PreInit, Start etc. If nothing is defined “Init” is taken by default.
[ADDED] Introduction	If the service is Init and the parameter EcuMModuleConfigurationRef has been set for that module, the corresponding pointer to the init structure (<Module>_ConfigType) and in case of multiple instantiation an uint8 value to identify the instance of the module(<MSN>_CtrlIdx) shall be passed as arguments.

2.6.1.16 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuM-

DriverRestartListEcuMDriverRestartList

[RTA-BSW] Description	Container for Driver Restart Block. Modules configured here should be a subset of Driver init list 0 and Driver init List one.
[AUTOSAR] Description	List of modules to be initialized.
[ADDED] Introduction	List of Module IDs. Entries in this list must appear in the same order as in the combined list of EcuMDriverInitListOne and EcuM_DriverInitListTwo.This list may be a real subset though. In all other cases , the generation tool shall report error. Note that the driver init list configured here is pre-compile. So the order and init items should be same in all configured EcuM variant. The init order will be same as that given in first EcuMConfiguration container.

2.6.1.17 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuM-DriverRestartList/EcuMDriverInitItem/EcuMModuleParameter

[RTA-BSW] Description	Definition of the function prototype and the parameter passed to the function. Applicable only for DriverInitListOne Items. If EcuMModuleParameter → “POSTBUILD” then EcuM will call Dem_PreInit(&Dem_Config) , If EcuMModuleParameter → “NULL_PTR” then EcuM will call Dem_PreInit (NULL_PTR) , If EcuMModuleParameter → “VOID” then EcuM will call Dem_PreInit ().
[AUTOSAR] Description	Definition of the function prototype and the parameter passed to the function.

2.6.1.18 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuM-DriverRestartList/EcuMDriverInitItem/EcuMModuleRef

[RTA-BSW] Description	Foreign reference to the configuration of a module instance which shall be initialized by EcuM. If EcuMModuleRef and EcuMModuleId both are configured EcuMModuleRef will be discarded by code generators.
[AUTOSAR] Description	Foreign reference to the configuration of a module instance which shall be initialized by EcuM
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

2.6.1.19 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuM-DriverRestartList/EcuMDriverInitItem/EcuMModuleService

[RTA-BSW] Description	The service to be called to initialize that module, e.g. Init, PreInit, Start etc. If the parameter is left unconfigured, by default ‘Init’ will be considerd
[AUTOSAR] Description	The service to be called to initialize that module, e.g. Init, PreInit, Start etc. If nothing is defined “Init” is taken by default.

[ADDED] Introduction	If the service is Init and the parameter EcuMModuleConfigurationRef has been set for that module, the corresponding pointer to the init structure (<Module>_ConfigType) and in case of multiple instantiation an uint8 value to identify the instance of the module(<MSN>_CtrlIdx) shall be passed as arguments.

2.6.1.20 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMOSResource

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

2.6.1.21 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMSleepMode

[RTA-BSW] Description	These containers describe the configured Sleep Modes. Sleep modes are actually reference to different Mcu Modes. So in-order to configure this container Mcu configuration of Modes is mandatory.
[AUTOSAR] Description	These containers describe the configured sleep modes.
[REMOVED] Introduction	The names of these containers specify the symbolic names of the different sleep modes.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

2.6.1.22 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMSleepMode/EcuMSleepModeId

[RTA-BSW] Description	This ID identifies the Sleep mode in Services like EcuM_SelectShutdownTarget. The id is auto-generated by EcuM.
[AUTOSAR] Description	This ID identifies this sleep mode in services like EcuM_SelectShutdownTarget.
[REMOVED] Value Range	0..255

2.6.1.23 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMSleepMode/EcuMSleepModeMcuModeRef

[RTA-BSW] Description	This parameter is a reference to the corresponding MCU mode for the sleep mode configured.
[AUTOSAR] Description	This parameter is a reference to the corresponding MCU mode for this sleep mode.

2.6.1.24 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuM-SleepMode/EcuMSleepModeSuspend

[RTA-BSW] Description	This flag is set to true,the CPU will be suspended, halted or powered off in the sleep mode (Halt mode).Else, if the flag is reset the CPU will enter into Polling mode.
[AUTOSAR] Description	Flag, which is set true, if the CPU is suspended, halted, or powered off in the sleep mode. If the CPU keeps running in this sleep mode, then this flag must be set to false.

2.6.1.25 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuM-SleepMode/EcuMWakeupSourceMask

[RTA-BSW] Description	These parameters are references to the the wakeup sources that shall be enabled for this sleep mode. In-order to wakeup from the configured Sleep Mode, only the list of wakeup sources configured in EcuMWakeupSource-Mask are valid.
[AUTOSAR] Description	These parameters are references to the wakeup sources that shall be enabled for this sleep mode.

2.6.1.26 EcuM/EcuMConfiguration/EcuMCommonConfiguration/ EcuMWakeupSource

[RTA-BSW] Description	These containers describe the configured wakeup sources. Maximum 32 wakeup sources are valid. Among this 32 , 5 are pre-defined in EcuM that are standard wakeup sources like ECUM_WKSOURCE_POWER, ECUM_ WKSOURCE_RESET, ECUM_WKSOURCE_INTERNAL_RESET, ECUM_ WKSOURCE_INTERNAL_WDG, ECUM_WKSOURCE_EXTERNAL_WDG. And these 5 sources are auto-configured by EcuM.
[AUTOSAR] Description	These containers describe the configured wakeup sources.

2.6.1.27 EcuM/EcuMConfiguration/EcuMCommonConfiguration/ EcuMWakeupSource/EcuMComMChannelRef

[RTA-BSW] Description	This parameter could reference multiple Networks (channels) defined in the Communication Manager. No reference indicates that the wakeup source is not a communication channel. If this reference is configured, the wakeup source detection will be indicated to ComM.
[AUTOSAR] Description	This parameter could reference multiple Networks (channels) defined in the Communication Manager. No reference indicates that the wakeup source is not a communication channel.

2.6.1.28 EcuM/EcuMConfiguration/EcuMCommonConfiguration/ EcuMWakeupSource/EcuMComMPNCRef

[RTA-BSW] Description	This is a reference to a one or more PNC's defined in the Communication Manager. No reference indicates that the wakeup source is not assigned to a partial network.
[AUTOSAR] Description	This is a reference to a one or more PNC's defined in the Communication Manager.
[REMOVED] Introduction	No reference indicates that the wakeup source is not assigned to a partial network.

2.6.1.29 EcuM/EcuMConfiguration/EcuMCommonConfiguration/ EcuMWakeupSource/EcuMResetReasonRef

[RTA-BSW] Description	This parameter describes the mapping of reset reasons detected by the MCU driver into wakeup sources. This mapping is needed during EcuM initialization to check the last reset reason.
[AUTOSAR] Description	This parameter describes the mapping of reset reasons detected by the MCU driver into wakeup sources.
[RTA-BSW] Upper Multiplicity	1
[AUTOSAR] Upper Multiplicity	*

2.6.1.30 EcuM/EcuMConfiguration/EcuMCommonConfiguration/ EcuMWakeupSource/EcuMValidationTimeout

[RTA-BSW] Description	The validation timeout (period for which the ECU State Manager will wait for the validation of a wakeup event) can be defined for each wakeup source independently. The timeout is specified in seconds. If this timeout is not configured there wont be any validation routine for this wakeup source and EcuM will consider this source as a valid one immediately. There is no separate timer for this, instead its considered as number of main function periods.
[AUTOSAR] Description	The validation timeout (period for which the ECU State Manager will wait for the validation of a wakeup event) can be defined for each wakeup source independently. The timeout is specified in seconds.

2.6.1.31 EcuM/EcuMConfiguration/EcuMCommonConfiguration/ EcuMWakeupSource/EcuMWakeupSourceId

[RTA-BSW] Description	This parameter defines the identifier of this wakeup source. This is auto-configured by EcuM

[AUTOSAR] Description	This parameter defines the identifier of this wakeup source.
[REMOVED] Introduction	The first five bits are reserved values from the EcuM_WakeupSourceType.
[RTA-BSW] Minimum Value	0
[AUTOSAR] Minimum Value	5

2.6.1.32 EcuM/EcuMConfiguration/EcuMCommonConfiguration/EcuMWakeupSource/EcuMWakeupSourcePolling

[RTA-BSW] Description	This parameter describes if Wakeup Source needs Polling.If this parameter is set, then wakeupsource needs polling. Else, if the parameter is reset, Wakeup source does not need polling.
[AUTOSAR] Description	This parameter describes if the wakeup source needs polling.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

2.6.1.33 EcuM/EcuMConfiguration/EcuMFlexConfiguration

[RTA-BSW] Lower Multiplicity	1
[AUTOSAR] Lower Multiplicity	0

2.6.1.34 EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMDriverInitListBswM

[RTA-BSW] Description	Container for BswM Init Block.
[AUTOSAR] Description	This container holds a list of modules to be initialized by the BswM.
[ADDED] Introduction	This container holds a list of modules to be initialized by the BswM. Note that the driver init list configured here is pre-compile. So the order and init items should be same in all configured EcuM variant. The init order will be same as that given in first EcuMFlexConfiguration container.

2.6.1.35 EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMDriverInitListBswM/EcuMDriverInitItem

[RTA-BSW] Description	These containers describe the entries in a driver initialization list. Each module in the list will be called for initialisation from BswM in the same order as configured in this list.

[AUTOSAR] Description	These containers describe the entries in a driver init list.

2.6.1.36 EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMDriverInitListBswM/EcuMDriverInitItem/EcuMModuleParameter

[RTA-BSW] Description	Definition of the function prototype and the parameter passed to the function. If EcuMModuleParameter → “POSTBUILD” then EcuM will call Dem_PreInit(&Dem_Config) , If EcuMModuleParameter → “NULL_PTR” then EcuM will call Dem_PreInit (NULL_PTR) , If EcuMModuleParameter → “VOID” then EcuM will call Dem_PreInit ().
[AUTOSAR] Description	Definition of the function prototype and the parameter passed to the function.
[ADDED] Default Value	VOID

2.6.1.37 EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMDriverInitListBswM/EcuMDriverInitItem/EcuMModuleRef

[RTA-BSW] Description	Foreign reference to the configuration of a module instance which shall be initialized by EcuM.
[AUTOSAR] Description	Foreign reference to the configuration of a module instance which shall be initialized by EcuM
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

2.6.1.38 EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMDriverInitListBswM/EcuMDriverInitItem/EcuMModuleService

[RTA-BSW] Description	The service to be called to initialize that module, e.g. Init, PreInit, Start etc. If the parameter is left unconfigured, by default ‘Init’ will be considerd
[AUTOSAR] Description	The service to be called to initialize that module, e.g. Init, PreInit, Start etc. If nothing is defined “Init” is taken by default.
[ADDED] Introduction	If the service is Init and the parameter EcuMModuleConfigurationRef has been set for that module, the corresponding pointer to the init structure (<Module>_ConfigType) and in case of multiple instantiation an uint8 value to identify the instance of the module(<MSN>_CtrlIdx) shall be passed as arguments.

2.6.1.39 EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMFlexUserConfig

[RTA-BSW] Upper Multiplicity	*
[AUTOSAR] Upper Multiplicity	256

2.6.1.40 EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMFlexUserConfig/EcuMFlexUser

[RTA-BSW] Description	Parameter used to identify one user. Module Function Id can be used for this instead of selecting some random numbers.
[AUTOSAR] Description	Parameter used to identify one user.
[RTA-BSW] Maximum Value	65535
[AUTOSAR] Maximum Value	255

2.6.1.41 EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMGoDownAllowedUsers

[RTA-BSW] Description	This container describes the collection of allowed users which are allowed to call the EcuM_GoDown API.
[AUTOSAR] Description	This container describes the collection of allowed users which are allowed to call the EcuM_GoDownHaltPoll API (only applies in the case that the previously set shutdown target is TARGET_RESET or TARGET_OFF).

2.6.1.42 EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMGoDownAllowedUsers/EcuMGoDownAllowedUserRef

[RTA-BSW] Description	These parameters describe the references to the users which are allowed to call the EcuM_GoDown API. The user which is very likely allowed to shutdown the system is BswM.
[AUTOSAR] Description	This references an allowed user.

2.6.1.43 EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMNormalMcuModeRef

[RTA-BSW] Description	This parameter is a reference to the normal MCU mode to be restored after waking up from sleep.
[AUTOSAR] Description	This parameter is a reference to the normal MCU mode to be restored after a sleep.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

2.6.1.44 EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMResetMode

[RTA-BSW] Description	These containers describe the configured reset modes. 3 reset modes are standard and this is auto-configured by EcuM like ECUM_RESET_MCU - ECUM_RESET_WDGM - ECUM_RESET_IO.

[AUTOSAR] Description	These containers describe the configured reset modes.
[RTA-BSW] Introduction	The name of these containers allows one of the following symbolic names to be given to the different reset modes: - ECUM_RESET_MCU - ECUM_RESET_WDGM - ECUM_RESET_IO.
[AUTOSAR] Introduction	The name of these containers allows one of the following symbolic names to be given to the different reset modes: * ECUM_RESET_MCU * ECUM_RESET_WDG * ECUM_RESET_IO.

2.6.1.45 EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMResetMode/EcuMResetModeId

[RTA-BSW] Description	This ID identifies this reset mode in services like EcuM_SelectShutdownTarget. This id is auto-configured by EcuM.
[AUTOSAR] Description	This ID identifies this reset mode in services like EcuM_SelectShutdownTarget.

2.6.1.46 EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMShutdownCause

[RTA-BSW] Description	These containers describe the configured shut down or reset causes. 4 shutdown causes are standard and are auto-configured by EcuM like ECUM_CAUSE_UNKNOWN, ECUM_CAUSE_ECU_STATE, ECUM_CAUSE_WDGM and ECUM_CAUSE_DCM
[AUTOSAR] Description	These containers describe the configured shut down or reset causes.
[RTA-BSW] Introduction	The name of these containers allows to give one of the following symbolic names to the diffenrent shut down causes: - ECUM_CAUSE_ECU_STATE - ECU state machine entered a state for shutdown, - ECUM_CAUSE_WDGM - WdgM detected failure, - ECUM_CAUSE_DCM - Dcm requests shutdown (split into UDS services?), - and values from configuration.
[AUTOSAR] Introduction	The name of these containers allows to give one of the following symbolic names to the different shut down causes: * ECUM_CAUSE_ECU_STATE - ECU state machine entered a state for shutdown, * ECUM_CAUSE_WDGM - WdgM detected failure, * ECUM_CAUSE_DCM - Dcm requests shutdown (split into UDS services?), * and values from configuration.

2.6.1.47 EcuM/EcuMConfiguration/EcuMFlexConfiguration/EcuMShutdownCause/EcuMShutdownCauseId

[RTA-BSW] Description	This ID identifies this shut down cause. This id is auto-configured by EcuM.
[AUTOSAR] Description	This ID identifies this shut down cause.

2.6.1.48 EcuM/EcuMFlexGeneral

[REMOVED] Introduction	Only applicable if EcuMFlex is implemented.

2.6.1.49 EcuM/EcuMFlexGeneral/EcuMModeHandling

[RTA-BSW] Description	If this parameter is enabled, EcuM Mode Management feature will be enabled. The ECU Mode Handling introduces a common interface for SW-Cs as known from ECU State Manger with fixed state machine (EcuMFixed). The ECU State Manager with flexible state machine (EcuM-Flex) provides interfaces for SW-Cs to request and release the modes RUN and POST_RUN optionally.
[AUTOSAR] Description	If false, Run Request Protocol is not performed.

2.6.1.50 EcuM/EcuMGeneral/EcuMDevErrorDetect

[RTA-BSW] Description	Switches the development error detection and notification on or off. true: detection and notification is enabled. false: detection and notification is disabled.
[AUTOSAR] Description	Switches the development error detection and notification on or off.
[REMOVED] Introduction	* true: detection and notification is enabled. * false: detection and notification is disabled.

2.6.1.51 EcuM/EcuMGeneral/EcuMMainFunctionPeriod

[RTA-BSW] Description	This parameter defines the schedule period of EcuM_MainFunction. Unit is in Seconds.
[AUTOSAR] Description	This parameter defines the schedule period of EcuM_MainFunction.

2.6.1.52 EcuM/EcuMGeneral/EcuMVersionInfoApi

[RTA-BSW] Description	Switches the version info API, EcuM_GetVersionInfo, ON or OFF
[AUTOSAR] Description	Switches the version info API on or off

2.7 API Definition

List of supported AUTOSAR API

`EcuM_GetVersionInfo()`	Returns the version information of this module.
`EcuM_GoDown()`	Instructs the EcuM to perform a power off or a reset depending on the selected shutdown target.

EcuM_GoHalt()	Instructs the EcuM to go into a Halting sleep mode depending on the selected shutdown target.
EcuM_GoPoll()	Instructs the EcuM to go into a Polling sleep mode depending on the selected shutdown target.
EcuM_Init()	Initializes the EcuM and carries out the startup procedure.
EcuM_StartupTwo()	Implements the STARTUP II state.
EcuM_Shutdown()	Typically called from the shutdown hook. Takes over execution control and will carry out GO OFF II activities.
EcuM_SetState()	Called by BswM to change State.
EcuM_RequestRUN()	Places a request for the RUN state.
EcuM_ReleaseRUN()	Releases a RUN request previously done with a call to EcuM_RequestRUN.
EcuM_RequestPOST_RUN()	Places a request for the POST RUN state.
EcuM_ReleasePOST_RUN()	Releases a POST RUN request previously done with a call to EcuM_RequestPOST_RUN.
EcuM_SelectShutdownTarget()	Selects the shutdown target.
EcuM_GetShutdownTarget()	Retrieves the currently selected shutdown target as set by EcuM_SelectShutdownTarget.
EcuM_GetLastShutdownTarget()	Retrieves the shutdown target of the previous shutdown process.
EcuM_SelectShutdownCause()	Selects the cause of a shutdown.
EcuM_GetShutdownCause()	Retrieves the selected shutdown cause as set by EcuM_SelectShutdownCause.
EcuM_GetPendingWakeupEvents()	Retrieves pending wakeup events.
EcuM_ClearWakeupEvent()	Clears wakeup events.
EcuM_GetValidatedWakeupEvents()	Retrieves validated wakeup events.
EcuM_GetExpiredWakeupEvents()	Retrieves expired wakeup events.
EcuM_SelectBootTarget()	Selects a boot target.
EcuM_GetBootTarget()	Retrieves the current boot target.
EcuM_MainFunction()	Implements all activities of the EcuM while the OS is up and running.
EcuM_CheckWakeup()	Called periodically during Poll sequence if the MCU is not halted or when handling a wakeup interrupt.
EcuM_SetWakeupEvent()	Sets the wakeup event.
EcuM_ValidateWakeupEvent()	Indicate to the EcuM that the wakeup events have been validated.

For Further information please refer to AUTOSAR_SWS_ECUStateManager.pdf

2.8 Configuration Advice

> **NOTE**
>
> Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

2.8.1 EcuMModuleParameter

The PduR module does not support Postbuild Selectable or Postbuild Loadable, so the EcuMModuleParameter configuration parameter should be set to "NULL_PTR" for PduR if the Postbuild Variant is being used for other modules (e.g. CanTp, DoIp, J1939Tp).

2.8.2 EcuMRbModeSwitchQueueLength

This extension parameter specifies whether mode switch communication is queued or not, by specifying the length of the call queue on the sender side. The value must be greater than or equal to 1.

- A value of 1 represents unqueued communication, which means that a newly received value overwrites the previous value of the data item. If a value is sent multiple times before it is received, the receiver can only access the last transmitted value.
- Queued communication (with a value greater than 1) means that the interface queues arrivals of the data item on the receiver side.

The RTE generates the functionality of the mode queue in Rte_Switch based on the following aspects:

- The mode user has to configure a <R-PORT-PROTOTYPE> which is of type synchronous mode switch.
- The mode user has to configure a runnable entity of type <MODE-ACCESS-POINTS> triggered by a <SWC-MODE-SWITCH-EVENT>.

2.8.3 EcuMRbNvMBlockDeviceId

This extension paramter which if configured gets forwarded as the NvM parameter NvMNvramDeviceId.If the EcuM parameter EcuMRbNvMBlockDeviceId is not configured and FEE module is present, then NvMNvramDeviceId forwarded with a value of 0.

If FEE module not present and EA module is present, then NvMNvramDeviceId forwarded with a value of 1, along with those warning info message is passed.In case both module are not present only info message will passed, All other mandatory parameters have already been configured to be forwarded to NvM.Please

refer NvM documentation for further details.

2.8.3.1 EcuMRbRunMinimumDuration

This extension paramter holds the value as duration in seconds and if a non-zero value configured then after the startup EcuM will continue to stay in RUN for that particular duration even if there is no Run requests from the SW-C users.

2.8.4 Wakeup Sources

The following standard wakeup sources will be auto-generated with the EcuM_Prepare action:

- ECUM_WKSOURCE_POWER
- ECUM_WKSOURCE_RESET
- ECUM_WKSOURCE_INTERNAL_RESET
- ECUM_WKSOURCE_INTERNAL_WDG
- ECUM_WKSOURCE_EXTERNAL_WDG

There is a relation between the configured values of EcuMMainFunctionPeriod and EcuMValidationTimeout because the validation timeout is considered as EcuMMainFunctionPeriod rounded off to the nearest integer.

For example, if EcuMMainFunctionPeriod is 1 and EcuMValidationTimeout is 0.75, then 1 count of EcuMMainFunction will be taken. If EcuMMainFunctionPeriod is 1 and EcuMValidationTimeout is 0.45, then no validation timeout will be considered.

EcuM will set the wakeup source during initialization, which it will obtain as a “reset reason” from the MCU. If no MCU reset reason is defined, it will set the wakeup source as ECUM_WKSOURCE_RESET.

Note: If the McuResetReason configuration parameter in MCU is not configured, the reference can’t be given in EcuMResetReasonRef, so detecting Wakeup reasons with help of McuResetReason won’t be possible.

If the wakeup source configured belongs to a communication channel, the corresponding ComM channel should be referred in EcuMComMChannelRef.

2.8.5 Multi-Core Environments

A multi-core environment is established when OsNumberOfCores is configured with a value greater than one. If OsnumberOfCores is not configured, a single core environment is assumed.

The bulk of the BSW is placed on the master core, with only the sub-set of BSW modules needed to support the OS and SW-Cs running on the slave cores. ECU management on the slave cores must therefore also only support that subset of the BSW. When multicore is enabled, all the shutdown services and wakeupservices are callable in any core.

In EcuM sleep mode, individual core resources have to be configured in OS so that once EcuM has acquired the core resource it cannot be interrupted by other Os

tasks.

The EcuM resource name should be RES_AUTOSAR_ECUM_COREx, where 'x' (core ID) should start from 0 and be continuous up to the number of OS cores.

2.8.6 Synch Point

At least two cores must be configured with Synch Points before this feature can be enabled.

To configure a Synch Point from a particular core, a DriverInitItem with EcuMModuleID configured as EcuM, and EcuMModuleService as Synch_Point_<id> (where <id> must be unique for each core).

Usage of Synch Points from the Master core requires that EcuMRbSlaveCoreEarlierStart must be set to TRUE to ensure that the Slave core is initialized beforehand. If EcuMRbSlaveCoreEarlierStart is set to FALSE, Synch Points can be configured only among Slave cores.

2.8.7 Miscellaneous

- EcuMMainFunctionPeriod should always be configured using a float value.
- Although the Autosar specification says that all the declarations with callbacks and callouts should be maintained in EcuM_Cbk.h, separate header files for callouts and callbacks are needed to overcome the cyclic header inclusion.
- EcuM exclusive areas can be defined either through EcuM or RTE, and by enabling or disabling the EcuCRbLegacySchMInUse parameter.

2.9 Integration Advice

2.9.1 Init Functions

EcuM is initialized through EcuM_Init. In multi-core environments, EcuM_Init should be added to all slave core main tasks (and EcuM_StartupTwo to all slave core auto-start tasks).

2.9.2 Delnit Functions

- EcuM_GoDown (or EcuM_GoDownHaltPoll) can be called to de-initialise EcuM and start the shutdown phase.
- The default Shutdown target should be EcuMStateOff.
- EcuM_Shutdown should be called during the ShutdownHook callout. In multi-core environments, this is executed only in the EcuM on the master core.

2.9.3 Scheduled Functions

EcuM_MainFunction is called at a cyclic interval defined by the EcuM_MainFunctionPeriod configuration parameter (in EcuMGeneral).

The timing event for configuring this schedulable entity is generated via EcuM configuration in EcuM_Cfg_BSWMD.arxml.

If the OS has only a single core configured (OS configuration parameter osNumberOfCores) then the timing event is named as TE_EcuM_MainFunction. If there are multiple cores, then the naming is TE_EcuM_MainFunction_Core<n>, where n varies from 0 to the number of configured cores, minus 1.

For mapping this timing event, the RteBswModuleInstance must first be created for EcuM in the Rte configuration file, and then, based on the number of Os cores, RteBswEventToTaskMapping containers need to be created.

EcuM_MainFunction has to run in all cores in a multi-core environment, so a timing event for each core has to be mapped to respective Core Os tasks.

- If the RTE is not used for mapping RTE Tasks to OS Tasks, then you must ensure that EcuM_Mainfunction is running on every core. This is needed for shutdown synchronization and for initialization of the BSW.
- For an RTE generator based on AR 4.0, a BSW module cannot be configured on multiple OS applications, which means that the above mentioned timing event mapping will throw an error. To solve this issue, if core shutdown is not done by EcuM, the timing event mapping can be removed from the configuration. If shutdown is done via EcuM, remove the above configuration and manually schedule EcuM_MainFunction on each core.

For APIs related to wakeup events, note that EcuM_ClearWakeupEvents must be called before EcuM_GoDownHaltPoll, EcuM_GoDown, EcuM_GoPoll and EcuM_GoHalt are called.

2.9.4 Define modules to be Initialised

The BSW modules and drivers to be initialised by `EcuM_AL_DriverInitZero()` and `EcuM_AL_DriverInitOne()` should be specified within ISOLAR-AB.

2.9.5 Mcu Reset Reason

EcuM has a dependency on the configuration of Mcu Reset. During initialization, EcuM will check the last Mcu Reset reason (via Mcu_GetResetReason) and maps the Reset to the corresponding Wakeup source (see the EcuMWakeupSource container).

2.9.6 Mcu Set Mode

The Sleep operation of EcuM has a dependency on the Mcu_SetMode service to switch the microcontroller to a different MCU operation mode to achieve the lower power consumption. Without the Mcu_SetMode service, the EcuM Sleep operation cannot be achieved.

2.9.7 Run Time Measurement

In order to use the Run time measurement feature in EcuM, the EcuMRbStartupDuration parameter has to be set to TRUE (the ECUM_STARTUP_DURATION macro will be generated in EcuM_Cfg.h as TRUE).

The MCU module must support the system timer feature, and the rba_BswSrv and rba_Reset modules must be present. The file EcuM_RunTime.h is required.

The EcuM_StartCheckWakeup function is provided as a callout. The EcuM provided code in EcuM_StartCheckWakeup should not be modified, and any code additions should follow after the default provided code. EcuM_StartCheck-Wakeup must be called from EcuM_CheckWakeup.

2.9.8 RTE Generator

If the project includes an RTE generator, The EcuC parameter EcuCRbRteInUse has to be set to TRUE. EcuM will expect the actual RTE generated files.

If the project does not include an RTE generator, EcuCRbRteInUse has to be set to FALSE, and RTE relevant information will be used from the code stub which is directly available in EcuM.

2.9.9 Wakeup State Changes (BswM)

When a wakeup state change occurs (e.g. from Pending to Validated) this is indicated to BswM via BswM_EcuM_CurrentWakeup. If any particular action needs to be executed with each wakeup source state change, you can configure this in BswM.

2.9.10 EcuMDriverInitListBswM container (BswM)

The EcuMDriverInitListBswM container is provided to initialize modules from BswM. Note that the driver init list configured here is pre-compile. If configured, EcuMDriverInitListBswM generates EcuM_AL_DriverInitBswM_<x> callback functions, which will be called by BswM. The EcuMRbSequenceID parameter is provided, which if configured, generates the init functions as per EcuMRbSequenceID. If EcuMRbSequenceID is not configured, init functions are called in the order of their configuration.

2.9.11 Wakeup Source (ComM)

If a communication channel is configured as a wakeup source, then from the corresponding wakeup source configuration its ComM channel should be referred. If that source is detected and validated, EcuM will give the wakeup indication to the respective channel (via ComM_EcuM_WakeupIndication). Note that EcuM is only gives an indication to ComM and BswM if a source is validated, and it's your responsibility to configure the necessary action in BswM to set the channel to full communication mode after detecting the EcuM indication.

2.9.12 EcuM_GetLastShutdownTarget (NvM)

The EcuM_GetLastShutdownTarget service has a dependency on the configuration of the NvM module.

- A dedicated block for EcuM has to be configured in NvM. This block should have the name ECUM_CFG_NVM_BLOCK.
- The EcuMRbNvMBlockDeviceId extension parameter (in EcuMGeneral) identifies the device where the NVRAM block named ECUM_CFG_NVM_BLOCK is

located. This block is forwarded during CodeGen to NvM (NvMNvramDeviceId). Device ID 0 is fixed to address Fee and device IDs 1 onwards address Eep devices. If this parameter is not configured, Device ID 0 (Fee) will be forwarded by default.

- All other mandatory parameters have already been configured to be forwarded to NvM. Please refer to the NvM documentation for further details.

For this service to function correctly, NvM_ReadAll has to be called from the startup section after the basic initialisation is completed. This can be triggered from BswM immediately after the NvM_Init. Once it is completed successfully, the callback EcuM_Rb_NvMSingleBlockCallbackFunction is called by NvM.

2.9.13 EcuM_Rb_GetLastShutdownInfo (NvM and Mcu)

The EcuM_Rb_GetLastShutdownInfo service has a dependency on the Mcu and the configuration of the NvM module.

- The Mcu has to provide the Mcu_Rb_GetSysTime service.
- The module rba_BswSrv has to be present to provide the measurement capability for the time taken to startup since the previous reset.
- A dedicated block for EcuM has to be configured in NvM. This block should have the name ECUM_CFG_NVM_BLOCK (i.e the same block that is used for storing the previous shutdown target and mode information - see EcuM_GetLastShutdownTarget above) and the value for element Device ID for ECUM_CFG_NVM_BLOCK needs to be configured, as described above.

See also the notes under EcuM_GetLastShutdownTarget for the calling of NvM_ReadAll from the startup section after the basic initialisation is completed.

2.9.14 EcuM_StartCheckWakeup

The EcuM provided code in EcuM_StartCheckWakeup should not be modified. Any code addition should follow after the default provided code. EcuM_StartCheckWakeup must be called from EcuM_CheckWakeup.

EcuM should call BswM_EcuM_CurrentWakeup (ECUM_WKSTATUS_DISABLED) in the wakeup restart sequence since ECUM_WKSTATUS_DISABLED is not defined in EcuM_WakeupStatusType (and so this function call is removed).

ECUM_WKSTATUS_ENABLED is not a possible value of EcuM_WakeupStatusType so this value is removed.

2.9.15 Implemented Callouts

The EcuM module can be configured to make the following call-outs into integrator code.

- EcuM_ErrorHook: Called by EcuM for serious error conditions where it is not possible to continue processing and Ecu has to be stopped.
- EcuM_AL_SetProgrammableInterrupts: Enables the interrupts on ECUs with

programmable interrupts.

- EcuM_AL_DriverInitZero: Provides driver initialization and other hardware-related startup activities for loading the post-build configuration data.
- EcuM_AL_DriverInitOne: Provides driver initialization and other hardware-related startup activities in the event of a power on reset.
- EcuM_AL_DriverRestart: Provides driver re-initialization and other hardware-related startup activities in Wakeup.
- EcuM_DeterminePbConfiguration: Sets and returns a pointer to the post-build configuration data.
- EcuM_OnGoOffOne: Notifies that the GO OFF I state has been reached.
- EcuM_OnGoOffTwo: Notifies that the GO OFF II state has been reached.
- EcuM_AL_SwitchOff: Shuts off the power supply of the ECU.
- EcuM_AL_Reset: Resets the ECU.
- EcuM_CheckWakeup: Called by a driver when a wakeup is detected.
- EcuM_StartWakeupSources: Starts the wakeup sources for validation.
- EcuM_StopWakeupSources: Stops the wakeup sources after an unsuccessful validation.
- EcuM_CheckValidation: Validates a wakeup source.
- EcuM_SwitchOsAppMode: Called from EcuM_Init in both Master and Slave core, required to support the Calibration Wakeup feature.
- EcuM_EnableWakeupSources: Called by EcuM to allow notify wakeup sources defined in the wakeupSource bitfield that SLEEP will be entered, and to adjust the source accordingly.
- EcuM_DisableWakeupSources: Called by EcuM to set the wakeup source(s) defined in the wakeupSource bitfield so that they are not able to wake up the ECU.
- EcuM_CheckRamHash: Provides a RAM integrity test, the goal of which is to ensure that after a long SLEEP duration, RAM contents are still consistent.
- EcuM_GenerateRamHash: Calculates the Hash/checksum of RAM contents.
- EcuM_StartCheckWakeup: Called by the ECU Firmware to start the CheckWakeupTimer for the corresponding WakeupSource.
- EcuM_SleepActivity: Invoked periodically in all reduced clock sleep modes to poll wakeup sources and to call wakeup notification functions to indicate the end of the sleep state to the ECU State Manager.

2.9.16 Additional step to configure EcuMDriverInitItem for MCAL modules

When an EcuMDriverInitItem (a module initialization function call) is added under EcuMDriverInitListOne and EcuMModuleParameter is POSTBUILD_PTR, RTA-BSW Code Generator (CodeGen) generates a default pointer of configuration structure named `<module>_Config`. However, the name of configuration structure is defined by another module. It may be different with `<module>_Config`.

For example:

Configuration of EcuMDriverInitItem “Can_Init” as image below

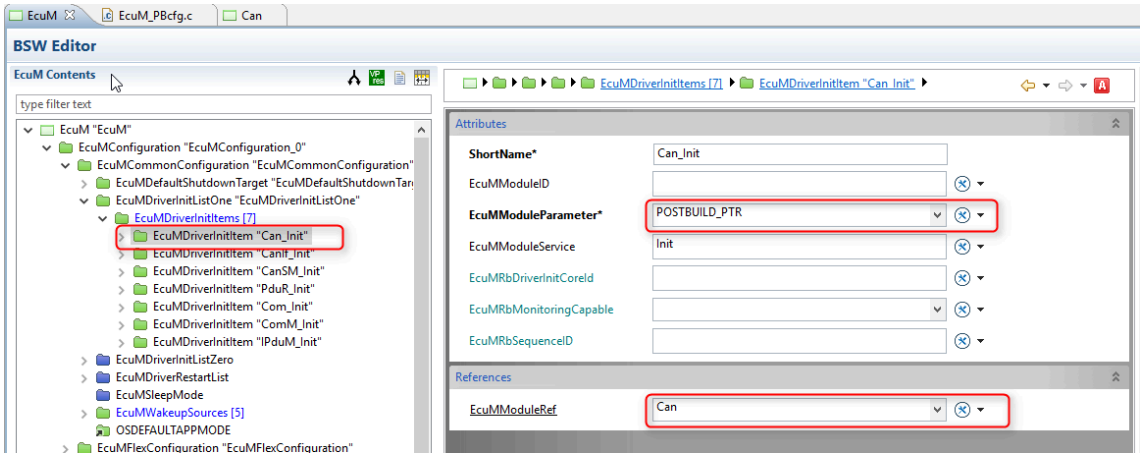


Figure 2.2.Configuration of EcuMDriverInitItem “Can_Init”

CodeGen will generate the name as `Can_Config` for the configuration structure of Can module.

```
CONST( EcuM_ConfigType, ECUM_CONST ) EcuM_Config =
{
    OSDEFAULTAPPMODE,
    {
        ECUM_SHUTDOWN_TARGET_OFF    ,       0    ,
#if (ECUM_STARTUP_DURATION == TRUE) /*will activate the Run time measurement*/
0x00,
0x00
#else
0x00
#endif//ECUM_STARTUP_DURATION
    }, /* DefaultShutdownTarget */
    {
            &Can_Config    /*   Can    */,
            NULL_PTR    /*   CanIf    */,
            NULL_PTR    /*   CanSM    */,
            &PduR_Config    /*   PduR    */,
            NULL_PTR    /*   Com    */,
            NULL_PTR    /*   ComM    */,
            NULL_PTR    /*   IpduM    */,

    &BswM_Config    /*BswM*/
    },
        &EcuM_Cfg_dataWkupPNCRef_cast[0], /* Pointer which refers to ComMPNC references associated with
wakeups */
    {
        0xD4 , 0x1D , 0x8C , 0xD9 , 0x8F , 0x00 , 0xB2 , 0x04 , 0xE9 , 0x80 , 0x09 , 0x98 , 0xEC , 0xF8
, 0x42 , 0x7E
    }
};
```

However, in the MCAL CAN module the configuration structure has a different name not `Can_Config`. The third party CAN module defined this configuration structure as Can_ConfigData, therefore build is not successfull as the above code has an undefined reference. *This problem will also occur with other MCAL modules*.

In this case, an integration header file should be created to redefine the names of the <module>_Config(s) to the real ones defined by the owner modules

- Create new header file named `EcuM_User.h`.

- Redefine `Can_Config` with corresponding configuration structure of MCAL modules.

```
#ifndef  _ECUM_USER_H
#define  _ECUM_USER_H

#define Can_Config Can_ConfigData

#endif /* _ECUM_USER_H */
```

- Add the name of the header file to the `EcuMRbIncludeHeader` container

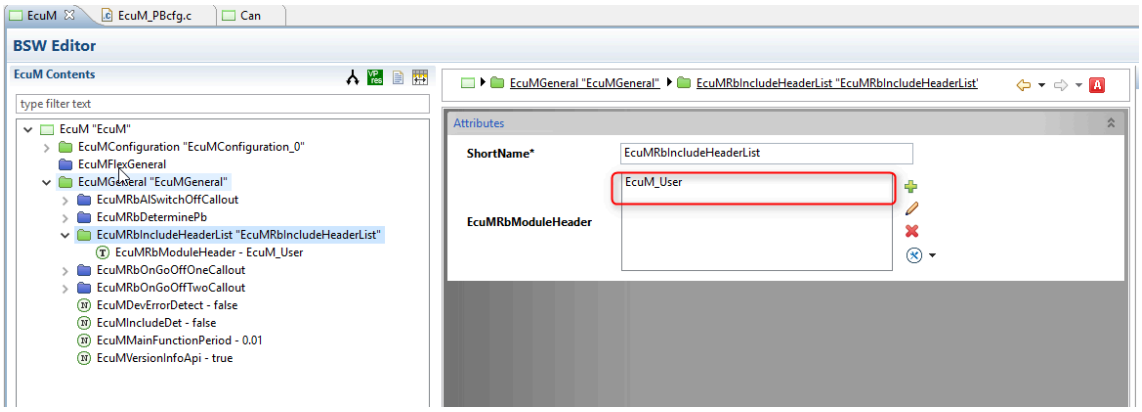


Figure 2.3.EcuMRbIncludeHeaderList configuration

2.9.17 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The EcuM module uses the following OS resources:

- EcuM
- EcuM

2.9.18 SchM_Start_Integration and SchM_StartTiming_Integration

SchM_Start_Integration and SchM_StartTiming_Integration (located in EcuM_Cfg_SchM.h) are integration macros to support the SchM_Start and SchM_StartTiming actions described in SWS_EcuM_02932 in AUTOSAR_SWS_ECUStateManager.pdf for AR 4.3.1 or higher. If the RTE gener-

atorcan generate SchM_Start and SchM_StartTiming (i.e it can support AR 4.3.1 or higher), SchM_Start_Integration and SchM_StartTiming_Integration must be defined by the RTE-generated SchM_Start and SchM_StartTiming. By default, SchM_Start_Integration and SchM_StartTiming_Integration are empty macros.

2.10 Hardware Requirements

Table 2.145.EcuM Hardware Requirements

Support	Usage
FLASH	4605 bytes
RAM	76 bytes

Usage calculated based on a sample configuration.

3 Synchronized Time-Base Manager

Module name	StbM
Package	RTA-BASE
AUTOSAR Module ID	160
AUTOSAR Version	22-11

3.1 Introduction

The purpose of the Synchronized Time-Base Manager (Stbm) is to provide synchronized time bases to its customers - i.e. time bases that are synchronized with time bases on other nodes of a distributed system.

The two main functions of the StbM are:

- Synchronization of Runnable Entities.
- Provision of an absolute time value.

3.1.1 Interaction with other Modules

Stbm interacts with the following modules.

OS

If OS is configured as a reference for local timebase (or if a triggered customer is configured), StbM uses OS to get the local time.

EthTSyn

If Eth is configured as a reference for local timebase, the timevalue is taken from EthTSyn

EthIf

If EthIf is configured as a reference for local timebase, the timevalue is taken from EthIf.

GPT

If GPT is configured as a reference for local timebase, the timevalue is taken from GPT.

NvM

During the initialization of StbM, if NvM is configured, the values are loaded from NvM. During shutdown, if NvM is configured, values are stored into NvM.

Module Configuration

This module is not currently supported by the RTA-BSW Configuration Generator (ConfGen) and must be manually configured.

3.2 Supported Features

- StbM provides absolute or relative time values to its users. The time values might be synchronized with other ECUs via communication bus systems like CAN/Ethernet/Flexray.

- An arbitrary number of RunnableEntities can be executed synchronously. Synchronous means that they start with a well-defined and guaranteed relative offset (e.g. relative offset “0” means the execution shall occur at the same point in time).

StbM also supports:

- Synchronized Time Bases
- Synchronization State within Global Time Master
- Synchronization State Supervision within Time Slaves
- Synchronization State Supervision within Time Gateways
- Offset Time Bases
- Pure Local Time Bases
- Time Gateway
- Customers (Active/Triggered/Notification)
- AUTOSAR Service Interface

3.2.1 Additional non-AUTOSAR Features

- Time Recording
- Rate Correction
- Offset Correction
- Time Correction
- Time Notification

See the Configuration Advice section for more details on these additional features.

3.3 Extensions

3.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

3.3.1.1 StbM/StbMFreshnessValueInformation/StbMFreshnessValue/ StbMProvideTxTruncatedFreshnessValue

Short Name	StbMProvideTxTruncatedFreshnessValue
Description	This parameter specifies if the FVM shall provide the truncated freshness value directly or if the FV shall be truncated by the StbM.
Introduction	* true: freshness truncation is done by FVM. * false: freshness truncation is done by StbM.
Multiplicity	1..1

Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

3.3.1.2 StbM/StbMGeneral/StbMRbNvMBlockReference

Short Name	StbMRbNvMBlockReference
Description	ETAS Specific parameter (vendorID: 11u): Reference of the Nvm block that should be configured for the timebase.
Introduction	In this case, the designated NvM block has to be configured correctly, meaning: For each timebase one NvM block has to be configured.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

3.3.1.3 StbM/StbMRbExternalCounter

Short Name	StbMRbExternalCounter
Description	ETAS Specific parameter (vendorID: 11u): References the External Counter for a SynchronizedTimeBase
Multiplicity	0..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

3.3.1.4 StbM/StbMRbExternalCounter/StbMRbExternalCounterGetCounterValueCallback

Short Name	StbMRbExternalCounterGetCounterValueCallback
Description	ETAS Specific parameter (vendorID: 11u): Name of the Hardware counter callback routine which shall be invoked to fetch the current counter value if external counter is configured as true. User needs to enter a valid name e.g GPT_Get_CounterValue_Cbk
Multiplicity	1..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

3.3.1.5 StbM/StbMRbExternalCounter/StbMRbExternalCounterGetElapsedValueCallback

Short Name	StbMRbExternalCounterGetElapsedValueCallback

Description	ETAS Specific parameter (vendorID: 11u): Name of the Hardware counter callback routine which shall be invoked to fetch the elapsed counter value if external counter is configured as true. User needs to enter a valid name e.g GPT_Get_ElapsedValue_Cbk
Multiplicity	1..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

3.3.1.6 StbM/StbMRbExternalCounter/StbMRbExternalCounterNanosecondsToTicksCallback

Short Name	StbMRbExternalCounterNanosecondsToTicksCallback
Description	ETAS Specific parameter (vendorID: 11u): Name of the Hardware counter callback routine which shall be invoked for conversion of the nanoseconds to ticks if external counter is configured as true. User needs to enter a valid name e.g GPT_Ns_To_Tick_Cbk
Multiplicity	0..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

3.3.1.7 StbM/StbMRbExternalCounter/StbMRbExternalCounterTicksTonsCallback

Short Name	StbMRbExternalCounterTicksTonsCallback
Description	ETAS Specific parameter (vendorID: 11u): Name of the Hardware counter callback routine which shall be invoked for conversion of the ticks to nanoseconds if external counter is configured as true. User needs to enter a valid name e.g GPT_Tick_To_Ns_Cbk
Multiplicity	1..1
Default Value	-
Type	ECUC-FUNCTION-NAME-DEF

3.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

3.3.2.1 StbM_GetMasterConfig

Syntax	`StbM_GetMasterConfig(StbM_SynchronizedTimeBaseType timeBaseId, StbM_MasterConfigType* masterConfig)`

Parameters In	`timeBaseId` Time base reference.
Parameters Out	`masterConfig` Indicates if system-wide master functionality is supported.
Description	Indicates if the functionality for a system-wide master (e.g. StbM_SetGlobal-Time) for a given Time Base is available or not.

3.3.2.2 StbM_TriggerTimeTransmission

Syntax	`StbM_TriggerTimeTransmission( StbM_SynchronizedTimeBaseType timeBaseId )`
Parameters In	`timeBaseId` Time base reference.
Description	Called by the <Upper Layer> to force the Timesync Modules to transmit the current Time Base again due to an incremented timeBaseUpdateCounter[-timeBaseId].

3.3.2.3 StbM_UpdateGlobalTime

Syntax	`StbM_UpdateGlobalTime(VAR(StbM_SynchronizedTimeBaseType,AUTOMATIC) timeBaseId, P2CONST(StbM_TimeStampType,AUTOMATIC,STBM_APPL_DATA) timeStamp, P2CONST(StbM_UserDataType,AUTOMATIC,STBM_APPL_DATA) userData)`
Parameters In	`timeBaseId` Refers to the TimebaseID for which Global time will be set. `timeStamp` Points to the structure with the updated time value that has to be set. `userData` Points to the structure with the updated user data that has to be set.
Description	This function allows the setting of the new global time, which has to be valid for the system, and which will be sent to the busses and modify HW registers behind the providers, if supported.

3.3.2.4 StbM_SetRateCorrection

Syntax	`StbM_SetRateCorrection( StbM_SynchronizedTimeBaseType timeBaseId, const StbM_RateDeviationType rateDeviation )`
Parameters In	`timeBaseId` Refers the TimebaseID for which offset time value will be set. `rateDeviation` Refers to value applied as rate deviation.
Description	Sets the rate of a Synchronized Time Base (e.g. either a Pure Local Time Base or not).

3.3.2.5 StbM_GetTimeLeap

Syntax	`StbM_GetTimeLeap( StbM_SynchronizedTimeBaseType timeBaseId, StbM_TimeDiffType* timeJump )`

Parameters In	`timeBaseId` Refers the TimebaseID for which Time Leap value will be set. `timeJump` Refers to the user variable that will receive the value of Time Leap if StbMTimeLeapFuture/PastThreshold is exceeded.
Description	Returns value of Time Leap if StbMTimeLeapFuture/PastThreshold is exceeded.

3.3.2.6 StbM_GetSyncTimeRecordHead

Syntax	`StbM_GetSyncTimeRecordHead( StbM_SynchronizedTimeBaseType timeBaseId, StbM_SyncRecordTableHeadType* syncRecordTableHead )`
Parameters In	`timeBaseId` Time base reference.
Parameters Out	`syncRecordTableHead` Refers to Synchronized Time Base Record Table Header.
Description	Accesses the recorded snapshot data Header of the table belonging to the Synchronized Time Base.

3.3.2.7 StbM_GetOffsetTimeRecordHead

Syntax	`StbM_GetOffsetTimeRecordHead( StbM_SynchronizedTimeBaseType timeBaseId, StbM_OffsetRecordTableHeadType* offsetRecordTableHead )`
Parameters In	`timeBaseId` Time base reference.
Parameters Out	`offsetRecordTableHead` Refers to Offset Time Base Record Table Header.
Description	Accesses the recorded snapshot data Header of the table belonging to the Offset Time Base.

3.3.2.8 StbM_GetRateDeviation

Syntax	`StbM_GetRateDeviation( StbM_SynchronizedTimeBaseType timeBaseId, StbM_RateDeviationType* rateDeviation )`
Parameters In	`timeBaseId` Refers the TimebaseID for which Global time will be set.
Parameters Out	`rateDeviation` Gives the value of the current rate deviation of a Time Base.
Description	Returns value of the current rate deviation of a Time Base.

3.3.2.9 StbM_StartTimer

Syntax	`StbM_StartTimer( StbM_SynchronizedTimeBaseType timeBaseId, StbM_CustomerIdType customerId, const StbM_TimeStampType* expireTime )`

Parameters In	`timeBaseId` Refers the TimebaseID for which Global time will be set.
	`customerId`
	`expireTime`

Description	Sets a time value, which the Time Base value is compared against.

3.3.2.10 StbM_GetTimeBaseStatus

Syntax	`StbM_GetTimeBaseStatus( StbM_SynchronizedTimeBaseType timeBaseId, StbM_TimeBaseStatusType* syncTimeBaseStatus, StbM_TimeBaseStatusType* offsetTimeBaseStatus )`
Parameters In	`timeBaseId` Time base reference.
Parameters Out	`syncTimeBaseStatus` Status of the Synchronized Time Base.
Parameters Out	`offsetTimeBaseStatus` Status of the Offset Time Base.
Description	Returns the detailed status of the Time Base. For Offset Time Bases the status of the Offset Time Base itself and the status of the underlying Synchronized Time Base is returned.

3.3.2.11 StbM_GetTimeBaseUpdateCounter

Syntax	`StbM_GetTimeBaseUpdateCounter( StbM_SynchronizedTimeBaseType timeBaseId )`
Parameters In	`timeBaseId` Refers the TimebaseID for which Global time will be set.
Description	Allows the Timesync modules to detect whether a Time Base should be transmitted immediately in the subsequent <Bus>TSyn_MainFunction() cycle.

3.3.2.12 StbM_GetCurrentVirtualLocalTime

Syntax	`StbM_GetCurrentVirtualLocalTime( StbM_SynchronizedTimeBaseType timeBaseId, StbM_VirtualLocalTimeType * localTimePtr)`
Parameters In	`timeBaseID` Refers to the TimebaseID for which Global time will be set.
Parameters Out	`localTimePtr` Holds the current Virtual Local Time value.
Description	Returns the Virtual Local Time of the referenced Time Base.

3.3.2.13 StbM_BusGetCurrentTime

Syntax	`StbM_BusGetCurrentTime(StbM_SynchronizedTimeBaseType timeBaseId, StbM_TimeStampType * globalTimePtr, StbM_VirtualLocalTimeType * localTimePtr, StbM_UserDataType * userData)`
Parameters In	`timeBaseID` Refers to the TimebaseID for which Global time will be set.

Parameters Out	`globalTimePtr` Holds the value of the local instance of the Global Time, which is sampled when the function is called. `localTimePtr` Holds the current Virtual Local Time value. `userData` Holds the User data of the Time Base.
Description	Returns the current Time Tuple, status and User Data of the Time Base.

3.4 Limitations

- StbM only supports 32-bit wide free-running upcounting counters as the local time clock. The counter must leverage the full uint32 value range (Hex 0x0…0xFFFFFFFF / Decimal 0…4294967295).

- The StbM Autosar requirement SWS_StbM_00353 is not fully implemented. Therefore if TimeCorrection - RateAdaption is configured and applied, then after the Adaption Interval period of time, instead of updating TV = TVTemp = TVSync + StbMOffsetCorrectionAdaptionInterval, the current virtual local time is read and used as TV/TVTemp. This is done since updating Os/Gpt/ExternalCounters with specific ticks is not straightforward simple and requires further analysis.

- The StbM Freshness Management feature is being delivered as a prototype version in this release.

3.5 Exclusions

The following features from AR v22-11 are not supported in version v12.7.0.

API functions not currently implemented:

- `StbM_CanSetSlaveTimingData`
- `StbM_FrSetSlaveTimingData`
- `StbM_EthSetSlaveTimingData`
- `StbM_CanSetMasterTimingData`
- `StbM_FrSetMasterTimingData`
- `StbM_EthSetMasterTimingData`
- `StbM_EthSetPdelayInitiatorData`
- `StbM_EthSetPdelayResponderData`
- `StbM_GetCurrentSafeTime`
- `StbM_GetFallbackVirtualLocalTime`

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- StbM/StbMFreshnessValueInformation/StbMFreshnessValue/StbMFreshnessValueTruncLength

- StbM/StbMGeneral/StbMEcucPartitionRef
- StbM/StbMSynchronizedTimeBase/StbMTimeValidation
- StbM/StbMSynchronizedTimeBase/StbMTimeValidation/StbMTimeValidationRecordTableBlockCount

3.6 Deviations

3.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

3.6.1.1 StbM

[RTA-BSW] Description	The purpose of the Synchronized Time-Base Manager is to provide synchronized time bases to its customers, i.e., time bases, which are synchronized with time bases on other nodes of a distributed system.
[AUTOSAR] Description	Configuration of the Synchronized Time-base Manager (StbM) module.

3.6.1.2 StbM/StbMFreshnessValueInformation

[RTA-BSW] Description	Container with the Freshness Value configurations.
[AUTOSAR] Description	Container with the Freshness Value configurations Tags: atp.Status=draft

3.6.1.3 StbM/StbMFreshnessValueInformation/StbMFreshnessValue

[RTA-BSW] Description	Container with the Freshness Value configurations.
[AUTOSAR] Description	Container with the Freshness Value configurations Tags: atp.Status=draft
[ADDED] Infinite Multiplicity	1
[REMOVED] Upper Multiplicity	*

3.6.1.4 StbM/StbMFreshnessValueInformation/StbMFreshnessValue/StbMFreshnessValueId

[RTA-BSW] Description	This parameter defines the Id of the Freshness Value. The Freshness Value might be a normal counter or a time value.
[AUTOSAR] Description	This parameter defines the Id of the Freshness Value. The Freshness Value might be a normal counter or a time value. Tags: atp.Status=draft

3.6.1.5 StbM/StbMFreshnessValueInformation/StbMFreshnessValue/StbMFreshnessValueLength

[RTA-BSW] Description	This parameter defines the complete length in bits of the Freshness Value. As long as the key doesn't change the counter shall not overflow. The length of the counter shall be determined based on the expected life time of the corresponding key and frequency of usage of the counter.
[AUTOSAR] Description	This parameter defines the complete length in bits of the Freshness Value. As long as the key doesn't change the counter shall not overflow. The length of the counter shall be determined based on the expected life time of the corresponding key and frequency of usage of the counter. Tags: atp.Status=draft

3.6.1.6 StbM/StbMFreshnessValueInformation/StbMGetRxFreshnessValueFuncName

[RTA-BSW] Description	Function pointer to call within StbM_GetRxFreshness() context.
[AUTOSAR] Description	Function pointer to call within StbM_GetRxFreshness() context. Tags: atp.Status=draft

3.6.1.7 StbM/StbMFreshnessValueInformation/StbMGetTxConfFreshnessValueFuncName

[RTA-BSW] Description	Function pointer to call within StbM_SPduTxConfirmation() context.
[AUTOSAR] Description	Function pointer to call within StbM_SPduTxConfirmation() context. Tags: atp.Status=draft

3.6.1.8 StbM/StbMFreshnessValueInformation/StbMGetTxFreshnessValueFuncName

[RTA-BSW] Description	Function pointer to call within StbM_GetTxFreshness() context.
[AUTOSAR] Description	Function pointer to call within StbM_GetTxFreshness() context. Tags: atp.Status=draft

3.6.1.9 StbM/StbMFreshnessValueInformation/StbMGetTxTruncFreshnessValueFuncName

[RTA-BSW] Description	Function pointer to call within StbM_GetTxFreshnessTruncData() context.

[AUTOSAR] Description	Function pointer to call within StbM_GetTxFreshnessTruncData() context. Tags: atp.Status=draft

3.6.1.10 StbM/StbMFreshnessValueInformation/StbMQueryFreshnessValue

[RTA-BSW] Description	This parameter specifies if the freshness value shall be determined through a C-function (CD) or a software component (SW-C).
[AUTOSAR] Description	This parameter specifies if the freshness value shall be determined through a C-function (CD) or a software component (SW-C). Tags: atp.Status=draft

3.6.1.11 StbM/StbMGeneral/StbMGetCurrentTimeExtendedAvailable

[RTA-BSW] Description	This allows to define whether an additional variant of the API GetCurrentTime with a 64 bit argument is provided.
[AUTOSAR] Description	This allows to define whether an additional variant of the API GetCurrentTime with a 64 bit argument is provided. Tags: atp.Status=obsolete

3.7 API Definition

Table 3.33.StbM Supported API functions

`StbM_Init`	Initializes the Synchronized Time-base Manager.
`StbM_GetVersionInfo`	Returns the version information of this module.
`StbM_GetCurrentTime`	Returns a time value (Local Time Base derived from Global Time Base) in standard format.
`StbM_GetCurrentTimeExtended`	Returns a time value (Local Time Base derived from Global Time Base) in extended format.
`StbM_SetGlobalTime`	Sets the new global time that has to be valid for the system (which will be sent to the busses and modify HW registers behind the providers) if supported.
`StbM_SetUserData`	Allows the Customer to set the new User Data that has to be valid for the system that will be sent to the busses.
`StbM_SetOffset`	Allows the Customers and the Timebase Provider Modules to set the offset time that has to be valid for the system.
`StbM_GetOffset`	Allows the Timebase Provider Modules to get the currentoffset time.

StbM_BusGetCurrentTime	Returns the current Time Tuple, status and User Data of the Time Base.
StbM_BusSetGlobalTime	Allows the Timebase Provider Modules to forward a new Global Time to the StbM that has been received from different busses.
StbM_MainFunction	This function will be called cyclically by a task body provided by the BSW Schedule.
StbM_GetCurrentVirtualLocalTime	Returns the Virtual Local Time of the referenced Time Base.
StbM_UpdateGlobalTime	Allows the Customers to set the Global Time that will be sent to the buses.
StbM_GetRateDeviation	Returns value of the current rate deviation of a Time Base.
StbM_SetRateCorrection	Allows to set the rate of a Synchronized Time Base (being either a Pure Local Time Base or not).
StbM_GetTimeLeap	Returns value of Time Leap.
StbM_GetTimeBaseStatus	Returns detailed status information for a Synchronized (or Pure Local) Time Base. Also if called for an Offset Time Base, for the Offset Time Base and the underlying Synchronized Time Base.
StbM_CloneTimeBase	Copies Time Base data (current time/user data/rate corrction) from Source Time Base to Destination Time Base.
StbM_StartTimer	Sets a time value of which the Time Base value is compared against.
StbM_GetSyncTimeRecordHead	Accesses to the recorded snapshot data Header of the table belonging to the Synchronized Time Base.
StbM_GetOffsetTimeRecordHead	Accesses to the recorded snapshot data Header of the table belonging to the Offset Time Base.
StbM_TriggerTimeTransmission	Called by the <Upper Layer> to force the Timesync Modules to transmit the current Time Baseagain due to an incremented timeBaseUpdateCounter[timeBaseId].
StbM_GetTimeBaseUpdateCounter	Allows the Timesync Modules to detect, whether a Time Base should be transmitted immediately in the subsequent <Bus>TSyn_MainFunction() cycle.
StbM_GetMasterConfig	Indicates if the functionality for a system wide master (e.g. StbM_SetGlobalTime) for a given Time Base is available or not.

StbM_GetBusProtocolParam	This API is used to get bus specific parameters from received Follow_Up message.
StbM_SetBusProtocolParam	This API is used to set bus specific parameters of a Time Master.
StbM_GetTxFreshness	Returns the freshness value from the most significant bits in the first byte of the freshness array (in Big Endian format).
StbM_GetTxFreshnessTruncData	Used by the StbM to obtain the current freshness value. It also provides the truncated freshness transmitted in the secured time sync message.
StbM_SPduTxConfirmation	This interface is used by the StbM to indicate that the Secured Time Synchronization Message has been initiated for transmission.
StbM_GetRxFreshness	Used by the StbM to query the current freshness value.

For further information on these functions, please refer to AUTOSAR_SWS_SynchronizedTimeBaseManager.pdf

3.8 Configuration Advice

> **NOTE**
>
> Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

3.8.1 Mandatory Float Values

The following parameters must always be configured using a float value.

- StbMMainFunctionPeriod
- StbMTriggeredCustomerPeriod
- StbMSynchronizedTimeBaseRef

3.8.2 StbM_GetCurrentTimeExtended

The use of the StbM_GetCurrentTimeExtended API is not currently recommended as it uses a structure which has a parameter defined as uint64 (and uint64 variable usage is not recommended). For this reason, the configuration parameter StbMGetCurrentTimeExtendedAvailable should be kept as FALSE.

3.8.3 StbMTimeNotificationCallback

For the Time Notification feature, an Internal Triggered Occured Event is configured to decouple the context from Interrupt to Task. This is applicable only when the StbMTimeNotificationCallback parameter is NULL and RTE is used. If StbMTimeNotificationCallback has been configured, the configured callback function will be executed in Interrupt context.

3.8.4 StbMStatusNotificationCallback

If StbMStatusNotificationCallback is not NULL, StbM will call the configured callback function to notify about Status related Events. If StbMStatusNotificationCallback is NULL and RTE is used, StbM will notify the application via the StatusNotification service interface.

3.8.5 NvM Configuration

If NvM is configured for a timebase, the following configuration of NvM is required.

- NvMNvBlockLength should be in multiples of 11* STBM_CFG_NO_OFTIMEBASES_NVM.
- NvMRamBlockDataAddress should be StbM_TimesourceArray_Nvm_ast
- NvMSelectBlockForReadAll should be True
- NvMSelectBlockForWriteAll should be True

3.8.6 Configuration of the LocaltimeRef of the Timebase

The Frequency and Prescalar for the LocaltimeRef of the Timebase should be configured correctly. For a ticks-to-nanoseconds conversion, the calculation is done as follows.

- It is assumed that StbM_Mainfunction is called in a < =100ms task, the Frequency is Max 100MHz, and the Prescalar is 8.
- Ns = (ticks * ((1000000000000/Frequency)* Prescalar)).
- 1000000000000 is taken into consideration to have a good precision of nanoseconds.
- The Frequency and Prescalar values are used to configure the HW counter.

3.8.7 Time Recording Feature

If StbMTimeRecording is not NULL and either StbMSyncTimeRecordBlockCallback (for Synchronized TimeBase) or StbMOffsetTimeRecordBlockCallback (for Offset TimeBase) is configured, StbM will call the configured callback function to pass all blocks with their elements to the application. If StbMTimeRecording is not NULL and StbMSyncTimeRecordBlockCallback and StbMOffsetTimeRecordBlockCallback are NULL and RTE is used, StbM will notify about the availability of a new recorded measurement data block to application via the Measurement Notification service interface.

3.8.8 Time Notification Feature

Note the following if StbMNotificationCustomer is configured:

- If StbMTimeNotificationCallback is NULL, StbM will call the NotifyTime service operation of the required port GlobalTime_TimeEvent_{TB-Name}_{CName}. StbM_TimerCallback is called in an interrupt context. The operation NotifyTime may however only be called from a task context, so StbM has to decouple the interrupt context from the task context by triggering an ExternalTriggerOccurredEvent. To enable this to happen, StbM_-TimerCallback_Wrapper should be mapped to a task during RTE generation. (Note: Time Notification can only be configured if RTE is being used).
- If StbMTimeNotificationCallback is not NULL, the configured callback function is called from StbM (the decoupling from an interrupt context is not required) and StbM_TimerCallback_Wrapper is called directly called from StbM_TimerCallback.

3.9 Integration Advice

3.9.1 Init Functions

StbM_Init is called at system startup, and it should be called before any service or the StbM_MainFunction is invoked. Before invoking StbM_Init, all underlying layers have to be initialized.

On invocation of StbM_Init, each configured Time Base should be initialized with zero and its synchronization status timeBaseStatus set for all Time Domains, and all bits set to 0x00.

For each Time Base configured to be stored non-volatile (StbMStoreTime-baseNonVolatile configured as STORAGE_AT_SHUTDOWN), the Time Base value will be loaded from NvM. If the restore is not successful, the Time Base will start with zero.

If the Time Recording feature has been turned on, then all the blocks corresponding to all the Time Base IDs will be initialized to zero.

3.9.1.1 NvM Considerations

If NvM is configured for a timebase it must be initialized before the initialization of StbM, since the time value is read from NvM during initialization. It should be defined under the action NvMReadAllFinishedActions during system initialization. See also the Configuration Advice section.

3.9.2 DeInit Functions

No specific handling necessary.

For each Time Base configured to be stored non-volatile (StbMStoreTime-baseNonVolatile configured as STORAGE_AT_SHUTDOWN), the latest Time Base value will be stored to NvM at shutdown.

3.9.3 Scheduled Functions

StbM_MainFunction must be periodically called. In a fully AUTOSAR-conforming system, this is done via RTE configuration.

Before invoking StbM_MainFunction (preferably before even invoking StbM_Init), all underlying layers should have been initialized.

StbM_MainFunction is triggered by an event named "TE_StbM_MainFunction", which is contained in StbM's BSW module description.

3.9.4 StbM and Tsyn

The StbM and Tsyn modules should run on the same core in the synchronization scenario.

3.9.5 OS Considerations

Interrupt locks or Spinlocks cannot be used for OS API's, so an OS Resource lock has to be configured for the OS API SyncScheduleTable, and all the tasks and ISRs that are running on the core should be linked with that Resource.

3.9.6 Time Notification feature

To enable the Time Notification feature (StbM_TimeNotification), GPT version 4.3 should be used and the RTE should be enabled.

3.9.7 Q-length support

For Q-length support the "–client-server-global-optimization=on" command should be added in the RTE_OPTIONS for RTE generation, and the Os Application should be a trusted application.

3.9.8 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The StbM module uses the following OS resources:

- StbM_Time_Source
- StbM_Sync_OsResource

3.10 Hardware Requirements

Table 3.34.StbM Hardware Requirements

Support	Usage
FLASH	10828 bytes
RAM	742 bytes

Usage calculated based on a sample configuration.

4 BSW Mode Manager

Module name	BswM
Package	RTA-BASE
AUTOSAR Module ID	42
AUTOSAR Version	22-11

4.1 Introduction

The BSW Mode Manager (BswM) is located in the system services abstraction layer, forming part of the base (RTA-BASE) stack.

BswM implements the part of the Vehicle Mode Management and Application Mode Management concept that resides in the BSW.

BswM's responsibility is to receive and arbitrate mode requests from application layer SW-Cs or other BSW modules (using simple rules) and to perform actions based on the arbitration result.

4.1.1 Interaction with other modules

BswM has interfaces to a large number of other BSW modules, the main ones of which are summarized here.

ComM

Mode Switch Indications originating from ComM go through BswM for further propagation to the SW-Cs. BswM can act as a ComM user requesting communication modes.

Com

BswM performs the handling of I-PDU Groups in Com. As a part of I-PDU group start/stop, it is possible to have the included signal values reset to their corresponding initialization values. BswM also handles enabling and disabling of deadline monitoring of signals in Com, and it can also trigger the transmission of an I-PDU.

PduR

BswM can enable and disable routing groups of I-PDUs in the PDU router.

CanSm

Mode switch indications from CanSm go through BswM for further propagation to the SW-Cs.

EthSm

Mode switch indications from EthSm go through BswM for further propagation to the SW-Cs.

FrSm

Mode switch indications from FrSm go through BswM for further propagation

to the SW-Cs.

LinSm

LinSm's mode switch indications go through BswM for further propagation to the SW-Cs. BswM also co-ordinates the switching of LinSm's Schedule Tables with the starting and stopping of the corresponding I-PDU groups in Com.

LinTp

LinTp requests modes from BswM to make sure that the correct LIN Schedule Table is active during LinTp operation.

Dcm

Dcm performs mode requests to BswM based on the diagnostic requests it receives. For example, Dcm can request "Disable Normal Communication". In this mode, BswM will turn off the corresponding I-PDU groups and Nm PDUs.

Nm

Nm communication is controlled (enabled/disabled) by BswM based on the current mode.

NvM

NvM reports the state of its blocks to BswM.

Sd

Sd informs BswM of the current state of a service.

J1939Dcm

J1939Dcm reports communication state changes to BswM for further propagation to the SW-Cs.

J1939Nm

After its state has changed, J1939Nm informs BswM of its current state.

J1939Rm

After its state has changed, J1939Rm informs BswM of its current state.

Module Configuration

This module is not currently supported by the RTA-BSW Configuration Generator (ConfGen) and must be manually configured.

4.2 Supported Features

BswM supports the following features, which are described in more detail in the sub-sections below.

- Mode Arbitration (Immediate or Deferred)
- Mode Control (Triggered or Conditional)
- Handling of I-PDU group switching

- Generic Request for Rte
- User Callout for Rte

4.2.1 Mode Arbitration

BswM's initiation of mode switches results from a rule-based arbitration of mode requests and mode indications received from SW-Cs or other BSW modules. The processing of this mode arbitration can occur in one of two ways:

4.2.1.1 Immediate

The requests are processed in the environment of the caller.

Care should be taken to ensure that execution of the action list does not jeopardize system performance or consistency, especially if the caller runs (or may run) in interrupt context - restrictions concerning usage of system functions in interrupt context apply.

4.2.1.2 Deferred

Requests are deferred and processed during a subsequent execution of BswM_Mainfunction.

If multiple deferred mode requests come from the same source before the arbitration begins, the latest request will be taken up for arbitration.

The scheduling of MainFunction can interrupt an ongoing new request. When the Mode Request API and MainFunction are executed in parallel, mode arbitration can occur simultaneously and it leads to a system inconsistency. So, the processing of deferred requests in BswM_MainFunction is delayed during the processing of an ongoing new request, and those delayed deferred requests are processed after the ongoing new request has been completely processed.

If there are further new requests during the processing of delayed requests, they will be delayed and processed in the next cycle of execution of the BswM mode request API or BswM_MainFunction.

4.2.2 Mode Control

The Mode Control part of BswM carries out all actions that should be taken based on the mode arbitration. This is done using Action Lists, which are ordered lists of actions that BswM executes when triggered by the Mode Arbitration.

There are two ways that an action list may be executed based on evaluation of rules. Either it is executed every time the rule is evaluated with the corresponding result (triggered), or only when the evaluation result has changed from the previous evaluation (conditional). The execution method for an Action List is configured using the BswMActionListExecution parameter.

4.2.3 Handling of I-PDU group switching

BswM is the only module that controls the starting and stopping of I-PDU groups, as well as the enabling/disabling of I-PDU group deadline monitoring. As such, Com does not provide accessor functions for the I-PDU group states, so BswM

must maintain several Com_IpduGroupVectors to implement Com_IpduGroup-Control and Com_ReceptionDMControl actions.

To perform I-PDU group switches (enable/disable) in an efficient and consistent way, BswM performs the actual I-PDU Group Control function call at the end of processing the main function or an immediate processing request. This means that the I-PDU Group Switch actions manipulate an I-PDU Group Vector that is internal to the BswM, and this Vector is passed as a parameter to COM.

4.2.4 Generic Request for Rte

For a SWC module to perform even a very basic operation with BswM it requires some considerable configuration of Rte, so the BswMGenericRequest Mode Request Port can optionally accept Mode names as simple hash defines (like compu-methods), using a Client-Server Interface.

There's no Mode Declaration Group involved, and so there's no Rte_Switch anywhere in the system. BswM is simply informed of the change, and then performs some action out of it.

See the Configuration advice section for more details.

4.2.5 User Callout for Rte

The BswM configuration includes an option for user callouts for SWC. This is done using a Client-Server Interface in Rte, where BswM is a client and SWC is a server.

See the Configuration advice section for more details.

4.3 Extensions

4.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

4.3.1.1 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/BswMModeRequestSource/BswMBswModeNotification/BswM-RbBswModeDeclarationGroup

Short Name	BswMRbBswModeDeclarationGroup
Description	In this Parameter, provide the reference to the mode declaration group. e.g: /TestCd_ComM_BSWMD/ModeDeclarationGroups/MDG_TestCd_ComM_BSWMD_Modes.
Multiplicity	1..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

4.3.1.2 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/BswMModeRequestSource/BswMGenericRequest/BswMRbGenericReqUserRef

Short Name	BswMRbGenericReqUserRef
Description	This parameter is a reference to one of the BswMRbGenericReqUser configured in BswMGeneral container.
Introduction	Upon configuring this parameter, the RequesterId is automatically generated. The generated Id will be available as a macro and shall be used by including BswM.h. The name of the macro is the same as the shortname of referenced BswMRbGenericReqUser in upper case with a prefix BSWM_CFG_USERID_. For eg: BSWM_CFG_USERID_MODEUSER0, where MODEUSER0 is the shortname of the referenced BswMRbGenericReqUser. This parameter shall be used as a substitute for BswMModeRequesterId but both cannot be configured together.
Multiplicity	0..1
Default Value	-
Type	ECUC-REFERENCE-DEF

4.3.1.3 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/BswMModeRequestSource/BswMGenericRequest/BswMRbRequestType

Short Name	BswMRbRequestType
Description	This parameter identifies whether the mode request is from BSW or SWC. If the BswMRbRequestType is BSW, the user request shall originate through BswM_RequestMode( ) API. If the BswMRbRequestType is SWC, BswM generates a Client-Server Interface in it's SWCD. The user shall configure a Syncronous-Server-Call-Point which refers to the Client-Server-Operation provided by BswM. The Client-Server-Operation inturn refers the BswM_RequestMode( ) API. If the BswMRbRequestType is SWC then user has to configure BswMModeCondition with BswMBswMode only. The BswMRbRequestType is considered BSW by default.
Multiplicity	0..1
Default Value	BSW
Enumeration Literals	BSW, SWC

Type	ECUC-ENUMERATION-PARAM-DEF

4.3.1.4 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/BswMModeRequestSource/BswMGenericRequest/BswMRequestedModeMax

Short Name	BswMRequestedModeMax
Description	This parameter defines the upper limit for the modes requested by this mode requester.
Introduction	The allowable range of this parameter shall coincide with the range of BswM_ModeType (which can be platform-dependent).
Multiplicity	1..1
Value Range	0..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

4.3.1.5 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/BswMModeRequestSource/BswMSwcModeRequest/BswMRbModeRequestQueueSize

Short Name	BswMRbModeRequestQueueSize
Description	This element specifies whether the mode communication is queued or not.Queued communication - When a queue length is configured with a non zero value,and a buffer is maintained to read the Modes. UnQueued communication - When queuelength is not configured or configured with zero,Modes are directly read .
Introduction	A non zero value specifies a queued communication and a non configured or zero queue length specifies a nonqueued communication.
Multiplicity	0..1
Value Range	0..255
Default Value	0
Type	ECUC-INTEGER-PARAM-DEF

4.3.1.6 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMRbSwcUserCallout

Short Name	BswMRbSwcUserCallout
Description	This container includes all details for the BswM to invoke a user defined function call in SWC. The return value (if any) of the user define function that is invoked will be ignored by BswM.

Multiplicity	0..1
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

4.3.1.7 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMRbSwcUserCallout/BswMRbSwcClientServerOperationRef

Short Name	BswMRbSwcClientServerOperationRef
Description	This is a reference to the Client-Server-Operation of Client-Server Interface from the SWC, e.g:/TestCd_BswM/TestCd_BswM_CSInterface/TestCd_BswM_SWC_UserCallout. BswM generates a Syncronous-Server-Call-Point which refers to the Client-Server-Operation provided by SWC. The Client-Server-Operation inturn refers the user defined function in SWC. Any return values passed by the callout will be ignored.
Multiplicity	1..1
Default Value	-
Type	ECUC-STRING-PARAM-DEF

4.3.1.8 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMRbSwcUserCallout/BswMRbSwcUserCalloutArgument

Short Name	BswMRbSwcUserCalloutArgument
Description	This parameter holds the value which is passed as input argument for the user defined function call. It is not in the scope of BswM to validate the data type of the arguments and the number of arguments passed, the integrator have to taken care of this.
Multiplicity	0..*
Default Value	-
Type	ECUC-STRING-PARAM-DEF

4.3.1.9 BswM/BswMConfig/BswMModeControl/BswMActionList/BswMActionListItem/BswMReportFailToDemRef

Short Name	BswMReportFailToDemRef
Description	If the reference is given, the DEM event shall be reported failed if this specific action returns E_NOT_OK; it shall be reported passed if this specific action returns E_OK.
Multiplicity	0..0

Default Value	-
Type	ECUC-SYMBOLIC-NAME-REFERENCE-DEF

4.3.1.10 BswM/BswMConfig/BswMModeControl/BswMRteModeRequestPort/BswMRbModeRequestQueueEnabled

Short Name	BswMRbModeRequestQueueEnabled
Description	This element specifies whether the sender receiver communication is queued or not.
Introduction	When true: Queue is Enabled and when false or not configured : Queue is disabled
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.11 BswM/BswMConfig/BswMModeControl/BswMRteModeRequestPort/BswMRbRteModeRequestPortInterfaceRef

Short Name	BswMRbRteModeRequestPortInterfaceRef
Description	This is a foreign reference to the variable data prototype used for the mode request e.g: /TestCd_BswM_RteModeReq_SWC_SWCD/SRI_TestCd_BswM_RteModeReq_1/Mode.
Multiplicity	1..1
Default Value	-
Type	ECUC-FOREIGN-REFERENCE-DEF

4.3.1.12 BswM/BswMConfig/BswMModeControl/BswMRteModeRequestPort/BswMRteModeRequestPortInterfaceMappingRef

Short Name	BswMRteModeRequestPortInterfaceMappingRef
Description	This is a foreign reference to the variable and parameter interface mapping used for the mode request.
Multiplicity	0..0
Default Value	-
Type	ECUC-FOREIGN-REFERENCE-DEF

4.3.1.13 BswM/BswMConfig/BswMModeControl/BswMSwitchPort/BswMRbModeGroupRef

Short Name	BswMRbModeGroupRef

Description	In this parameter, provide the reference to the ModeGroup e.g: /TestCd_ComM_SWCD/MSI_TestCd_ComM_SWCD/ModeDeclarationGroupPrototype.
Multiplicity	1..1
Default Value	-
Type	ECUC-FOREIGN-REFERENCE-DEF

4.3.1.14 BswM/BswMConfig/BswMModeControl/BswMSwitchPort/BswMRbModeSwitchQueueLength

Short Name	BswMRbModeSwitchQueueLength
Description	This element specifies whether the mode switch communication is queued or not, the length of the mode switch queue (which is generated by the RTE) on the provider port. Unqueued communication - When a mode manager initiates a mode switch, RTE will begin processing the request. When the mode switch is currently in progress for the mode instance, any further mode switches will be discarded/overwritten by RTE. Queued communication - When a mode manager initiates a mode switch, RTE will begin processing the request. When the mode switch is currently in progress for the mode instance, any further mode switches will be queued. The size of the mode switch queue is determined by BswMRbModeSwitchQueueLength.
Multiplicity	0..1
Value Range	1..255
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

4.3.1.15 BswM/BswMGeneral/BswMRbComEnabled

Short Name	BswMRbComEnabled
Description	enable/disable Com module related BswM API:
Introduction	true: Enabled false: Disabled
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.16 BswM/BswMGeneral/BswMRbDebugEnable

Short Name	BswMRbDebugEnable

Description	Enable : BswM records into NvM if any mode request is interrupted ; Disable: no information will be recorded. Additionally, the first element of RAM variable BswM_Rb_ctrlnterrupt_au8 will indicate the number of times a request is being interrupted.
Introduction	Hint : When this switch is enabled, NvM block for BswM needs to be configured for storing Interrupt related information (example configuration is provided in BswM_Ecucvalues.arxml file). When this switch is disabled, no NvM block should be configured.
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.17 BswM/BswMGeneral/BswMRbGenericReqUser

Short Name	BswMRbGenericReqUser
Description	This container describes the Generic request ModeRequesterIds in a human readable format.
Introduction	This container is referenced by BswMRbGenericReqUserRef in BswMGenericRequest mode request port. The values for the ModeRequesterIds are autogenerated and shall be used by including BswM.h. Please refer the description of BswMRbGenericReqUserRef, on how to use this ModeRequesterId
Multiplicity	0..*
Default Value	-
Type	ECUC-PARAM-CONF-CONTAINER-DEF

4.3.1.18 BswM/BswMGeneral/BswMRbIntrptQueueMaxSize

Short Name	BswMRbIntrptQueueMaxSize
Description	In a system where there are multiple requests to BswM more frequently, BswM processes one request at a time and delayes any further requests in a queue. The parameter BswMRbIntrptQueueMaxSize shall be configured accordingly which determines the number of requests which can be delayed in the queue

Introduction	Hint : The BswMRbDebugEnable parameter when enabled will maintain a record of the number of times a request is interrupted.Additionally include 30%(Calculating the buffer percentage does not have a specific formula, so it should be treated as an revised rather than an exact calculation) buffer value.Ensure that falls within the maximum range of 255. This information can be used to configure an appropriate value for the maximum queue size.
Multiplicity	1..1
Value Range	1..255
Default Value	1
Type	ECUC-INTEGER-PARAM-DEF

4.3.1.19 BswM/BswMGeneral/BswMRbMaxNumOfRules

Short Name	BswMRbMaxNumOfRules
Description	This parameter specifies the maximum number of Rules which could be configured for each variant in BswM Module.
Introduction	This is a pre-compile parameter, which limits the number of BswM Rules that could be configured during post-build time.
Multiplicity	0..1
Value Range	1..65535
Default Value	-
Type	ECUC-INTEGER-PARAM-DEF

4.3.1.20 BswM/BswMGeneral/BswMRbSoAdEnabled

Short Name	BswMRbSoAdEnabled
Description	Enable/disable SoAd module related BswM API.
Introduction	true: Enabled false: Disabled
Multiplicity	0..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.21 BswM/BswMGeneral/BswMSchMEnabled

Short Name	BswMSchMEnabled
Description	enable/disable SchM module related BswM API:
Introduction	true: Enabled false: Disabled
Multiplicity	1..1

Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.1.22 BswM/BswMGeneral/BswMWdgMEnabled

Short Name	BswMWdgMEnabled
Description	enable/disable WdgM module related BswM API:
Introduction	true: Enabled false: Disabled
Multiplicity	0..0
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

4.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

Syntax	`BswM_EthIf_PortGroupLinkStateChg`
Parameters	`PortGroupIdx` `PortGroupState`
Description	EthIf function call to indicate the current state of EthIf SwitchPortGroup.

Syntax	`BswM_EcuM_CurrentState`
Parameter	`CurrentState`
Description	Function called by EcuM to indicate the current ECU Operation Mode.

Syntax	`BswM_EcuM_RequestedState`
Parameters	`CurrentStatus`
Description	Function called by EcuM to notify about current Status of the Run Request Protocol.

Syntax	`BswM_WdgM_RequestPartitionReset`
Parameters	None
Description	Function called by WdgM to request a partition reset.

Syntax	`BswM_Rb_Timer_CurrentState`
Parameters	`CurrentState`
Description	A timer which can be used for time-dependent rules.

4.4 Limitations

It is assumed that in a particular Mainfunction cycle no two rules will refer to the

same action list. If an action list is called by multiple rules within the same BswM Mainfunction cycle, the action list will be executed only once in that particular cycle.

BswM is supported with or without Post Build feature, in BswM the Post Build feature is applicable for both BswMRule and BswMActionList. The current implementation supports the Post Build feature only in BswMActionList, therefore BswMRule Post Build support is not currently implemented.

4.5 Exclusions

The following features from AR v22-11 are not supported in version v12.7.0.

API functions not currently implemented:

- `BswM_BswMPartitionRestarted`
- `BswM_CanSM_CurrentIcomConfiguration`

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- BswM/BswMConfig/BswMArbitration/BswMEventRequestPort/BswMEventRequestSource/BswMDcmApplicationUpdatedIndication
- BswM/BswMConfig/BswMArbitration/BswMEventRequestPort/BswMEventRequestSource/BswMModeSwitchErrorEvent
- BswM/BswMConfig/BswMArbitration/BswMEventRequestPort/BswMEventRequestSource/BswMModeSwitchErrorEvent/BswMRteSwitchPortRef
- BswM/BswMConfig/BswMArbitration/BswMEventRequestPort/BswMEventRequestSource/BswMPartitionRestarted
- BswM/BswMConfig/BswMArbitration/BswMEventRequestPort/BswMEventRequestSource/BswMSwitchAckNotification
- BswM/BswMConfig/BswMArbitration/BswMEventRequestPort/BswMEventRequestSource/BswMSwitchAckNotification/BswMSwitchPortRef
- BswM/BswMConfig/BswMArbitration/BswMModeCondition/BswMConditionValue/BswMCompuScaleModeValue
- BswM/BswMConfig/BswMArbitration/BswMModeCondition/BswMConditionValue/BswMCompuScaleModeValue/BswMCompuConstText
- BswM/BswMConfig/BswMArbitration/BswMModeCondition/BswMConditionValue/BswMCompuScaleModeValue/BswMCompuMethodRef
- BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/BswMModeInitValue/BswMCompuScaleModeValue
- BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/BswMModeInitValue/BswMCompuScaleModeValue/BswMCompuConstText
- BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/BswMModeInitValue/BswMCompuScaleModeValue/BswMCompuMethodRef
- BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/BswMModeRequestSource/BswMBswModeNotification/BswMBswModeDeclara-

tionGroupPrototypeRefBswMBswModeDeclarationGroupPrototypeRef

- BswM/BswMConfig/BswMArbitration/BswMRule/BswMNestedExecutionOnly
- BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMEthIfStartAllPorts
- BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMRteStart
- BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMRteStop
- BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMSchMSwitch/BswMSchMModeDeclarationGroupRef
- BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMSchMSwitch/BswMSchMSwitchPortRef
- BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMTimerControl/BswMTimerRef
- BswM/BswMConfig/BswMModeControl/BswMRteModeRequestPort/BswMRteModeRequestPortInterfaceRef
- BswM/BswMConfig/BswMModeControl/BswMRteModeRequestPort/BswMRteModeRequestVariableDataPrototypeSRRef
- BswM/BswMConfig/BswMModeControl/BswMSwitchPort/BswMModeSwitchInterfaceRef
- BswM/BswMConfig/BswMPartitionRef

4.6 Deviations

4.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

4.6.1.1 BswM

[RTA-BSW] Description	Configuration of the BswM (Basic SW Mode Manager) module (with vendor specific enhancements of RB).
[AUTOSAR] Description	Configuration of the BswM (Basic SW Mode Manager) module.

4.6.1.2 BswM/BswMConfig

[RTA-BSW] Upper Multiplicity	1
[AUTOSAR] Upper Multiplicity	*

4.6.1.3 BswM/BswMConfig/BswMArbitration/BswMEventRequestPort

[ADDED] Introduction	This choice container specifies the source of the event request. The sender of the event can be another BSW Module, such as ComM.

4.6.1.4 BswM/BswMConfig/BswMArbitration/BswMEventRequestPort/BswMEventRequestSource/BswMComMInitiateReset

[ADDED] Introduction	Before Configuring the ERP, make sure the ComM is enabled in BswMGeneral Container. The BswMComMIndication is configured only once.

4.6.1.5 BswM/BswMConfig/BswMArbitration/BswMEventRequestPort/BswMEventRequestSource/BswMNmCarWakeUpIndication

[ADDED] Introduction	Channel assosciated with CarWakeup

4.6.1.6 BswM/BswMConfig/BswMArbitration/BswMEventRequestPort/BswMEventRequestSource/BswMNmCarWakeUpIndication/BswMNmChannelRef

[ADDED] Introduction	This parameter relates the communication channel associated with the car wakeup indication

4.6.1.7 BswM/BswMConfig/BswMArbitration/BswMLogicalExpression/BswMArgumentRef

[RTA-BSW] Introduction	In case the BswMLogicalExpression.BswMLogicalOperator equals BSWM_NAND only two operands are supported.
[AUTOSAR] Introduction	In case the BswMLogicalExpression.BswMLogicalOperator equals BSWM_NAND only two operands are supported. In case the BswMLogicalExpression.BswMLogicalOperator equals BSWM_NOT only one operand is supported.

4.6.1.8 BswM/BswMConfig/BswMArbitration/BswMLogicalExpression/BswMLogicalOperator

[RTA-BSW] Description	This parameter specifies the logical operator to be used in the logical expression. If the expression only consists of a single condition this parameter shall not be used.
[AUTOSAR] Description	This parameter specifies the logical operator to be used in the logical expression. If the logical operator is set to something other than BSWM_NOT, and the expression only consists of a single condition, then this parameter will have no effect.

4.6.1.9 BswM/BswMConfig/BswMArbitration/BswMModeCondition/BswMConditionType

[RTA-BSW] Description	This parameter specifies what kind of comparison that is made for the evaluation of the mode condition. For BSWM_EQUALS and BSWM_EQUALS_NOT, the BswMModeRequestPort port referenced by BswMConditionMode is compared with the value configured in BswMConditionValue for equality or not-equality. For BSWM_EVENT_IS_SET and BSWM_EVENT_IS_CLEARED, the BswMEventRequestPort port referenced by BswMConditionMode is checked for being set or cleared (not-set).
[AUTOSAR] Description	This parameter specifies what kind of comparison that is made for the evaluation of the mode condition.
[REMOVED] Introduction	For BSWM_EQUALS and BSWM_EQUALS_NOT, the BswMModeRequestPort port referenced by BswMConditionMode is compared with the value configured in BswMConditionValue for equality or not-equality. For BSWM_EVENT_IS_SET and BSWM_EVENT_IS_CLEARED, the BswMEventRequestPort port referenced by BswMConditionMode is checked for being set or cleared (not-set).

4.6.1.10 BswM/BswMConfig/BswMArbitration/BswMModeCondition/BswMConditionValue/BswMBswMode

[RTA-BSW] Description	This container defines the mode condition value and type of a mode in the BSW,If the related MRP is BswMDcmApplicationUpdatedIndication or BswMJ1939DcmBroadcastStatus configure “BswMBswRequestedMode” value in ModeCondition as TRUE or FALSE.
[AUTOSAR] Description	This container defines the value of a mode in the BSW.

4.6.1.11 BswM/BswMConfig/BswMArbitration/BswMModeCondition/BswMConditionValue/BswMModeDeclaration

[RTA-BSW] Description	When the mode corresponds to a mode request or mode indication interface the mode is defined by a mode declaration. The mode declarations belonging to a mode declaration group are defined in the SW-C Template and hence a foreign reference to the corresponding Mode Declaration is used. It is only relevant for Rte.
[AUTOSAR] Description	When the mode corresponds to a mode request or mode indication interface the mode is defined by a mode declaration. The mode declarations are defined in the SW-C Template and hence a foreign reference to the corresponding Mode Declaration is used.

4.6.1.12 BswM/BswMConfig/BswMArbitration/BswMModeCondition/BswMConditionValue/BswMModeDeclaration/BswMModeValueRef

[RTA-BSW] Description	This is a foreign reference to the Mode Declaration used for the mode requests corresponding to this condition. e.g. /TestCd_ComM_SWCD/MDG_TestCd_ComM_SWCD/COMM_FULL_COMMUNICATION
[AUTOSAR] Description	This is a foreign reference to the Mode Declaration used for the mode requests corresponding to this condition.

4.6.1.13 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/BswMModeRequestSource/BswMBswModeNotification

[RTA-BSW] Description	The source of the mode request is a Bsw Module. After BSW performs the Mode Switch action, the latest mode is updated in RTE. The updated mode is notified to BswM through BswMBswModeNotification.
[AUTOSAR] Description	This is a mode request source emanating from another BSW Module.

4.6.1.14 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/BswMModeRequestSource/BswMCanSMIndication/BswMCanSMChannelRef

[ADDED] Introduction	Before Configuring the MRP, make sure the CanSM is enabled in BswMGeneral Container. The BswMCanSMChannelRef has to unique for all BswMCanSMIndication.

4.6.1.15 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/BswMModeRequestSource/BswMComMIndication/BswMComMChannelRef

[ADDED] Introduction	Before Configuring the MRP, make sure the ComM is enabled in BswMGeneral Container. The BswMComMChannelRef has to unique for all BswMComMIndication.

4.6.1.16 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/BswMModeRequestSource/BswMComMPncRequest/BswMComMPncRef

[ADDED] Introduction	Before Configuring the MRP, make sure the ComM is enabled in BswMGeneral Container. The BswMComMPncRef has to unique for all BswMComMPncRequest.

4.6.1.17 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/ BswMModeRequestSource/BswMDcmComModeRequest

[ADDED] Introduction	The source of the mode request is the Diagnostic Communication Manager.

4.6.1.18 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/ BswMModeRequestSource/BswMDcmComModeRequest/BswMD-cmComMChannelRef

[ADDED] Introduction	This is a reference from DcmModeRequest to the ComM channel that the indication corresponds to.

4.6.1.19 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/ BswMModeRequestSource/BswMEcuMRUNRequestIndication/ BswMEcuMRUNRequestProtocolPort

[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

4.6.1.20 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/ BswMModeRequestSource/BswMEcuMWakeupSource

[ADDED] Introduction	Notification of state of EcuM Wakeup Source.

4.6.1.21 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/ BswMModeRequestSource/BswMEcuMWakeupSource/BswME-cuMWakeupSrcRef

[ADDED] Introduction	Reference to EcuM Wakeup Source.

4.6.1.22 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/ BswMModeRequestSource/BswMEthIfPortGroupLinkStateChg

[ADDED] Introduction	Notification of link state of a Ethernet interface switch port group.

4.6.1.23 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/ BswMModeRequestSource/BswMEthIfPortGroupLinkStateChg/ BswMEthIfSwitchPortGroupRef

[ADDED] Introduction	Reference to Ethernet Interface Switch Port Group.

4.6.1.24 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/BswMModeRequestSource/BswMGenericRequest/BswMModeRequesterId

[ADDED] Introduction	The allowable range of this parameter shall coincide with the range of BswM_UserType (which can be platform-dependent).
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

4.6.1.25 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/BswMModeRequestSource/BswMLinTpModeRequest/BswMLinTpChannelRef

[RTA-BSW] Description	This is a reference to the LIN Interface Channel that the mode
[AUTOSAR] Description	This is a reference to the LIN Interface Channel that the mode request corresponds to.
[ADDED] Introduction	request corresponds to.

4.6.1.26 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/BswMModeRequestSource/BswMNmStateChangeNotification/BswMNmChannelRef

[ADDED] Introduction	Before Configuring the MRP, make sure the Nm is enabled in BswM-General Container. The BswMNmChannelRef has to unique for all BswMNmStateChangeNotification.

4.6.1.27 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/BswMModeRequestSource/BswMNvMJobModeIndication

[RTA-BSW] Description	Indicates the current status of the multiblock job. The job is identified via BswMNvmService, e.g. 0x0c for NvmReadAll, 0x0d for NvmWriteAll, 0xfb for NvMRbFirstInitAll, 0xfe for NvMRbRemoveNonResistant. Possible Values are:
[AUTOSAR] Description	Indicates the current status of the multiblock job. The job is identified via BswMNvmService. Possible values for this indication are the possible values of NvM_RequestResultType.
[ADDED] Introduction	NvM_RequestResultType NVM_REQ_OK NVM_REQ_NOT_OK NVM_REQ_PENDING NVM_REQ_INTEGRITY_FAILED NVM_REQ_BLOCK_SKIPPED NVM_REQ_NV_INVALIDATED NVM_REQ_CANCELED NVM_REQ_REDUNDANCY_FAILED NVM_REQ_RESTORED_FROM_ROM

4.6.1.28 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/BswMModeRequestSource/BswMNvMJobModeIndication/BswMN-

vmServiceBswMNvmService

[RTA-BSW] Enumeration Literals	NvmReadAll, NvmWriteAll, NvMRbFirstInitAll, NvMRbRemoveNonResistant
[AUTOSAR] Enumeration Literals	NvmCancelWriteAll, NvmFirstInitAll, NvmReadAll, NvmValidateAll, NvmWriteAll

4.6.1.29 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/BswMModeRequestSource/BswMSwcModeNotification/BswMSwcModeNotificationModeDeclarationGroupPrototypeRef

[RTA-BSW] Description	This is a foreign reference to the ModeDeclarationGroupPrototype. Give the reference of mode declaration group prototype e.g: /TestCd_ComM_SWCD/MSI_TestCd_ComM_SWCD/ModeDeclarationGroupPrototype
[AUTOSAR] Description	This is a foreign reference to the ModeDeclarationGroupPrototype.

4.6.1.30 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/BswMModeRequestSource/BswMSwcModeRequest

[RTA-BSW] Description	The source of the mode request is a SW Component. The SWC module places a Mode request towards BswM using BswMSwcModeRequest and BswM performs a mode switch as a result of Rte_Switch action.
[AUTOSAR] Description	The source of the mode request is a SW Component.

4.6.1.31 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/BswMModeRequestSource/BswMSwcModeRequest/BswMSwcModeRequestVariableDataPrototypeRef

[RTA-BSW] Description	This is a foreign reference to the requestedMode data element of Sender-Receiver Interface from the App Mode Manager. e.g: /TestCd_BswM_SWC2_SWCD/SRI_TestCd_BswM_SWC2_SWCD/Mode
[AUTOSAR] Description	This is a reference to the VariableDataPrototype.

4.6.1.32 BswM/BswMConfig/BswMArbitration/BswMModeRequestPort/BswMModeRequestSource/BswMTimer

[RTA-BSW] Description	This is a timer which can be used for time dependent rules. This mode request port can be in one of three modes (depending on the state of the timer) BSWM_TIMER_STOPPED (intial) (The timer has been stopped by an action) BSWM_TIMER_STARTED (The timer has been started by an action) BSWM_TIMER_EXPIRED (The timer has expired)

[AUTOSAR] Description	This is a timer which can be used for time dependent rules. This mode request port can be in one of three modes (depending on the state of the timer):
[REMOVED] Introduction	* BSWM_TIMER_STOPPED (intial) (The timer has been stopped by an action) * BSWM_TIMER_STARTED (The timer has been started by an action) * BSWM_TIMER_EXPIRED (The timer has expired)

4.6.1.33 BswM/BswMConfig/BswMArbitration/BswMRule/BswMRuleInitState

[RTA-BSW] Description	This parameter is a part of the reset/initialization behavior of BswM. Action lists(Triggered) are executed when the result of a rule evaluation have changed since the last evaluation. This parameter defines the “previous evaluation result” of a rule to be used after initialization of the BswM.
[AUTOSAR] Description	This parameter is a part of the reset/initialization behavior of BswM.

4.6.1.34 BswM/BswMConfig/BswMDataTypeMappingSets/BswMDataTypeMappingSetRef

[RTA-BSW] Description	Reference to DataTypeMappingSet. Provide reference to the data type mapping defined for Mode declaration group e.g: /TestCd_ComM_SWCD/DTMS_TestCd_ComM_SWCD
[AUTOSAR] Description	Reference to DataTypeMappingSet.

4.6.1.35 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMClearEventRequest/BswMClearEventRequestPortRef

[ADDED] Introduction	This parameter references the BswMEventRequestPort which will be cleared.

4.6.1.36 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMCoreHaltMode/BswMCoreHaltActivationState

[ADDED] Introduction	This reference corresponds to the parameter “IdleMode” of the function ControlIdle.

4.6.1.37 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMCoreHaltMode/BswMTargetCoreRef

[ADDED] Introduction	This reference corresponds to the parameter “CoreID” of the function ControlIdle.

4.6.1.38 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMEcuMGoDownHaltPoll/BswMEcuMUserIdRef

[ADDED] Introduction	This is a reference to a EcuM UserId.

4.6.1.39 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMEcuMSelectShutdownTarget/BswMEcuMResetModeRef

[ADDED] Introduction	Symbolic name reference to EcuMResetMode.

4.6.1.40 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMEcuMSelectShutdownTarget/BswMEcuMSleepModeRef

[ADDED] Introduction	Symbolic name reference to EcuMSleepMode

4.6.1.41 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMEcuMStateSwitch/BswMEcuMState

[RTA-BSW] Enumeration Literals	BSWM_ECUM_STATE_APP_POST_RUN, BSWM_ECUM_STATE_APP_RUN, BSWM_ECUM_STATE_SHUTDOWN, BSWM_ECUM_STATE_SLEEP, BSWM_ECUM_STATE_STARTUP
[AUTOSAR] Enumeration Literals	BSWM_ECUM_STATE_POST_RUN, BSWM_ECUM_STATE_RUN, BSWM_ECUM_STATE_SHUTDOWN, BSWM_ECUM_STATE_SLEEP, BSWM_ECUM_STATE_STARTUP

4.6.1.42 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMEthIfSwitchPortGroupRequestMode/BswMEthTrcvMode

[RTA-BSW] Enumeration Literals	BSWM_ETHTRCV_MODE_ACTIVE, BSWM_ETHTRCV_MODE_DOWN
[AUTOSAR] Enumeration Literals	BSWM_ETH_MODE_ACTIVE, BSWM_ETH_MODE_ACTIVE_WITH_WAKEUP_REQUEST, BSWM_ETH_MODE_DOWN

4.6.1.43 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMFrSMAllSlots/BswMFrSMAllSlotsNetworkHandleRef

[ADDED] Introduction	This references the FlexRay cluster. The reference corresponds to the parameter “NetworkHandle” of the function FrSM_AllSlots.

4.6.1.44 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvail-

ableActions/BswMIdsMBlockStateChangeRequest

[RTA-BSW] Description	This container includes all parameters related to switching the Block State of the IdsM.The IdsM_BswM_StateChanged API is called when this action is executed.
[AUTOSAR] Description	This container defines the action to switch a BlockState of the IdsM. Tags: atp.Status=draft

4.6.1.45 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMIdsMBlockStateChangeRequest/BswMIdsMBlockStateChangeRequestRef

[RTA-BSW] Description	This is a reference to the Block State, that shall be set as current Block State of the IdsM. This reference corresponds to the parameter “state” of the function IdsM_BswM_StateChanged.
[AUTOSAR] Description	This is a reference to a BlockState of the IdsM. Tags: atp.Status=draft
[ADDED] Introduction	This is a reference to the Block State, that shall be set as current Block State of the IdsM. This reference corresponds to the parameter “state” of the function IdsM_BswM_StateChanged.
[RTA-BSW] Lower Multiplicity	0
[AUTOSAR] Lower Multiplicity	1

4.6.1.46 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMJ1939RmStateSwitch/BswMJ1939RmChannelRef

[RTA-BSW] Description	This reference points to the unique channel defined by the ComMChannel and provides access to the unique channel index value in ComMChannelId. This reference corresponds to the parameter “channel” of the function J1939Rm_SetState.
[AUTOSAR] Description	This reference points to the unique channel defined by the ComMChannel and provides access to the unique channel index value in ComMChannelId.

4.6.1.47 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMJ1939RmStateSwitch/BswMJ1939RmNodeRef

[RTA-BSW] Description	This reference points to a J1939NmNode and provides access to the unique J1939NmNodeId. This reference corresponds to the parameter “node” of the function J1939Rm_SetState.
[AUTOSAR] Description	This reference points to a J1939NmNode and provides access to the unique J1939NmNodeId.

4.6.1.48 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMPduGroupSwitch/BswMEnabledPduGroupRef

[RTA-BSW] Introduction	TThis reference corresponds to the parameter “IpduGroupId” of the function Com_IpduGroupStart.
[AUTOSAR] Introduction	This reference corresponds to the parameter “IpduGroupId” of the function Com_IpduGroupStart.

4.6.1.49 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMRteModeRequest/BswMRequestedModeRef

[RTA-BSW] Description	This is a foreign reference to the Mode Declaration used for the mode request e.g: /TestCd_ComM_SWCD/MDG_TestCd_ComM_SWCD/COMM_SILENT_COMMUNICATION
[AUTOSAR] Description	This is a foreign reference to the Mode Declaration used for the mode request

4.6.1.50 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMRteModeRequest/BswMRteModeRequestPortRef

[RTA-BSW] Description	This is a reference to a BswMRteModeRequestPort.This will defines the PPorts for Rte_Write action e.g: BswMRteModeRequestPort_1. where BswMRteModeRequestPort_1 is a BswMRteModeRequest port defined in BswM configurations.
[AUTOSAR] Description	This is a reference to a BswMRteModeRequestPort.

4.6.1.51 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMRteSwitch

[RTA-BSW] Description	This container defines a mode switch indication that the BswM provides to the SW-C that need to be notified about the mode switch. RTE_Switch is called when this action is configured. Note: SWC mode request and Rte switch action both are coupled each other. If configuring BswMSwcModeRequest and not configured BswMRteMSwitch action will generate validation error/warning.
[AUTOSAR] Description	This container defines a mode switch indication that the BswM provides to the SW-C that need to be notified about the mode switch. RTE_Switch is called when this action is configured.

4.6.1.52 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMRteSwitch/BswMRteSwitchPortRef

[RTA-BSW] Description	This is a reference to the BswMSwitchPort.This will defines PPorts for Rte_Switch action e.g: BswMSwitchPort_ComM_Modes. where BswMSwitchPort_ComM_Modes is a switch port defined in BswM configurations.
[AUTOSAR] Description	This is a reference to the BswMSwitchPort.

4.6.1.53 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMRteSwitch/BswMSwitchedMode

[RTA-BSW] Description	This parameter contains the integer value that corresponds to a certain mode in a Mode Declaration Group. e.g: /TestCd_ComM_SWCD/MDG_TestCd_ComM_SWCD/COMM_FULL_COMMUNICATION.
[AUTOSAR] Description	This parameter contains the integer value that corresponds to a certain mode in a Mode Declaration Group.

4.6.1.54 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMSchMSwitch/BswMSchMSwitchedMode

[RTA-BSW] Description	This parameter contains the value that corresponds to a certain mode in a Mode Declaration Group. e.g: /TestCd_ComM_BSWMD/ModeDeclarationGroups/MDG_TestCd_ComM_BSWMD_Modes/COMM_FULL_COMMUNICATION
[AUTOSAR] Description	This parameter contains the integer value that corresponds to a certain mode in a Mode Declaration Group.

4.6.1.55 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMSdServiceGroupSwitch/BswMDisabledSdServiceGroupRef

[RTA-BSW] Description	This is a reference to a SdServiceGroup that should be disabled.
[AUTOSAR] Description	This is a reference to a SdServiceGroup that should be disabled. This reference corresponds to the parameter “ServiceGroupId” of the function Sd_ServiceGroupStop.
[ADDED] Introduction	This reference corresponds to the parameter “ServiceGroupId” of the function Sd_ServiceGroupStop.

4.6.1.56 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMSdServiceGroupSwitch/BswMEnabledSdServiceGroupRef

[RTA-BSW] Description	This is a reference to a SdServiceGroup that should be enabled.
[AUTOSAR] Description	This is a reference to a SdServiceGroup that should be enabled. This reference corresponds to the parameter “ServiceGroupId” of the function Sd_ServiceGroupStart.
[ADDED] Introduction	This reference corresponds to the parameter “ServiceGroupId” of the function Sd_ServiceGroupStart.

4.6.1.57 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMTimerControl/BswMTimerAction

[RTA-BSW] Description	Specify the action for the timer. The timer can be started or stopped..
[AUTOSAR] Description	Specify the action for the timer. The timer can be started or stopped.
[REMOVED] Default Value	BSWM_TIMER_START

4.6.1.58 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMTimerControl/BswMTimerValue

[ADDED] Introduction	Specify the timer value (in seconds) that is used when the timer is started.
[RTA-BSW] Lower Multiplicity	1
[AUTOSAR] Lower Multiplicity	0
[REMOVED] Value Range	0..INF

4.6.1.59 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMUserCallout

[RTA-BSW] Description	This container include file containing the function declaration to be added in BswMUserIncludeFiles of BswMGeneral container.
[AUTOSAR] Description	This container includes all details needed for a user defined function call.

4.6.1.60 BswM/BswMConfig/BswMModeControl/BswMAction/BswMAvailableActions/BswMUserCallout/BswMUserCalloutFunction

[RTA-BSW] Introduction	Example usage can be: Actions to initialize other BSW modules Action to call Rte_Start() Action to call Rte_Stop() Action to call NvM_ReadAll() Action to call NvM_WriteAll()

[AUTOSAR] Introduction	Example usage can be: Actions to initialize other BSW modules Action to call NvM_ReadAll() Action to call NvM_WriteAll()

4.6.1.61 BswM/BswMConfig/BswMModeControl/BswMActionList/BswMActionListExecution

[RTA-BSW] Description	This parameter controls if the corresponding action list shall be executed every time the rule is evaluated or only when the result of the evaluation changes. “BSWM_CONDITION” - action list shall be executed every time the rule is evaluated. “BSWM_TRIGGER” - action list shall be executed, only when the result of the evaluation changes.
[AUTOSAR] Description	This parameter controls if the corresponding action list shall be executed every time the rule is evaluated or only when the result of the evaluation changes. This parameter does not have an effect when this action list is executed within another action list.

4.6.1.62 BswM/BswMConfig/BswMModeControl/BswMActionList/BswMActionListItem/BswMActionListItemRef

[RTA-BSW] Description	The action item can either be an atomic action or a reference to another action list. An action item refering to a Rule (Nested Rule) is going to be deprecated in future (Rfc 61155 C#24). Hence it is removed.
[AUTOSAR] Description	The action item can either be an atomic action or a reference to another action list or rule.

4.6.1.63 BswM/BswMConfig/BswMModeControl/BswMActionList/BswMActionListItem/BswMReportFailRuntimeErrorId

[RTA-BSW] Description	If this parameter is configured, and this specific action returns E_NOT_OK, the BswM will report a Det Runtime Error. The ErrorId reported in the Runtime Error is given by the value configured in this parameter.The value configured in the parameter is considered only if BswMActionListItemRef referred to an action
[AUTOSAR] Description	If this parameter is configured, and this specific action returns E_NOT_OK, the BswM will report a Det Runtime Error. The ErrorId reported in the Runtime Error is given by the value configured in this parameter.

4.6.1.64 BswM/BswMConfig/BswMModeControl/BswMSwitchPort

[RTA-BSW] Description	This container specifies PPorts , which the BswM has to create in its SWCD. If the container is referenced by one or more BswMRteSwitchActions the BswM shall create a corresponding PPort in its Service Description.

[AUTOSAR] Description	Represents an output mode-switch port to be generated by the BswM. If BswMModeSwitchInterfaceRef is configured then a PPortPrototype is generated in the SWCD. If BswMSchMModeDeclarationGroupRef is configured then a ModeDeclaration GroupPrototype is generated in the ProvidedModeGroups of the BSWMD. If both BswMModeSwitch InterfaceRef and BswMSchMModeDeclarationGroupRef are configured then an SwcBswSynchronizedModeGroupPrototype is also generated in the BSWMD (see Chapter 6.11 of the BSW Module Description Template SWS and EXP ModemanagementGuide)..

4.6.1.65 BswM/BswMGeneral/BswMDevErrorDetect

[RTA-BSW] Description	Switches the Default Error Tracer (Det) detection and notification ON or OFF.
[AUTOSAR] Description	Switches the development error detection and notification on or off.
[RTA-BSW] Introduction	* true: enabled (ON). * false: disabled (OFF).
[AUTOSAR] Introduction	* true: detection and notification is enabled. * false: detection and notification is disabled.

4.6.1.66 BswM/BswMGeneral/BswMMainFunctionPeriod

[RTA-BSW] Maximum Value	Inf
[AUTOSAR] Maximum Value	INF

4.6.1.67 BswM/BswMGeneral/BswMUserIncludeFiles/BswMUserIncludeFile

[RTA-BSW] Description	Header file name which shall be included by the BswM. The value of this parameter shall be used as h-char-sequence or q-char-sequence according to ISO C90 section 6.10.2 “source file inclusion”. The parameter value MUST NOT represent a path, since ISO C90 does not specify how such a path is treated (i.e., this is implementation defined (and additionally depends on the operating system and the underlying file system)). eg: UserInclude.h, “UserInclude.h”, <UserInclude.h>.
[AUTOSAR] Description	Header file name which shall be included by the BswM.

[RTA-BSW] Introduction	The value of this parameter shall be used as h-char-sequence or q-char-sequence according to ISO C90 section 6.10.2 “source file inclusion”. The parameter value MUST NOT represent a path, since ISO C90 does not specify how such a path is treated (i.e., this is implementation defined (and additionally depends on the operating system and the underlying file system)).
[AUTOSAR] Introduction	The value of this parameter shall be used as h-char-sequence or q-char-sequence according to ISO C99. The parameter value shall not represent a path.

4.6.1.68 BswM/BswMGeneral/BswMVersionInfoApi

[AUTOSAR] Default Value	false
[RTA-BSW] Default Value	true

4.7 API Definition

Table 4.97.BswM Supported API functions

`BswM_Init`	Initializes BswM.
`BswM_Deinit`	Deinitializes BswM.
`BswM_GetVersionInfo`	Returns the version information of this module.
`BswM_MainFunction`	Main function of BswM.
`BswM_RequestMode`	Generic function call to request modes.
`BswM_CanSM_CurrentState`	Called by CanSM to indicate its current state.
`BswM_ComM_CurrentMode`	Called by ComM to indicate the current communication mode of a ComM channel.
`BswM_ComM_CurrentPNCMode`	Called by ComM to indicate the current mode of the PNC.
`BswM_ComM_InitiateReset`	Called by ComM to signal a shutdown.
`BswM_Dcm_ApplicationUpdated`	Called by Dcm to report an updated application.
`BswM_Dcm_CommunicationMode_CurrentState`	Called by Dcm to inform BswM about the current state of the communication mode.
`BswM_EcuM_CurrentWakeup`	Called by EcuM to indicate the current state of a wakeup source.
`BswM_EthSM_CurrentState`	Called by EthSM to indicate its current state.
`BswM_FrSM_CurrentState`	Called by FrSM to indicate its current state.
`BswM_J1939DcmBroadcastStatus`	Tells BswM the desired communication status of the available networks.
`BswM_J1939Nm_StateChangeNotification`	Notification of current J1939Nm state after state changes.

BswM_LinSM_CurrentSchedule	Called by LinSM to indicate the currently active schedule table for a specific LIN channel.
BswM_LinSM_CurrentState	Called by LinSM to indicate its current state.
BswM_LinTp_RequestMode	Called by LinTP to request a mode for the corresponding LIN channel.
BswM_Nm_CarWakeUpIndication	Called by Nm to indicate a CarWakeup.
BswM_NvM_CurrentBlockMode	Called by NvM to indicate the current block mode of an NvM block.
BswM_NvM_CurrentJobMode	Called by NvM to inform BswM about the current state of a multi block job.
BswM_Sd_ClientServiceCurrentState	Called by Sd to indicate current state of the Client Service (available/down).
BswM_Sd_ConsumedEventGroupCurrentState	Called by Sd to indicate current status of the Consumed EventGroup (available/down).
BswM_Sd_EventHandlerCurrentState	Called by Sd to indicate current status of the EventHandler (requested/released).
BswM_SoAd_SoConModeChg	Function called by SoAd (Socket Adaptor) to notify state change of a socket connection.

For further information on these functions, please refer to AUTOSAR_SWS_BSWModeManager.pdf

4.8 Configuration Advice

NOTE

Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the “Configuration Generation” section of the RTA-CAR User Guide.

4.8.1 Example Configuration

This step-by-step example illustrates a simple configuration of BswM for a ComM indication. This is just a basic example, but it will help you to understand what is required for a full configuration of BswM.

4.8.1.1 Step 1: Create a new ModeRequestPort

1. Create a new ModeRequestPort.

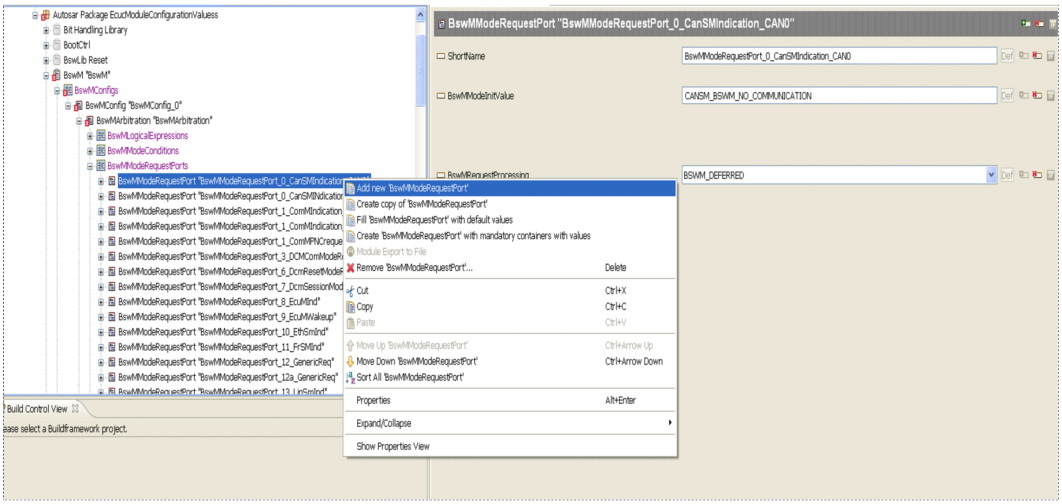


Figure 4.1.Create a new ModeRequestPort

4.8.1.2 Step 2: Configure the ModeRequestPort

1. Give the new ModeRequestPort a unique ShortName.
2. Configure BswMModeInitValue if required. Refer to the ModeDeclarationGroup (of corresponding ModeRequestType) for possible values to this parameter.
3. Configure BswMRequestProcessing.

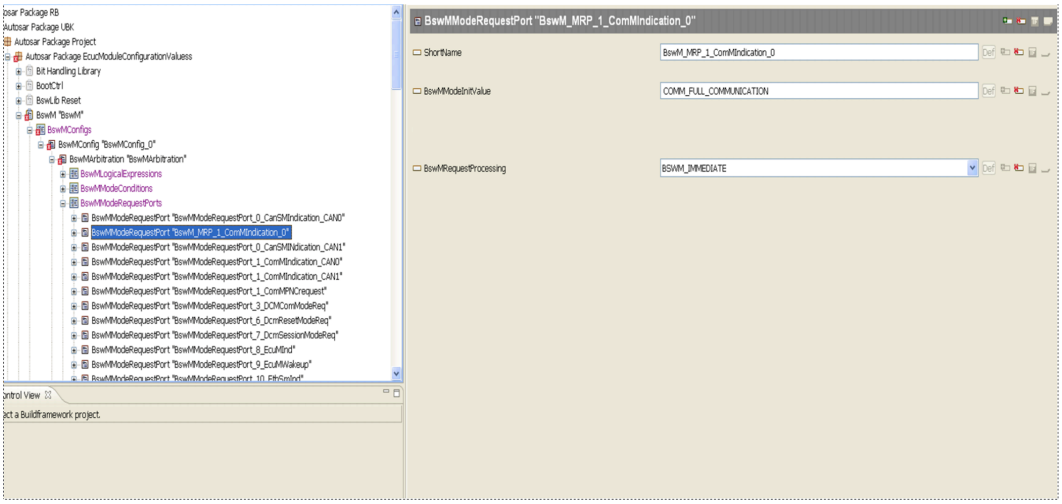


Figure 4.2.Configure the ModeRequestPort

4.8.1.3 Step 3: Configure a ModeRequestSource for the new ModeRequestPort

1. In this example we're configuring BswMComMIndication.

2. Choose a ComMChannel as input for this ModeRequestPort.

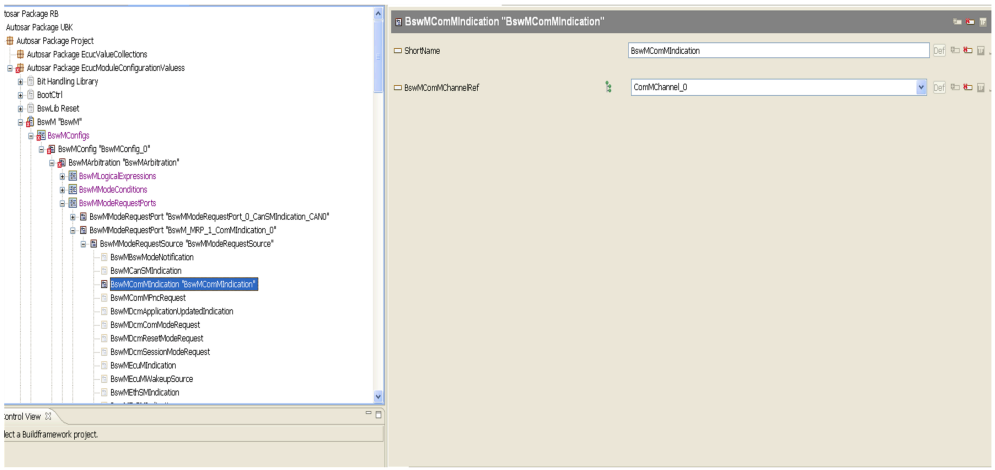


Figure 4.3.Configure a ModeRequestSource for the new ModeRequestPort

4.8.1.4 Step 4: Create a new ModeCondition

1. Give it a unique ShortName.
2. Set the appropriate ConditionType.
3. Set the ConditionMode to refer to the ModeRequestPort that you created in step 1 and configured in step 2.

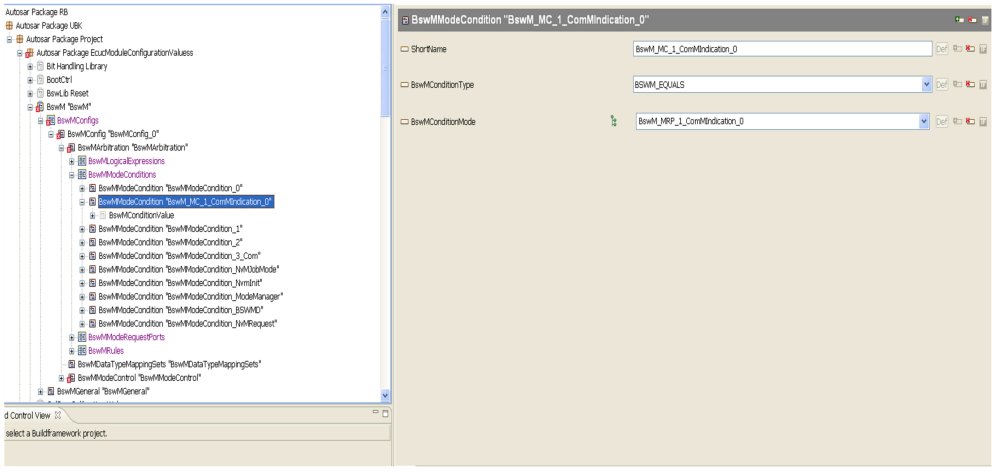


Figure 4.4.Create a new ModeCondition

4.8.1.5 Step 5: Configure the ModeConditionValue

1. The BswMBswModeSourceType parameter is optional (the type is already known from BswMConditionMode).

2. Set the appropriate ModeConditionValue in BswMBswRequestedMode. This is the value that is considered for Mode arbitration (in our examples it's COMM_NO_COMMUNICATION).

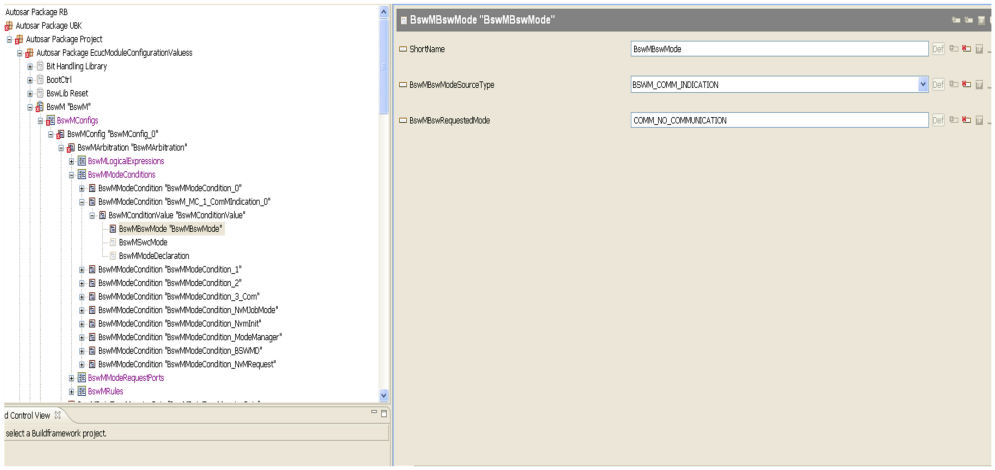


Figure 4.5.Configure the ModeConditionValue

4.8.1.6 Step 6: Create and configure a new LogicalExpression

1. Give the new LogicalExpression a unique ShortName.
2. Set BswMArgumentRef to refer to the ModeCondition that we created in steps 4 and 5. (In this example we're actually referring to two ModeConditions, BswM_MC_1_ComMIndication_0 and BswM_MC_2_ComMIndication_0).
3. The logical operator BSWM_OR (configured in BswMLogicalOperator) is used to combine these ModeConditions.

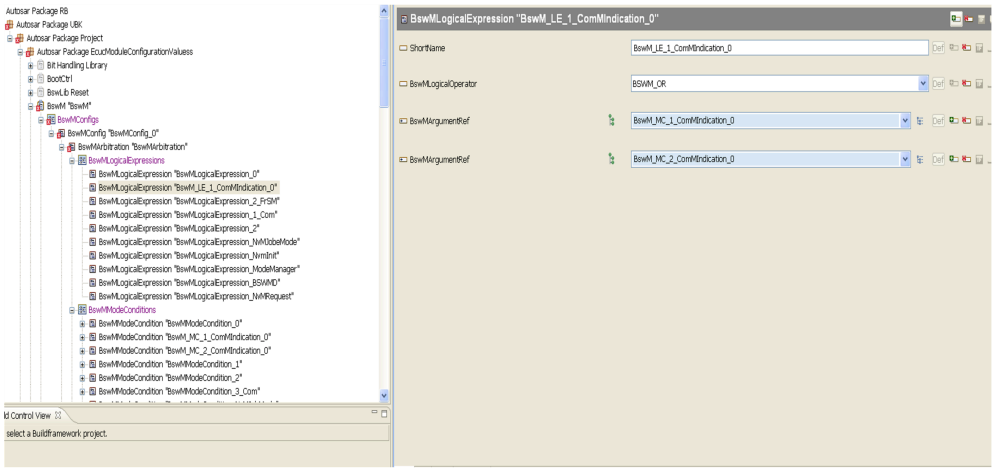


Figure 4.6.Create and configure a new LogicalExpression

4.8.1.7 Step 7: Create and configure a new Action

1. Create a new Action and provide a unique ShortName (which in our example is BswM_AC_1_ComMIndication_0).
2. Select from the list of available BswMAvailbleActions (we've selected BswMUserCallout).
3. Set the BswMUserCalloutFunction parameter to the required action (e.g. fun_comunicate).

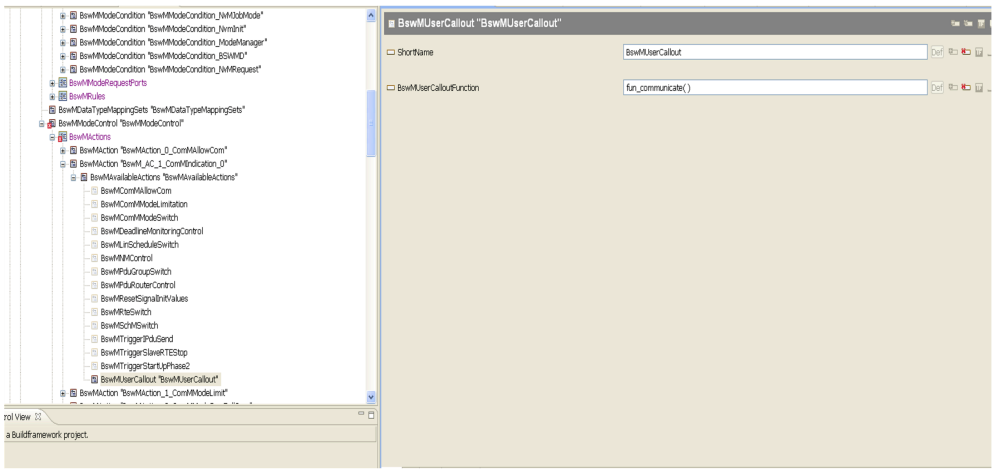


Figure 4.7.Create and configure a new Action

4.8.1.8 Step 8: Create a new ActionList

1. Give the new ActionList a unique ShortName.
2. Choose an appropriate ActionListExecution. If BSWM_CONDITION is chosen, the ActionList is executed every time at the result of the rule evaluation. If BSWM_TRIGGERED is chosen, the ActionList is executed only when the result of the rule evaluation has changed since the last evaluation.

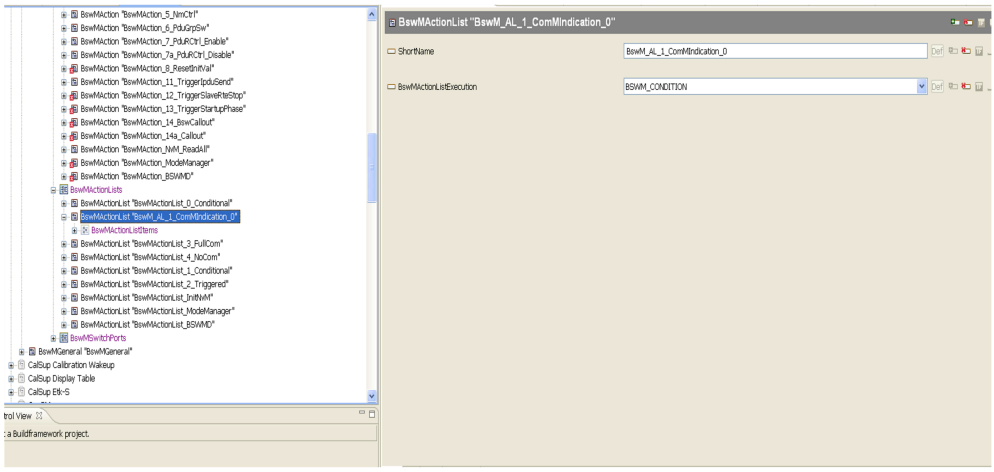


Figure 4.8.Create a new ActionList

4.8.1.9 Step 9: Configure the new ActionList

1. Create a new ActionListItem inside the ActionList that you created in Step 8. (An ActionList can have more than one ActionListItem).
2. Give the new ActionListItem an index (via BswMActionListTemIndex) that is unique within this ActionList.
3. Set BswMActionListItemRef to the Action that needs to be executed (it can also be an ActionList or a Rule). In our example we've selected the Action that we created in Step 7 (BswM_AC_1_ComMIndication_0).
4. The BswMAbortOnFail parameter is used to abort the execution of the ActionList.

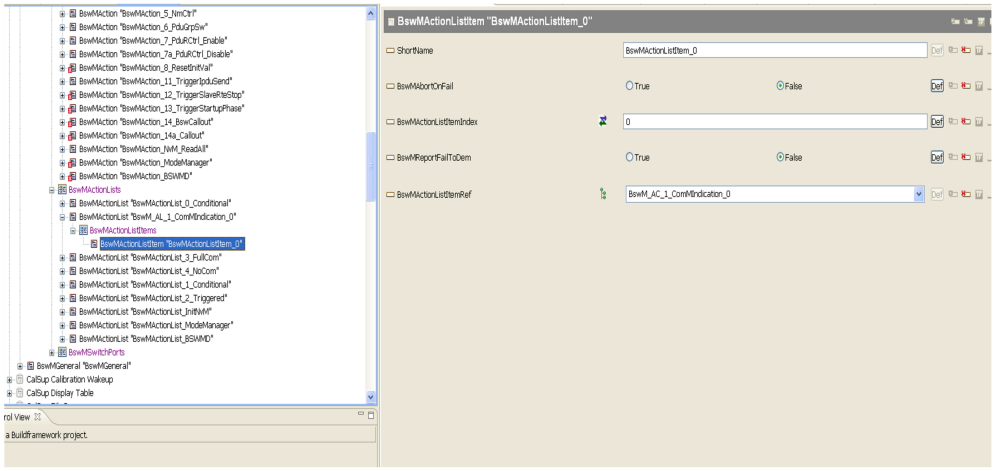


Figure 4.9.Configure the new ActionList

4.8.1.10 Step 10: Create and Configure a new Rule

1. Give the new BswMRule Container a unique ShortName.
2. Set the BswMRuleInitState parameter to the required reset/initialization state (or the “previous evaluation result” of a rule, during initialization/reset) of the BswM arbitration rule.
3. Set BswMRuleExpressionRef to the logical expression to be used for the evaluation. In this example, we’re pointing to BswM_LE_1_ComMIndication_0 that we created in Step 6.
4. Set BswMRuleFalseActionList and BswMRuleTrueActionList to the appropriate ActionLists. In our example we’re setting BswMRuleTrueActionList to the BswM_AL_1_ComMIndication_0 that we created in Step 8.
5. The ActionList’s ActionListExecution that we setup in Step 8 will decide whether it is executed based on condition or triggered (in our example it is conditional).

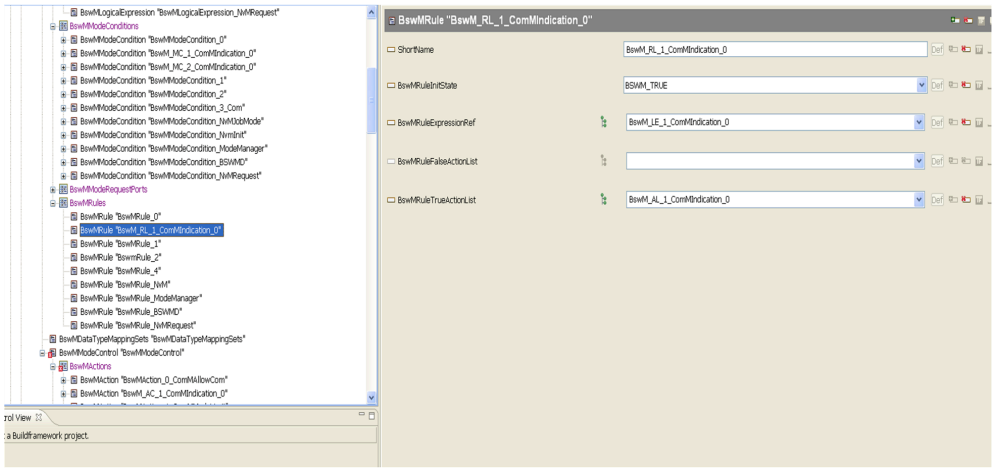


Figure 4.10.Create and Configure a new Rule

> **NOTE**
>
> The BswMRuleTrueActionList and BswMRuleFalseActionList lists all the BswMActionList irrespective of the variant. If the BswMRule of BswMRuleTrueActionList or BswMRuleFalseActionList is specific to a variant then the BswMActionList selection should be applied to the BswMRule variant.

4.8.2 Code Generation based on configuration class

BswM supports 2 kinds of configuration class -

- Pre-compile

- Post-build Loadable

If the Post-Build Loadable option is to be used for BswM, the following two configuration steps must first be completed.

Step 1 Select the Configuration Class

Go to the BswM module configuration, and then the “Properties” tab.

ImplementationConfigVariant will be set to “variant-pre-compile” by default, but if Post-Build Loadable will be used for BswM, select the “variant-post-build” option.

Step 2 EcuM Configuration

If ImplementationConfigVariant is set to “variant-post-build”, the EcuM configuration for BswM must be configured with POSTBUILD_PTR.

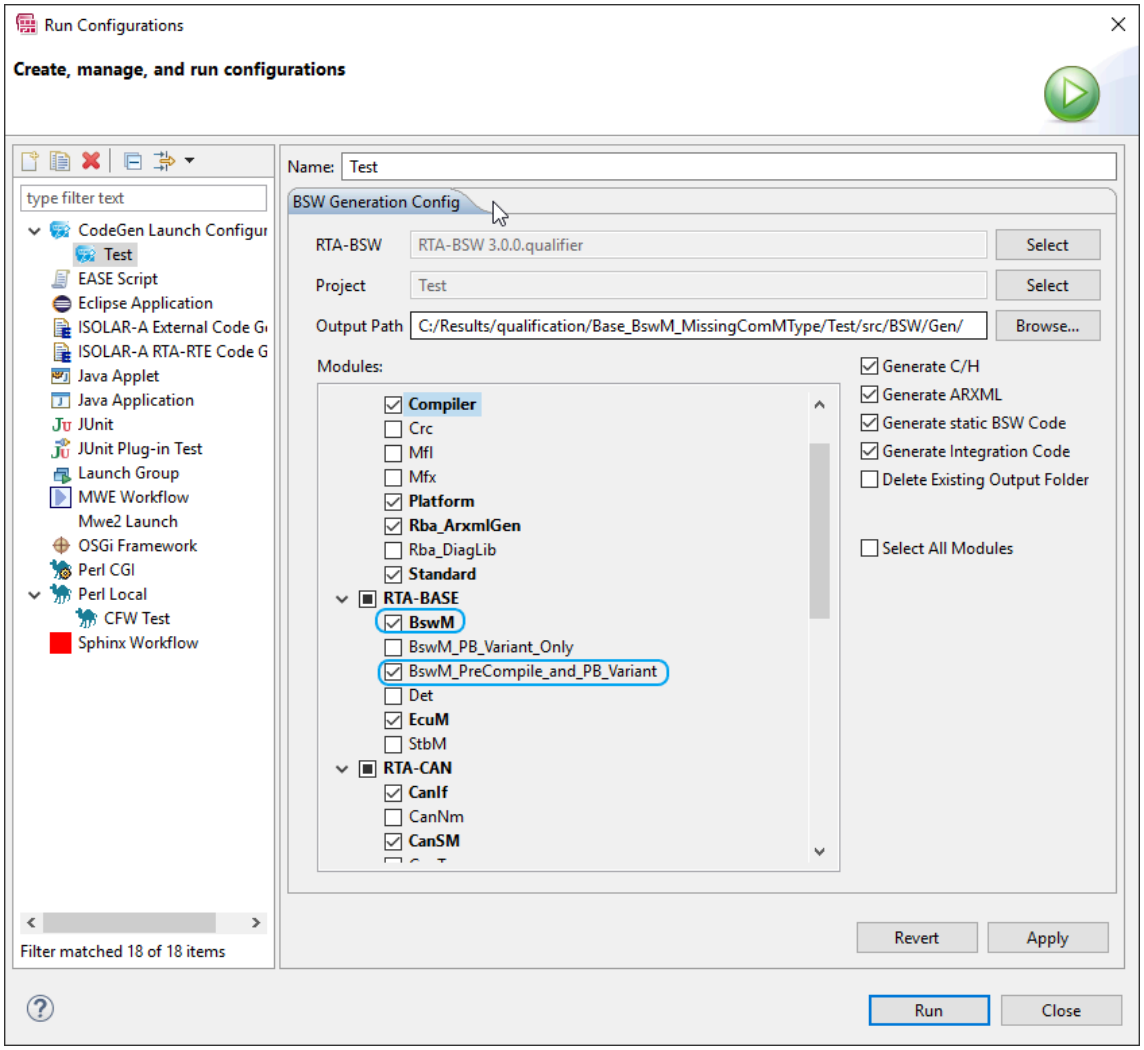
4.8.2.1 Code Generation Options

Depending on the above setting of ImplementationConfigVariant, the following options can be deployed at Code Generation.

Pre-Compile

If this option is chosen, all ECUs using the generated BSW will have the same configuration and there is no way to change this configuration without re-generating and re-compiling the whole of the ECU’s software.

As shown below, select the “BswM_PreCompile_and_PB_Variant” option for the BswM module. The other options on the right can be set at your discretion.

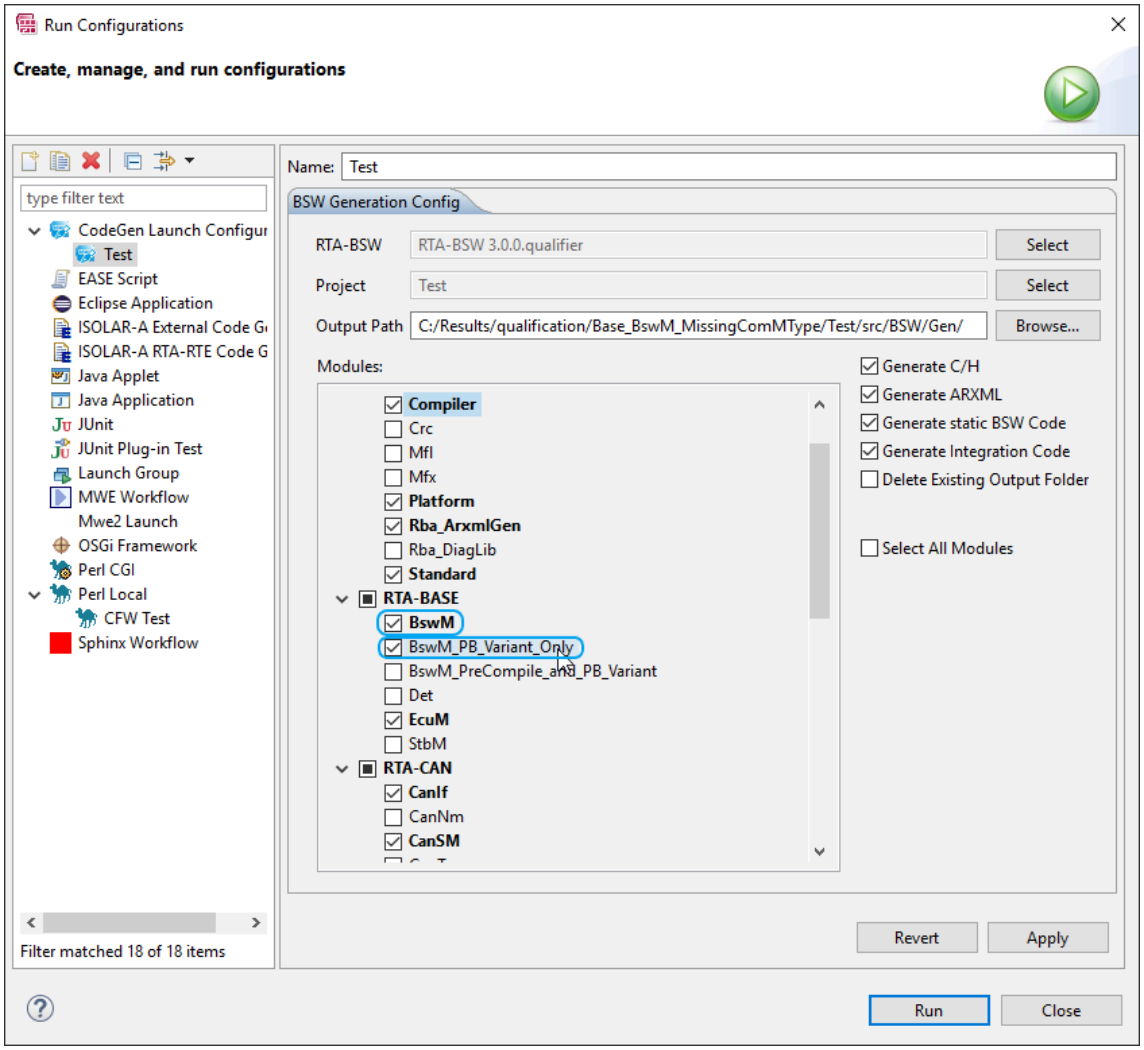


Post-Build Loadable

This option results in the generation of only the configuration for the BswM module, and it does not generate any of the other code.

Post-Build Loadable is only available after the BswM module's main code has initially been fully generated in a Post-Build configurable form, and this is achieved with a one-time Code Generation using the BswM_PreCompile_and_P-B_Variant option, as shown above.

Then, if the module's configuration is subsequently changed, as shown below, the BswM_PB_Variant_Only option can be used so that only the configuration files are re-generated (the main BswM code will not be re-generated).



Note that the “Delete Existing Output Folder” option must be un-ticked. This results in the loadable parts being allocated into a different folder from the full generation that was previously created by the initial full build.

4.8.3 Other Configuration Advice

This final section includes other miscellaneous items of configuration advice.

4.8.3.1 Generic Request for Rte

To make use of this feature, you need to configure the BswMRbRequestType parameter in the BswMGenericRequest container.

BswMRbRequestType accepts either BSW or SWC as input, and if SWC is chosen, BswM will generate the required RTE-related configuration for a Client-Server Interface in its SWCD file. In this context, BswM acts as a server, and any SWC module can request a service. The Client-Server interface defines a Client-Server operation which is invoked on an Operation-Invoked event.

BswM doesn't create any new runnables. Instead it re-uses the existing BswM_RequestMode API, which acts as a runnable entity. When a SWC requests a service, this API is called directly by RTE as a result of the Operation-Invoked event. The SWC makes a call to Rte_Call_<r-port>_<client-server- operation> and then BswM_RequestMode is called.

4.8.3.2 User Callout for Rte

To make use of this feature, you need to configure a BswMRbSwcUserCallout container in BswMAvailableActions.

The parameter BswMRbSwcClientServerOperationRef is used to refer to the Client-Server Operation. BswM uses this information to generate the required RTE-related configuration for the Client-Server Interface in its SWCD file. BswM acts as a client in this case.

BswM performs an Rte_Call_<r-port>_<client-server- operation> as an action, and then the runnable entity or user callout corresponding to the Client-Server Operation is called by RTE. The arguments passed in the callout are configured using the BswMRbSwcUserCalloutArgument parameter in the BswMRbSwcUser-Callout container.

4.8.3.3 BswMGenericRequest

The ModeRequestSource of BswMGenericRequest is for requests that originate from a requestor that is not among the list of standardized mode requestors (i.e. the different resource managers).

BswMGenericRequest contains the parameter BswMRequestedModeMax, which defines the upper limit for the modes requested by each mode requester. For example, for a range from 0-10, this parameter shall be set to 10. It is recommended that the Mode User defines the modes with range starting from 0 and then contiguous numbers. The Det check from BswM may not work properly otherwise.

4.8.3.4 BswMSwcModeRequest

The ModeRequestSource of BswMSwcModeRequest is for requests whose source is an SW component. The SWC module sends a Mode request to BswM using BswMSwcModeRequest, and BswM performs a mode switch as a result of an Rte_Switch action.

BswMSwcModeRequest contains the parameter BswMRbModeRequestQueue-Size, which specifies whether the mode communication is queued or not.

- Queued communication - When a queue length is configured with a non-zero value, and a buffer is maintained to read the Modes.
- UnQueued communication - When queue length is not configured or configured with zero, Modes are directly read.

4.8.3.5 BswMRbIntrptQueueMaxSize

The Supported Features section describes scenarios in which requests are inter-

rupted.These requests are stored in a queue, the maximum size of which is determined by the `BswMRbIntrptQueueMaxSize` configuration parameter in the `BswMGeneral` container.

Hint There is no formula, as such, for calculating the maximum queue size. As a guide, add an extra 30 per cent to the buffer value, ensuring it falls within the max. range of 255, and revise as necessary.

The parameter `BswMRbDebugEnable` has been introduced for version v12.7.0. Enabling it keeps a record of the number of times an interrupted request is missed i.e. that the queue size is too small.

When executing a normal request, if a new request from one of the other BSW modules arrives, the former is stored in the queue. If the queue size is too small, `BswMRbDebugEnable` records the missed requests, incrementing by 1.

It is the responsibility of the integrator to configure an appropriate value for this parameter. Choosing a random value may lead to system misbehavior or wastage of memory resources.

4.8.3.6 BswMDcmApplicationUpdatedIndication

If a ModeRequestPort is configured with the source as BswMDcmApplicationUpdatedIndication, then BswMBswRequestedMode should be configured as TRUE or FALSE in the corresponding ModeCondition.

Setting BswMBswRequestedMode to TRUE means that the BswM_Dcm_ApplicationUpdated API is called. Setting it as FALSE means that the API is not called.

4.8.3.7 BswMMainFunctionPeriod

BswMMainFunctionPeriod should always be configured using a float value.

4.8.3.8 BswM Exclusive Areas

The BswM exclusive areas can be defined via BswM or RTE, and then by enabling or disabling EcuCRbLegacySchMInUse.

4.9 Integration Advice

4.9.1 Init Functions

BswM is initialized by a call from EcuM to BswM_Init.

4.9.2 DeInit Functions

The BswM module is deinitialized by BswM_DeInit, which is called by EcuM just before shutting down the OS.

4.9.3 Scheduled Functions

BswM_MainFunction should be called cyclically, the period of which is controlled by BswMMainFunctionPeriod.

4.9.4 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The following is a list of all exclusive areas that this module might use regardless of the user configuration. It is the job of the integrator to ensure that the implementation of these exclusive areas is correctly defined. This implementation can be done using the RTE exclusive area configuration wizard or by manual implementation of the corresponding BSW API calls to ensure exclusivity.

The BswM module uses the following OS resources:

- BswM
- BswM

4.10 Hardware Requirements

Table 4.98.BswM Hardware Requirements

Support	Usage
FLASH	8404 bytes
RAM	1681 bytes

Usage calculated based on a sample configuration.

5 Time Services

Module name	Tm
Package	RTA-BASE
AUTOSAR Module ID	14
AUTOSAR Version	22-11

5.1 Introduction

The Time Service module provides services for time based functionality. Use cases are:

- Time Measurement
- Time Based State Machine
- Timeout Supervision
- Busy Waiting

The Time Service module is part of the System Services Layer. All layers interacting with the Services Layer may use the Time Service. The services of the Time Service module are not accessible by AUTOSAR Software Components which are located above the RTE.

The functionality of the Time Service module is based on hardware timers provided by (General Purpose Timer) GPT Driver.

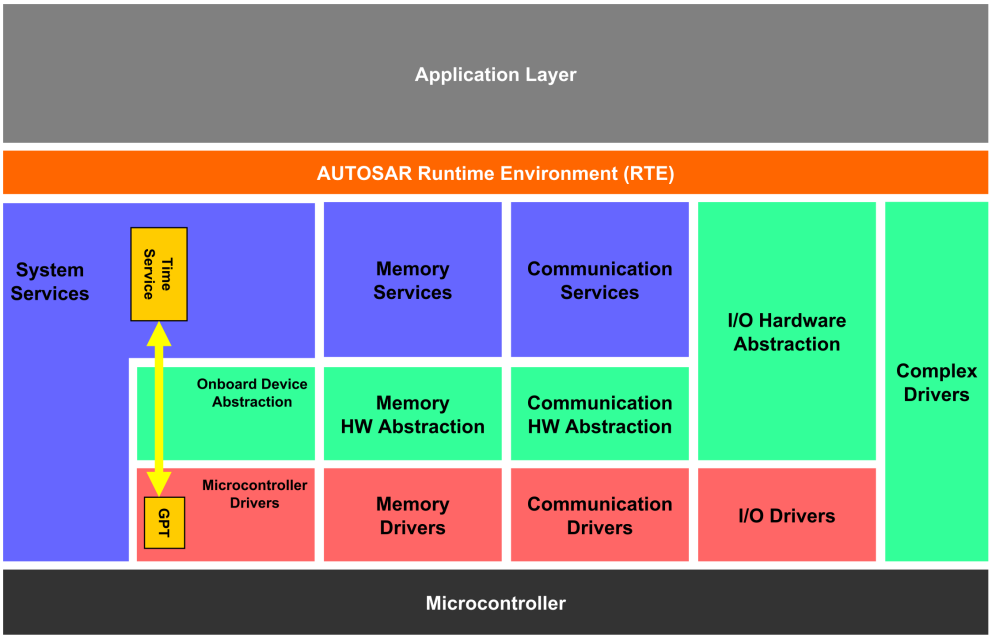


Figure 5.1.Architectural Overview

Module Configuration

This module is not currently supported by the RTA-BSW Configuration Generator (ConfGen) and must be manually configured.

5.2 Supported Features

The Time Service module offers a set of services which are available for individual timer types. The following sections describe the services and the timer types.

5.2.1 Timer Types

The following Time Service Predef Timer types are defined (note: The Time Service Predef Timer types correspond exactly to the GPT timer types which also define 1us16bit, 1us24bit, 1us32bit and 100us32bit timer types):

- Tm_PredefTimer1us16bitType
- Tm_PredefTimer1us24bitType
- Tm_PredefTimer1us32bitType
- Tm_PredefTimer100us32bitType

Each Predef Timer has a predefined tick duration(1µs or 100µs) and a number of bits which represent the physical range of the timer values. A 32bit timer type has a longer time span until wrap-around than 16bit or 24bit timers. The availability of the timer types depend on the hardware and the GPT driver configuration.

5.2.2 Characteristics of Time Service Predef Timer Types

Data Type	Tick Duration	Max. Tick Value	Number of Bits	Max. Time Span (ca.)
Tm_PredefTimer1us16bitType	1µs	65536	16	65 ms
Tm_PredefTimer1us24bitType	1µs	16777215	24	16 s
Tm_PredefTimer1us32bitType	1µs	4294967296	32	71 min
Tm_PredefTimer100us32bitType	100µs	4294967296	32	4.9 days

5.2.3 Timer Instance Variables

The Time Service services useinstancesof timer types which are local (stack) or global variables. Such an instance of a timer type stores areference timevalue which is a snapshot time value of the underlaying GPT Predef timer. The Time Service services use the timer instances for initialization, calculation and busy wait functionality.

5.2.4 Services

Following services are provided by the Time Service module. For a detailed description see next chapter.

Service Name	Description
ResetTimer	Initialize the reference time of a timer instance with the GPT timer value.
GetTimeSpan	Calculate the difference between two given timer instances and return the value in timer ticks (1µs or 100µs depending on the timer type).
ShiftTimer	Shift the reference time of a given timer instance by a given value.
SyncTimer	Set the reference time of a destination timer instance to the reference time value of a source timer instance.
BusyWait	Busy wait (active spinning) a given time. BusyWait uses timer instances, ResetTimer and GetTimeSpan services internally. Note: BusyWait service is only available for 1us16bit, 1us24bit and 1us32bit timer types, not for 100us32bit type.

5.3 Extensions

5.3.1 Configuration Extensions

The following configuration parameters are available in version v12.7.0 in addition to those defined in AR v22-11

5.3.1.1 Tm/TmGeneral/TmBusyWaitLoopSnooze

Short Name	TmBusyWaitLoopSnooze
Description	Switches on or off adding a C macro “TM_BUSYWAIT_LOOP_SNOOZE_OPERATION” to the active waiting loop implementation of BusyWait services. This C macro must be populated by the user in Tm_Ext.h. ON or OFF. This parameter is an extension to the standard Tm.
Multiplicity	1..1
Default Value	false
Type	ECUC-BOOLEAN-PARAM-DEF

5.3.2 Additional Services

The following API are available in version v12.7.0 in addition to those defined in AR v22-11

5.4 Limitations

5.5 Exclusions

The following features from AR v22-11 are not supported in version v12.7.0.

The following configuration parameters from AR v22-11 are not supported in version v12.7.0.

- There are no known exclusions in this version.

5.6 Deviations

5.6.1 Configuration Deviations

Configuration parameters which differ from the AR v22-11 specification are listed below.

5.6.1.1 Tm

[RTA-BSW] Description	The Time Service module (Tm) provides services for time based functionality. Use cases are: - Time measurement - Time based state machine - Timeout supervision - Busy waiting
[AUTOSAR] Description	Configuration of the Time Service module.

5.6.1.2 Tm/TmGeneral/TmEnablePredefTimer100us32bit

[RTA-BSW] Description	Specifies if the Predef Timer 100us32bit shall be enabled (functionality and set of API services). ON or OFF.
[AUTOSAR] Description	Specifies if the Predef Timer 100µs32bit shall be enabled (functionality and set of API services). ON or OFF.

5.6.1.3 Tm/TmGeneral/TmEnablePredefTimer1us16bit

[RTA-BSW] Description	Specifies if the Predef Timer 1us16bit shall be enabled (functionality and set of API services). ON or OFF.
[AUTOSAR] Description	Specifies if the Predef Timer 1µs16bit shall be enabled (functionality and set of API services). ON or OFF.

5.6.1.4 Tm/TmGeneral/TmEnablePredefTimer1us24bit

[RTA-BSW] Description	Specifies if the Predef Timer 1us24bit shall be enabled (functionality and set of API services). ON or OFF.
[AUTOSAR] Description	Specifies if the Predef Timer 1µs24bit shall be enabled (functionality and set of API services). ON or OFF.

5.6.1.5 Tm/TmGeneral/TmEnablePredefTimer1us32bit

[RTA-BSW] Description	Specifies if the Predef Timer 1us32bit shall be enabled (functionality and set of API services). ON or OFF.
[AUTOSAR] Description	Specifies if the Predef Timer 1µs32bit shall be enabled (functionality and set of API services). ON or OFF.

5.6.1.6 Tm/TmGeneral/TmVersionInfoApi

[REMOVED] Default Value	false

5.7 API Definition

Table 5.11.Tm Supported API functions

`Tm_GetVersionInfo`	Returns the version information of this module.
`Tm_ResetTimer1us16bit`	Resets a timer instance (user point of view).
`Tm_GetTimeSpan1us16bit`	Delivers the time difference (current time - reference time).
`Tm_ShiftTimer1us16bit`	Shifts the reference time of the timer instance.
`Tm_SyncTimer1us16bit`	Synchronizes two timer instances.
`Tm_BusyWait1us16bit`	Performs busy waiting by polling with a guaranteed minimum waiting time.
`Tm_ResetTimer1us24bit`	Resets a timer instance (user point of view).
`Tm_GetTimeSpan1us24bit`	Delivers the time difference (current time - reference time).
`Tm_ShiftTimer1us24bit`	Shifts the reference time of the timer instance.
`Tm_SyncTimer1us24bit`	Synchronizes two timer instances.
`Tm_BusyWait1us24bit`	Performs busy waiting by polling with a guaranteed minimum waiting time.
`Tm_ResetTimer1us32bit`	Resets a timer instance (user point of view).
`Tm_GetTimeSpan1us32bit`	Delivers the time difference (current time - reference time).
`Tm_ShiftTimer1us32bit`	Shifts the reference time of the timer instance.
`Tm_SyncTimer1us32bit`	Synchronizes two timer instances.
`Tm_BusyWait1us32bit`	Performs busy waiting by polling with a guaranteed minimum waiting time.
`Tm_ResetTimer100us32bit`	Resets a timer instance (user point of view).
`Tm_GetTimeSpan100us32bit`	Delivers the time difference (current time - reference time).
`Tm_ShiftTimer100us32bit`	Shifts the reference time of the timer instance.
`Tm_SyncTimer100us32bit`	Synchronizes two timer instances.

For further information on these functions, please refer to AUTOSAR_SWS_Time-Service.pdf

5.8 Configuration Advice

> **NOTE**
>
> Before configuring this module, please note that some of the default values used by ConfGen to produce the default ECU Configuration can be changed, and this is done by either adding a Conf-Gen Enhancer Mapping (CEM) or by adding a rule to an *algo.properties* file. For more information, see the "Configuration Generation" section of the RTA-CAR User Guide.

5.8.1 Error Handling

In the Tm Service module all services are of one of the following return types:

- void
- Std_ReturnType

All Tm services with a void return type such as GetVersionInfo, ShiftTimer and SyncTimer will return immediately if an error of any kind occurs. As these services are exclusively read/modify timer instance/version information variables in memory the only actions omitted are writing data to structures.

The remaining services ResetTimer, GetTimeSpan and BusyWait utilize the GPT service *Gpt_GetPredefTimerValue()* which is dependent on a hardware timer. If *Gpt_GetPredefTimerValue()* returns a status different from E_OK, the effected Tm service will return E_NOT_OK and the service will abort immediately. This especially effects the service BusyWait which has multiple calls to *Gpt_GetPredefTimerValue()*, and this service returns after the first failed call to *Gpt_GetPredefTimerValue()* immediately.

If the Default Error Tracer (Det) is enabled for Tm, all Tm services utilize the Det service *Det_ReportError()* to report the following error information:

TM_E_PARAM_POINTER If a service parameter pointer is a NULL pointer.

TM_E_PARAM_VALUE If a service parameter value is invalid.

All Tm services utilize Det service *Det_ReportRuntimeError()* to report following error information:

TM_E_HARDWARE_TIMER If *Gpt_GetPredefTimerValue()* returned a status different from E_OK.

5.8.2 ResetTimer Service

The ResetTimer service resets a timer instance from the users point of view. It initializes the reference time of the timer instance variable with the value obtained from the respective GPT Timer.

When successful the ResetTimer returns E_OK, if there are parameter errors or GPT errors, then the ResetTimer returns E_NOT_OK.

If the underlying GPT timer returns an error, then the ResetTimer will raise the

TM_E_HARDWARE_ERROR.

If the default error detection for the Time Service Module is enabled:

- If the timer instance parameter is NULL, the ResetTimer will raise the TM_E_PARAM_POINTER.

5.8.3 GetTimeSpan Service

The GetTimeSpan service calculates the time difference between a given timer instance and the current GPT timer time (current time - reference time) and delivers the time span value result. If the current time value is less than the reference time value, then the GetTimeSpan functions will perform a proper wrap-around handling.

When successful the GetTimeSpan functions return E_OK, if there are parameter errors or GPT errors, then the GetTimeSpan functions return E_NOT_OK. In case of any error including parameter error, it will set the resulting time span to “0” if the respective memory pointer is not NULL.

When the underlying GPT timer returns an error, then the GetTimeSpan will raise the TM_E_HARDWARE_ERROR.

If the default error detection for the Time Service Module is enabled:

- If any parameter is NULL, then the GetTimeSpan will raise the TM_E_-PARAM_POINTER.

Usage notes:

The accuracy of the GetTimeSpan is +/- 1 tick, and a measured time span value of 1 µs tick may come from an actual time difference between 0..2 µs. If a GetTimeSpan function is used to check a guaranteed minimum time, e.g. for: Timeout supervision, then busy waiting n+1 ticks must be observed by user software to ensure that an interval of at least n ticks has passed. Refer to the Time Quantization Error Example:

Time Quantization Error Example

In the following diagram there are three examples for measurements performed on the “actual time” scale (x-axis). Each measurement is performed by calls to ResetTimer and GetTimeSpan functions based on a 1 µs timer. The results of GetTimeSpan are mapped on the “quantized time” scale (y-axis) which is quantized by the underlying GPT timer.

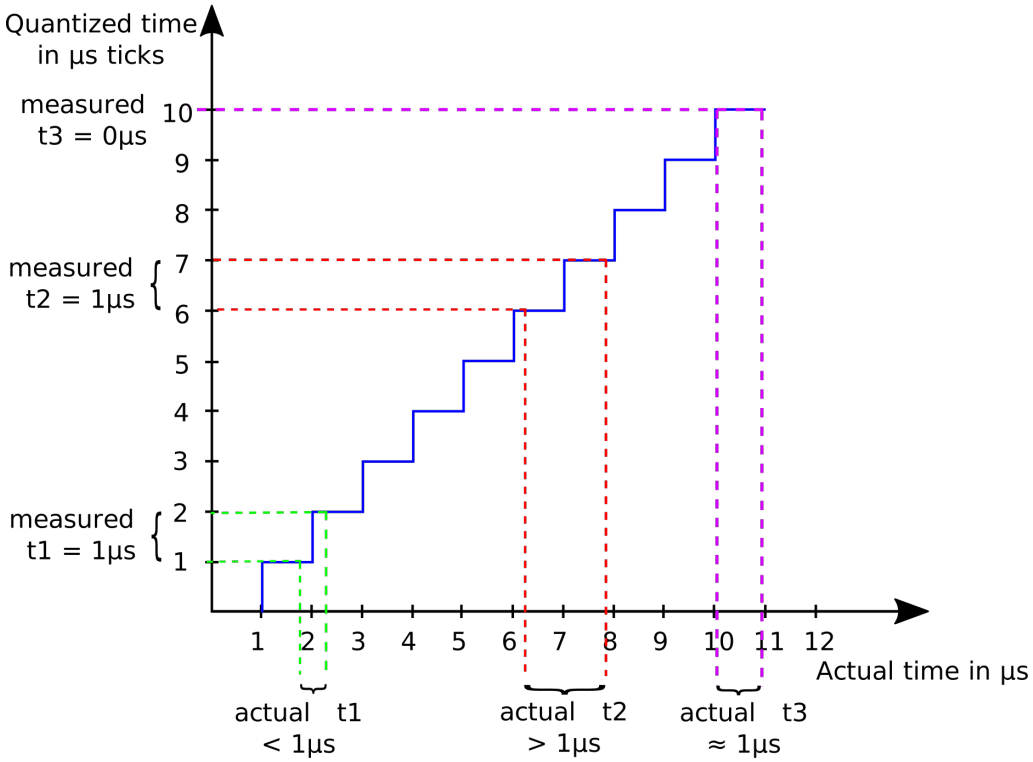


Figure 5.2.Time Quantization Error

The first time span measurement Δt1 defines an actual time delta of less than 1 µs, but contains a GPT timer tick change from “1” to “2” on the GPT time scale. GetTimeSpan returns 1 µs difference because of the GPT timer tick change, but actually less than 1 µs has been passed. In the second measurement, Δt2 results in the same timer tick difference of 1 µs returned by GetTimeSpan, although more than 1 µs has passed in the real time. The third time span Δt3 is less than, but almost 1 µs on the actual time scale. Because there was no change in the GPT time scale during that time span, the resulting time span measured is 0 µs.

5.8.4 ShiftTimer Service

The ShiftTimer functions shift the reference time of a given timer instance by a given value. This means, a given TimeValue is added to the reference time of the timer instance. If the sum of TimeValue and reference time exceeds the timer value range, a proper wrap-around handling is performed.

The ShiftTimer function of the 24bit Timer accepts only TimeValues in the 24bit range 0..0x00FFFFFFu.

If the default error detection for the Time Service Module is enabled:

- When the timer instance parameter is NULL, the ShiftTimer will raise the TM_E_PARAM_POINTER.
- In the case of the 24bit timer: If the TimeValue parameter is larger than 0x00FFFFFFu, the ShiftTimer will raise the TM_E_PARAM_VALUE.

5.8.5 SyncTimer Service

The SyncTimer functions synchronize two timer instances by setting the reference time of the given destination timer instance with the reference time of the given source timer instance.

If the default error detection for the Time Service Module is enabled:

- If any timer instance parameter is NULL, the SyncTimer will raise the TM_E_PARAM_POINTER.

5.8.6 BusyWait Service

The BusyWait service performs a busy waiting by polling with a guaranteed minimum waiting time. There are only implementations available for timers with a 1 µs tick. The WaitingTime parameter is of uint8 type, and therefore the busy waiting time is limited to 0..255us. The BusyWait is implemented using the ResetTimer and GetTimeSpan services.

When successfull, BusyWait returns E_OK. If the underlying GPT timer returns an error, then BusyWait returns E_NOT_OK and aborts waiting immediately.

If the underlying GPT timer returns an error, then the BusyWait will raise the TM_E_HARDWARE_ERROR. Note that BusyWait utilizes ResetTimer and GetTimeSpan services, and the TM_E_HARDWARE_ERROR is raised by those functions in case of such an error.

Usage notes:

The underlying GetTimeSpan service is affected by time quantization errors. BusyWait guarantees a minimum waiting time: It adds 1 tick (=1 µs) to the specified waiting time passed by the parameter in order to shift the resulting effective quantization latency to +0..2 ticks instead of -1..+1 ticks. So even if the user passes “0” as WaitingTime parameter to BusyWait, it will wait until the underlying GPT timer tick will change at least by 1. See GetTimeSpan service description for more details for the nature of quantization errors.

Because BusyWait is neither disabling nor using interrupts, the resulting actual waiting time may be even larger than given waiting time + quantization latency, if BusyWait gets interrupted by ISRs/tasks with higher priority and the interruption consumes substantial time. Note: The user is responsible for avoiding unintentional interruption of BusyWait.

Advanced usage notes:

The implementation of busy waiting may cause unintended load on internal microcontroller bus(es) when the main waiting loop is constantly checking for the exit criteria (minimal waiting is over) without a break. The loading of this could interfere with other processes requiring the same internal microcontroller bus(es).

In order to reduce such unintended load the BusyWait service implementation provides an option for the user to add code to the main waiting loop of BusyWait services. This code would be executed at each waiting cycle of the main waiting

loop in the BusyWait service.In order to use this option, the user has do the following:

- Enable the configuration option TmBusyWaitLoopSnooze.
- Add a file “Tm_Ext.h” to the project which defines the C macro TM_BUSY-WAIT_LOOP_SNOOZE_OPERATION with the code the user wants to add to the BusyWait service main loop. Refer to the BusyWait Loop Snoozing example in the Use Cases section.

Note: Unless explicitely needed, it is recommended to switch TmBusyWaitLoop-Snooze OFF.

5.8.7 General

There are some constraints which have to be considered by the user when using Time Service functions:

- Time Service functions do not use interrupts nor disable interrupts.

This means for all Time Services that they are interruptable, and for the BusyWait service this may cause the real waiting time to be greater than the desired waiting time. Refer to the BusyWait description for details.

- Time Service functions handle counter wrap-arounds transparently.

The services BusyWait, GetTimeSpan and ShiftTimer handle timer value wrap-arounds correctly so that the resulting value is always valid. With no further actions required by the user.

- Consideration should be made to the maximum time span supported by the used timer (16bit, 24bit, 32bit) when using/measuring time spans. For example GetTimeSpan cannot calculate a time span which is larger than the total time span, as its timer range covers (e.g. 65ms for 16bit timers).

5.8.8 Use Cases

Time Measurement Example Code:

```
#include "Os.h"
#include "Tm.h"

TASK(Task10ms)
{
   Tm_PredefTimer1us32bit Timer1; /* timer instance */
   uint32 TimeUs_u32;

   (void)Tm_ResetTimer1us32bit(&Timer1); /* init timer instance with current time */

   /* consume some time */

   (void)Tm_GetTimeSpan1us32bit(&Timer1, &TimeUs_u32); /* get time span: current - Timer1 */
   if (TimeUs_u32 > MY_CRAZY_THRESHOLD_US)
   {
   /* do something crazy */
   }
}
```

Time Supervision Example Code:

```
#include "Register.h"
#include "Tm.h"

#define TIMEOUT_US 30 /* 30 us timeout */

void ExampleFunction(void)
{
   Tm_PredefTimer1us24bitType Timer1;  /* timer instance */
   uint16 StatusRegisterBit0_u16;
   uint32 TimeElapsed_u32;

   (void)Tm_ResetTimer1us24bit(&Timer1); /* init timer instance with current time */

   do {
     StatusRegisterBit0_u16 = HW_SOME_REG&0x0001u;/* get time span in [us] relative to Timer1 (-/+ 1us
accuracy!) */
     (void)Tm_GetTimeSpan1us24bit(&Timer1,&TimeElapsed_u32);
   } /* wait for bit 0, timeout 30us  */
   while( (StatusRegisterBit0_u16 != 0x0001u) &&
        (TimeElapsed_u32<= TIMEOUT_US+1) ); /* use 30+1 to ensure that 30us are guaranteed */
}
```

Busy Waiting Example Code:

```
#include "Tm.h"

void NotifyAckSignal(void)
{
  SetAckPinHigh(); /* set Acknowledge pin high */
  (void)Tm_BusyWait1us32bit(50); /* Busy wait at least 50us */
  SetAckPinLow(); /* set Acknowledge pin low */
}
```

BusyWait Loop Snoozing Example Code:

```
/* Tm_Ext.h */
#ifndef TM_EXT_H
#define TM_EXT_H

/* BusyWait loop snooze operation */
#if defined(__ghs__) && defined(__RH850__)
/* Example for RH850 using GHS compiler */
#define TM_BUSYWAIT_LOOP_SNOOZE_OPERATION asm("SNOOZE")
#else
/* leave empty */
#define TM_BUSYWAIT_LOOP_SNOOZE_OPERATION
#endif

#endif /* TM_EXT_H */
```

5.9 Integration Advice

5.9.1 Components & Dependencies

The functionality of the Time Service module is based on General Purpose Timers (GPT) "GPT Predef Timers". A GPT Predef Timer is a free running increment counter provided by the GPT driver. For each Time Service Predef Timer type the corresponding GPT Predef Timer must be enabled as well. Otherwise calls to Time Service functions will fail and TM_E_HARDWARE_ERROR will be raised, and therefore it is recommended to enable all GPT Predef Timers.

5.9.2 Integration Notes

Note: The integration of the Time Service module only requires a nominal amount of configuration settings. Its services can only be used on System Services layer and not in/or above the RTE.

Configuration:

- Make sure that the GPT driver module is available in MCAL and supports GPT Predef timers (Predef timers are available from Autosar 4.1.1 onwards).
- Configure the desired timer type (1us 16bit, 1us 24bit, 1us 32bit, 100us 32bit) in GPT. It is recommended that all timer types are enabled.
- Enable desired timer(s) in Time Service component configuration.

Code:

- Using the services, in each unit using the Tm functions, include "Tm.h".
- Take care that GPT initialization has to be performed in advance of using Tm services.

Safety Notice:

When using the BusyWait service in connection with a safety function:

- Note: The BusyWait service requires an underlying GPT timer that always advances its tick value with progressing time. If a GPT timer on which a Busy-Wait service depends would stop advancing, then the BusyWait service would be permenantly waiting because its exit condition would never be true. For this reason a safety function using the BusyWait service must handle this risk, a possibility to detect a stalling-GPT-timer situation would be a periodic check that the GPT timer is always advancing.

5.9.2.1 Time Service Initialization

There is no Tm Service module specific initialization required as there are no Tm Service objects which have to be initialized. The functionality of Time Service is based on the GPT Predef Timers. Any GPT driver initialization, if required, has to be performed in advance of using Tm Service functions.

5.9.2.2 Time Service De-initialization

There is no Tm Service module specific shutdown function. The functionality of Time Service is based on the GPT Predef Timers. If GPT driver functionality is available in shutdown phase, all Time Serivce services are available as well. If there is a GPT shutdown functionality and Tm Service functions are used after GPT shutdown, Tm Service functions will return errors.

5.9.2.3 OS Resources

The number of the active OS resources, and their usage, depends on the user configuration of the AUTOSAR System and Basic Software. It is the user's responsibility to ensure that the `ENTER` and `EXIT` resource API are implemented to disable/enable interrupts as required. Alternatively they must be configured in the OS with the correct task/ISR allocation.

The Tm module does not use any exclusive OS resources.

5.10 Hardware Requirements

Information relating to the hardware resources required by this module is not currently available.

6 Glossary

BSW
Basic Software

BswM
AUTOSAR BSW Mode Manager module

CAN
Controller Area Network

ComM
AUTOSAR Communication Manager module

Dem
AUTOSAR Diagnostic Event Manager module

Det
AUTOSAR Default Error Tracer module

Dlt Diagnostic Log and Trace

EcuM
AUTOSAR ECU State Manager module

EthIf
AUTOSAR Ethernet Interface module

EthTSyn
AUTOSAR Time Synchronization over Ethernet module

EVM
Error Vector Magnitude

GPT
The General Purpose Timer module

ISR Interrupt Service Routine

NvM
AUTOSAR NVRAM Manager module

RTE
AUTOSAR Run-Time Environment

StbM
AUTOSAR Synchronized Time Base Manager module

SWC/SW-C
Software Component

Tm AUTOSAR Time Service module